z/OS
3.1

MVS Initialization and Tuning Guide

SA23-1379-60

Note

Before using this information and the product it supports, read the information in “Notices” on page 97.

This edition applies to IBM® z/OS® 3.1 (5655-ZOS) and to all subsequent releases and modifications until otherwise indicated in new editions.

Last updated: 2024-07-22

Contents

Figures

Tables

About this information

This information is a preliminary tuning guide for the MVS™ element of z/OS. The information describes how to initialize the system and how to get improved system performance.

This information is a companion to the *z/OS MVS Initialization and Tuning Reference*.

You can find more information about system capacity planning best practices, techniques, and tooling at IBM Techdocs (www.ibm.com/support/techdocs/atsmastr.nsf/Web/TechDocs).

For information about how to install the software products that are necessary to run z/OS, see *z/OS Planning for Installation*.

Who should use this information

This information is for anyone whose job includes designing and planning to meet installation needs based on system workload, resources, and requirements. For that audience, the information is intended as a guide to what to do to implement installation policies.

The information is also for anyone who tunes the system. This person must be able to determine where the system needs adjustment, to understand the effects of changing the system parameters, and to determine what changes to the system parameters will bring about the desired effect.

z/OS information

This information explains how z/OS references information in other documents and on the web.

When possible, this information uses cross-document links that go directly to the topic in reference using shortened versions of the document title. For complete titles and order numbers of the documents for all products that are part of z/OS, see *z/OS Information Roadmap*.

To find the complete z/OS library, go to IBM Documentation (www.ibm.com/docs/en/zos).

Learning resources: Getting started with IBM zSystems (developer.ibm.com/learningpaths/get-started-ibmz/) provides online learning about IBM Z® basics and Red Hat® Open Shift on IBM Z.

How to provide feedback to IBM

We welcome any feedback that you have, including comments on the clarity, accuracy, or completeness of the information. For more information, see How to send feedback to IBM.

Summary of changes

This information includes terminology, maintenance, and editorial changes. Technical changes or additions to the text and illustrations for the current edition are indicated by a vertical line to the left of the change.

Note: IBM z/OS policy for the integration of service information into the z/OS product documentation library is documented on the z/OS Internet Library under IBM z/OS Product Documentation Update Policy (www.ibm.com/docs/en/zos/latest?topic=zos-product-documentation-update-policy).

Summary of changes for z/OS 3.1

The following content is new, changed, or no longer included in z/OS 3.1.

New

The following content is new.

July 2024 refresh

- "Improving module fetch performance with LLA" on page 53 is updated to include the new subsection "Securing LLA" on page 54.

September 2023 release

- "Processor storage overview" on page 6 has a new subsection on Dedicated Memory.

Changed

The following content is changed.

April 2024 refresh

- "LLA notes" on page 54 is updated to add that in z/OS 3.1 and 2.5, LLA is set to service class SYSTEM.
- "Memory pools" on page 10 is updated to add a consideration in "Things to consider when your installation uses memory pools" on page 12.

September 2023 release

- Updates are made in "Selective enablement for I/O" on page 89.

Deleted

The following content was deleted.

September 2023 release

- None.

Chapter 1. Storage management overview

To tailor the system's storage parameters, you need a general understanding of the system initialization and storage initialization processes.

For information about the storage management subsystem (SMS), see the DFSMS library.

Initialization process

The system initialization process prepares the system control program and its environment to do work for the installation. The process essentially consists of:

- System and storage initialization, including the creation of system component address spaces.
- Master scheduler initialization and subsystem initialization.

When the system is initialized and the job entry subsystem is active, the installation can submit jobs for processing by using the START, LOGON, or MOUNT command.

The initialization process begins when the system operator selects the LOAD function at the system console. MVS locates all of the usable central storage that is online and available to the system, and creates a virtual environment for the building of various system areas.

IPL includes the following major initialization functions:

- Loads the DAT-off nucleus into central storage.
- Loads the DAT-on nucleus into virtual storage so that it spans above and below 16 megabytes (except the prefixed storage area (PSA), which IPL loads at virtual zero).
- Builds the nucleus map, NUCMAP, of the DAT-on nucleus. NUCMAP resides in virtual storage above the nucleus.
- Allocates the system's minimum virtual storage for the system queue area (SQA) and the extended SQA.
- Allocates virtual storage for the extended local system queue area (extended LSQA) for the master scheduler address space.

The system continues the initialization process, interpreting and acting on the system parameters that were specified. NIP carries out the following major initialization functions:

- Expands the SQA and the extended SQA by the amounts that are specified on the SQA system parameter.
- Creates the pageable link pack area (PLPA) and the extended PLPA for a cold start IPL; resets tables to match an existing PLPA and extended PLPA for a quick start or a warm start IPL. For more information about quick starts and warm starts, see *z/OS MVS Initialization and Tuning Reference*.
- Loads modules into the fixed link pack area (FLPA) or the extended FLPA. NIP carries out this function only if the FIX system parameter is specified.
- Loads modules into the modified link pack area (MLPA) and the extended MLPA. NIP carries out this function only if the MLPA system parameter is specified.
- Allocates virtual storage for the common service area (CSA) and the extended CSA. The amount of storage that is allocated depends on the values that are specified on the CSA system parameter at IPL.
- Allocates virtual storage for the restricted use common service area (RUCSA) and the extended RUCSA. The amount of storage that is allocated depends on the values that are specified on the RUCSA system parameter at IPL. If the RUCSA system parameter is not specified, there will be no RUCSA storage.
- Page protects the: NUCMAP, PLPA and extended PLPA, MLPA and extended MLPA, FLPA and extended FLPA, and portions of the nucleus.

  Note: An installation can override page protection of the MLPA and FLPA by specifying NOPROT on the MLPA and FIX system parameters.

See Figure 1 on page 4 for the relative position of the system areas in virtual storage. Most of the system areas exist both below and above 16 megabytes, providing an environment that can support both 24-bit and 31-bit addressing. However, each area and its counterpart above 16 megabytes can be thought of as a single logical area in virtual storage.

Creating address spaces for system components

In addition to initializing system areas, MVS creates address spaces for system components. MVS establishes an address space for the master scheduler (the master scheduler address space) and other system address spaces for various subsystems and system components. Some of the component address spaces are listed in Table 1 on page 2.

Table 1. Partial list of component address spaces

Address space	**Description**
MASTER	Master address space
ABARS, ABARxxxx	1 to 15 DFSMShsm secondary address spaces to perform aggregate backup or aggregate recovery processing.
ALLOCAS	Allocation services and data areas
ANTMAIN	Concurrent copy support
APPC	APPC/MVS component
ASCH	APPC/MVS scheduling
CATALOG	Catalog functions. Also known as CAS (catalog address space).
BPXOINIT	z/OS UNIX System Services
CONSOLE	Communications task
DFM	Distributed File Manager
DFMCAS	Distributed File Manager
DLF	Data lookaside facility
DUMPSRV	Dumping services
HSM	DFSMShsm
HZSPROC	IBM Health Checker for z/OS
FTPSERVE	FTP servers; can be user-specified names.
GDEDFM	For each Distributed File Manager/MVS user conversation that is active, an address space named GDEDFM is created.
GRS	Global resource serialization
IEFSCHAS	Scheduler address space
IOSAS	I/O supervisor, ESCON, I/O recovery
IXGLOGR	System logger
JES2	JES2
JES2AUX	JES2 additional support
JES2CI*xx*	1-25 JES2 address spaces used to perform z/OS converter and interpreter functions
JES2EDS	JES2 Email Delivery Services (EDS)

Table 1. Partial list of component address spaces (continued)	
Address space	**Description**
JES2MON	JES2 address space monitor
JES3	JES3
JES3AUX	JES3 additional support
JES3DLOG	JES3 hardcopy log (DLOG)
JESXCF	JES common coupling services address space
LLA	Library Lookaside
NFS	Network File System address space
OAM	DFSMSdfp Object Access Method (OAM). In a classic OAM configuration, there may be one address space to perform tape library and/or object processing; in a multiple OAM configuration there may be multiple Object OAM address spaces and/or a Tape Library OAM address space.
OMVS	z/OS UNIX System Services
OTIS	DFSMSdfp Object Access Method (OAM) Thread Isolation Support. In either a classic OAM configuration or a multiple OAM configuration, there might be one address space to perform object-related processing and enable subsystem deletion.
PCAUTH	Cross-memory support
PORTMAP	Portmapper function
RASP	Real storage manager (includes support for advanced address space facilities)
RMM	DFSMSrmm
RRS	Resource recovery services (RRS)
SMF	System management facilities
SMS	Storage management subsystem
SMSPDSE1	Optional restartable PDSE address space. If the SMSPDSE1 address space is started, SMSPDSE manages PDSEs in the LINKLST concatenation and SMSPDSE1 manages all other PDSEs.
SMSVSAM	VSAM record level sharing
TCP/IP	TCP/IP
TRACE	System trace
VLFf	Virtual lookaside facility
XCFAS	Cross system coupling facility
VTAM	VTAM
WLM	Workload management

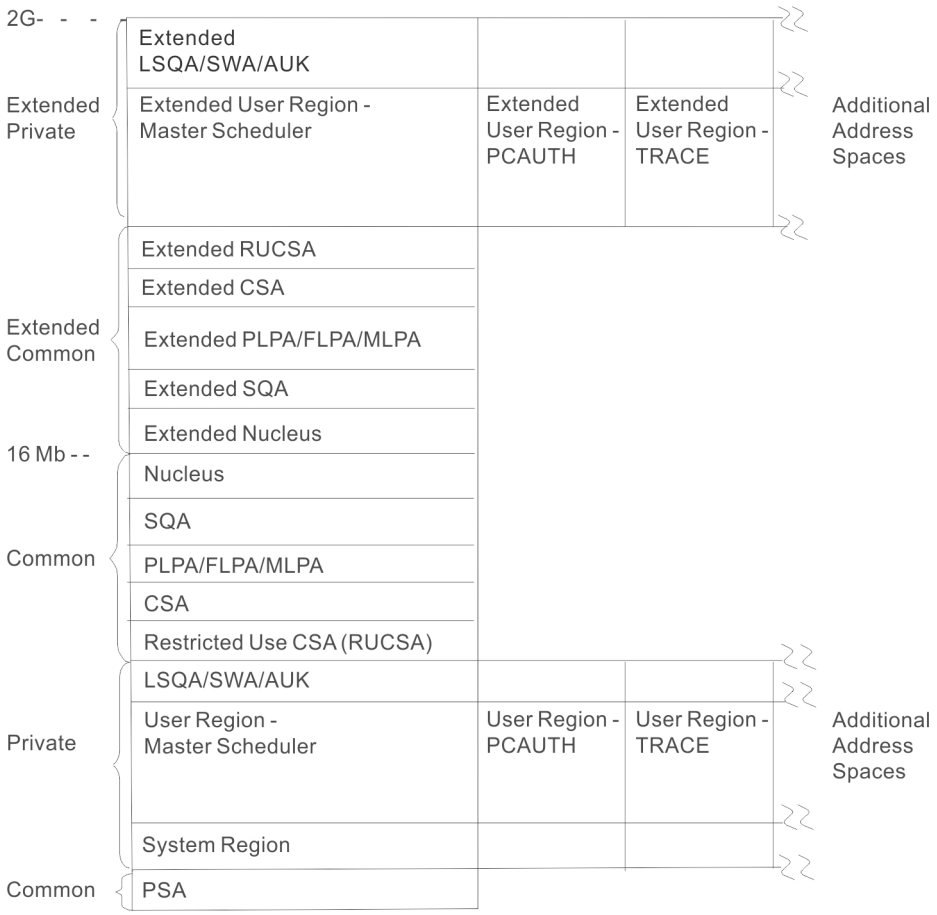


Figure 1. Virtual storage layout for multiple address spaces

Address spaces differ in their ability to use system services depending on whether the address space is a limited function or full function address space.

- Limited function address space

  If the specific initialization routines provided by the components that use IEEMB881 enter a wait state, pending Master Scheduler Initialization, STC does no additional address space initialization. Thus, the functions that component address spaces can perform are limited. Components with limited function address spaces cannot:

  – Allocate data sets.
  – Read JCL procedures from SYS1.PROCLIB.
  – Allocate a SYSOUT file through the Job Entry Subsystem.
  – Use some system services because the components frequently run in cross memory mode.

- Full function address space

  If, after a component completes its own initialization, it returns to IEEPRWI2 and completes STC processing, the address space is fully initialized. Such an address space is called a full function address space.

  The component creating a full function address space does not need to provide a procedure in SYS1.PROCLIB. If specified to IEEMB881, a common system address space procedure, IEESYSAS, will invoke a specified program to run in the address space.

  For example, if a full function address space called FFA is to be started using the module IEEMB899, the component would ordinarily need to supply a procedure of the following form:

```
//FFA    PROC
//       EXEC PGM=IEEMB899
```

The procedure will be invoked as follows:

```
//IEESYSAS  JOB
//FFA       EXEC IEESYSAS
```

The procedure IEESYSAS consists of the following statements:

```
//IEESYSAS  PROG=IEFBR14
//          EXEC PGM=&PROG
```

Master scheduler initialization

Master scheduler initialization routines initialize system services such as the system log and communications task, and start the master scheduler itself. They also cause creation of the system address space for the job entry subsystem (JES2 or JES3), and then start the job entry subsystem.

Note: When JES3 is the primary job entry subsystem, a second JES3 address space (JES3AUX)can be optionally initialized after master scheduler initialization completes. The JES3AUX address space is an auxiliary address space that contains JES3 control blocks and data.

Subsystem initialization

Subsystem initialization is the process of readying a subsystem for use in the system. IEFSSNxx members of SYS1.PARMLIB contain the definitions for the primary subsystems, such as JES2 or JES3, and the secondary subsystems, such as VPSS and DB2®. For detailed information about the data contained in IEFSSNxx members for secondary systems, please refer to the installation information for the specific system.

During system initialization, the defined subsystems are initialized. You should define the primary subsystem (JES) first, because other subsystems, such as DB2, require the services of the primary subsystem in their initialization routines. Problems can occur if subsystems that use the subsystem affinity service in their initialization routines are initialized before the primary subsystem. After the primary JES is initialized, then the subsystems are initialized in the order in which the IEFSSNxx parmlib members are specified by the SSN parameter. For example, for SSN=(aa,bb) parmlib member IEFSSNaa would be processed before IEFSSNbb.

Note: The storage management subsystem (SMS) is the only subsystem that can be defined before the primary subsystem. Refer to the description of parmlib member IEFSSNxx in *z/OS MVS Initialization and Tuning Reference* for SMS considerations.

Using IEFSSNxx to initialize the subsystems, you can specify the name of a subsystem initialization routine to be given control during master scheduler initialization, and you can specify the input parameter to be passed to the subsystem initialization routine. IEFSSNxx is described in more detail in *z/OS MVS Initialization and Tuning Reference*.

START/LOGON/MOUNT processing

After the system is initialized and the job entry subsystem is active, jobs may be submitted for processing. When a job is activated through START (for batch jobs), LOGON (for time-sharing jobs) or MOUNT, a new address space must be allocated. Note that before LOGON, the operator must have started TCAM or VTAM/TCAS, which have their own address spaces. Figure 1 on page 4 is a virtual storage map containing or naming the basic system component address spaces, the optional TCAM and VTAM® system address spaces, and a user address space.

The system resources manager decides, based on resource availability, whether a new address space can be created. If not, the new address space will not be created until the system resources manager finds conditions suitable.

Processor storage overview

Processor storage only consists of real storage (formerly called central storage) in the z/Architecture® mode. This section provides an overview of real storage. Note that unlike the combination of central and expanded storage in the ESA/390 environment, expanded storage is not supported in the z/Architecture mode.

The system uses a portion of both central storage and virtual storage. To determine how much central storage is available to the installation, the system's fixed storage requirements must be subtracted from the total central storage. The central storage available to an installation can be used for the concurrent execution of the paged-in portions of any installation programs.

The real storage manager (RSM) controls the allocation of central storage during initialization and pages in user or system functions for execution. Some RSM functions:

- Allocate central storage to satisfy GETMAIN requests for SQA and LSQA.
- Allocate central storage for page fixing.
- Allocate central storage for an address space that is to be swapped in.

If there is storage above 16 megabytes, RSM allocates central storage locations above 16 megabytes for SQA, LSQA, and the pageable requirements of the system. When non-fixed pages are fixed for the first time, RSM:

- Ensures that the pages occupy the appropriate type of frame
- Fixes the pages and records the type of frame used

Pages that must reside in central storage below 16 megabytes include:

- SQA subpool 226 pages.
- Fixed pages obtained using the RC, RU, VRC, or VRU form of GETMAIN if one of the following is true:
  - LOC=24 is specified.
  - LOC=RES, the default, is either specified or taken, and the program issuing the GETMAIN resides below 16 megabytes, runs in 24-bit mode, and has not requested storage from a subpool supported only above 16 megabytes.
- Fixed pages obtained using the LU, LC, EU, EC, VU, VC, or R form of GETMAIN.
- Storage whose virtual address and real address are the same (V=R pages).

Pages that can reside in central storage above 16 megabytes include:

- Nucleus pages.
- SQA subpools 239 and 245 pages.
- LSQA pages.
- All pages with virtual addresses greater than 16 megabytes.
- Fixed pages obtained using the RC, RU, VRC, or VRU form of GETMAIN if one of the following is true:
  - LOC=(24,31) is specified.
  - LOC=(RES,31) is specified.
  - LOC=31 is specified.
  - LOC=(31,31) is specified.
  - LOC=RES, the default, is either specified or taken, and the program issuing the GETMAIN resides above 16 megabytes virtual.
  - LOC=RES, the default, is either specified or taken, and the program issuing the GETMAIN resides below 16 megabytes virtual, but runs in 31-bit mode and has requested storage from a subpool supported only above 16 megabytes.
- Any non-fixed page.

Note: The system backs nucleus pages in real storage below 2 gigabytes. You can however, back SQA and LSQA pages above 2 gigabytes when you specify LOC=(24,64) or LOC=(31,64).

Each installation is responsible for establishing many of the central storage parameters that govern RSM's processing. The following overview describes the function of each area composing central storage.

The primary requirements/areas composing central storage are:

1. The basic system fixed storage requirements — the nucleus, the allocated portion of SQA, and the fixed portion of CSA.
2. The private area fixed requirements of each swapped-in address space — the LSQA for each address space and the page-fixed portion of each virtual address space.

Once initialized, the basic system fixed requirements (sometimes called global system requirements) remain the same until system parameters are changed. Fixed storage requirements (or usage) will, however, increase as various batch or time sharing users are swapped-in. Thus, to calculate the approximate fixed storage requirements for an installation, the fixed requirements for each swapped-in address space must be added to the basic fixed system requirements. Fixed requirements for each virtual address space include system storage requirements for the LSQA (which is fixed when users are swapped in) and the central storage estimates for the page-fixed portions of the installation's programs.

The central storage for the processor, reduced by the global fixed and paged-in virtual storage required to support installation options, identifies the central storage remaining to support swapped-in address spaces. The total number of jobs that can be swapped in concurrently can be determined by estimating the working set (the amount of virtual storage that must be paged in for the program to run effectively) for each installation program. The working set requirements will vary from program to program and will also change dynamically during execution of the program. Allowances should be made for *maximum* requirements when making the estimates.

System preferred area

To enable a V=R allocation to occur and storage to be varied offline, MVS performs special handling for the following types of pages:

- SQA
- LSQA for non-swappable address spaces
- Fixed page frame assignments for non-swappable address spaces.

Because MVS cannot, upon demand, free the frames used for these page types, central storage could become fragmented (by the frames that could not be freed). Such fragmentation could prevent a V=R allocation or prevent a storage unit from being varied offline. Therefore, for all storage requests for the types of pages noted, RSM allocates storage from the preferred area to prevent fragmentation of the non-preferred "reconfigurable" area.

A system parameter, RSU, allows the installation to specify the number of storage units that are to be kept free of long-term fixed storage allocations, and thus be available for varying offline. Once this limit is established, the remainder of central storage, excluding storage reserved for V=R allocation, the LFAREA (when used as 4 KB frames) and some system reserved areas, is marked as preferred area storage and used for long-term fixed storage allocation.

Nucleus area

The nucleus area contains the nucleus load module and extensions to the nucleus that are initialized during IPL processing.

The nucleus includes a base and an architectural extension.

The fixed link pack area (FLPA)

An installation can elect to have some modules that are normally loaded in the pageable link pack area (PLPA) loaded into the fixed link pack area (FLPA). This area should be used only for modules that

significantly increase performance when they are fixed rather than pageable. Modules placed in the FLPA must be reentrant and refreshable.

The FLPA exists only for the duration of an IPL. Therefore, if an FLPA is desired, the modules in the FLPA must be specified for each IPL (including quick-start and warm-start IPLs).

It is the responsibility of the installation to determine which modules, if any, to place in the FLPA. Note that if a module is heavily used and is in the PLPA, the system's paging algorithms will tend to keep that module in central storage. The best candidates for the FLPA are modules that are infrequently used but are needed for fast response to some terminal-oriented action.

`Specified by:` A list of modules to be put in FLPA must be established by the installation in the fixed LPA list (IEAFIXxx) member of SYS1.PARMLIB. Modules from any partitioned data set can be included in the FLPA. FLPA is selected through specification of the FIX system parameter in IEASYSxx or from the operator's console at system initialization.

Any module in the FLPA will be treated by the system as though it came from an APF-authorized library. Ensure that you have properly protected any library named in IEAFIXxx to avoid system security and integrity exposures, just as you would protect any APF-authorized library. This area may be used to contain reenterable routines from either APF-authorized or non-APF-authorized libraries that are to be part of the pageable extension to the link pack area during the current IPL.

System queue area (SQA-Fixed)

SQA is allocated in fixed storage upon demand as long-term fixed storage and remains so until explicitly freed. The number of central frames assigned to SQA may increase and decrease to meet the demands of the system.

All SQA requirements are allocated in 4K frames as needed. These frames are placed within the preferred area (above 16 megabytes, if possible) to keep long-term resident pages grouped together.

If no space is available within the preferred area, and none can be obtained by stealing a non-fixed/unchanged page, then the "reconfigurable area" is reduced by one storage increment and the increment is marked as preferred area storage. An increment is the basic unit of physical storage. If there is no "reconfigurable area" to be reduced, a page is assigned from the V=R area. Excluded from page stealing are frames that have been fixed (for example, through the PGFIX macro), allocated to a V=R region, placed offline using a CONFIG command, have been changed, have I/O in progress, or contain a storage error.

Fixed LSQA storage requirements

Except for the extended private area page tables, which are pageable, the local system queue area (LSQA) for any swapped-in address space is fixed in central storage (above 16 megabytes, if possible). It remains so until it is explicitly freed or until the end of the job step or task associated with it. The number of LSQA frames allocated in central storage might increase or decrease to meet the demands of the system. If preferred storage is required for LSQA and no space is available in the preferred area, and none can be obtained by stealing a non-fixed/unchanged page, then the "reconfigurable area" is reduced by one storage increment, and the increment is marked as preferred area storage. If there is no "reconfigurable area" to be reduced, a page is assigned from the V=R area.

V=R area

This area is used for the execution of job steps specified as fixed because they are assigned to V=R regions in virtual storage (see “Real regions” on page 40). Such jobs run as nonpageable and nonswappable.

The V=R area is allocated starting directly above the system region in central storage. The virtual addresses for V=R regions are mapped one-to-one with the central addresses in this area. When a job requests a V=R region, the lowest available address in the V=R area in central storage, followed by a contiguous area equal in size to the V=R region in virtual storage, is located and allocated to the region.

If there is not enough V=R space available in the V=R area, the allocation and execution of new V=R regions are prohibited until enough contiguous storage is made available.

The V=R area can become fragmented because of system allocation for SQA and LSQA or because of long-term fixing. When this happens, it becomes more difficult — and may be impossible — for the system to find contiguous storage space for allocating V=R regions. Such fragmentation may last for the duration of an IPL. It is possible that fragmentation will have a cumulative effect as long-term fixed pages are occasionally assigned frames from the V=R area.

Specified by:

- The REAL parameter of the IEASYSxx member
- Use of the REAL parameter from the operator's console during NIP.

Dedicated Memory

In the demand paging model, real storage is assigned to the application upon demand, whether that is at the time of an IARV64 GETSTOR or page fault. Some applications perform better when they do not have to compete with other applications for memory.

Dedicated Memory provides a way to assign memory directly to the application at job-step start, while ensuring that the memory remains with the job step until it ends. Even if the system is in a frame shortage, Dedicated Memory is not paged. Logically, Dedicated Memory is defined at the top of Central Storage to take advantage of systems configured with large amounts (>4 Terabytes) of memory that would otherwise be managed as part of the 2G LFAREA.

The installation defines the amount of Dedicated Memory at system initialization time in IARPRMxx and memory is then assigned to applications based on SMFLIMxx specifications. SMFLIMxx has a broad set of filters to determine which applications are to be assigned Dedicated Memory including Sysname, Pgmname, and Jobname. At job-step start, the specified amount of memory in 2G units of real storage is assigned to the address space where the job step is about to run. Once the memory is assigned, Dedicated Memory is locally managed within the address space, minimizing system overhead. Dedicated Memory is restricted to High Virtual private, but can be used for all frame types: 4K, 1M fixed, 1M Pageable and 2G. While the job step is running, the system passively reforms 1M frames, but not 2G frames.

Before the IPL, the System Administrator must decide how much Dedicated Memory to define on the system. DUMPSRV, zCX instances, and other applications that place high demands on memory are recommended applications. The OMVS address space is not eligible for a Dedicated Memory Assignment. Historical real memory usage for High Virtual Private provides a good estimate of the amount of Dedicated Memory to assign to each application. The following SMF 30 fields provide this information:

- SMF30HVR – High water mark of the number of real storage frames that are used to back 64-bit private storage.
- SMF30HVA – High water mark of the amount of auxiliary storage that is used to back 64-bit private storage.
- SMF30_InUseAs2GHWM – High water mark of the number of 2G frames in use by the job step.
- SMF30_Num2GFailed – Number of 2G frames that could not be obtained because none were available at the time of the IARV64 GETSTOR request.

((SMF30HVR+SMF30HVA)*4K+SMF30_InUseAs2GHWM*2G) is a rough upper bound on memory usage for a particular job step and can be used to estimate the amount of Dedicated Memory to assign. Since Dedicated Memory is only used for High Virtual storage, the amount of Dedicated Memory that is assigned to a job step should not exceed its Memlimit.

About 2G out of every 124G of Dedicated Memory is used by the system to manage Dedicated Memory and therefore is ineligible to be assigned to a job step.

Dedicated Memory can be configured offline and moved to a different LPAR. For more information about storage reconfiguration, see *z/OS MVS System Commands*.

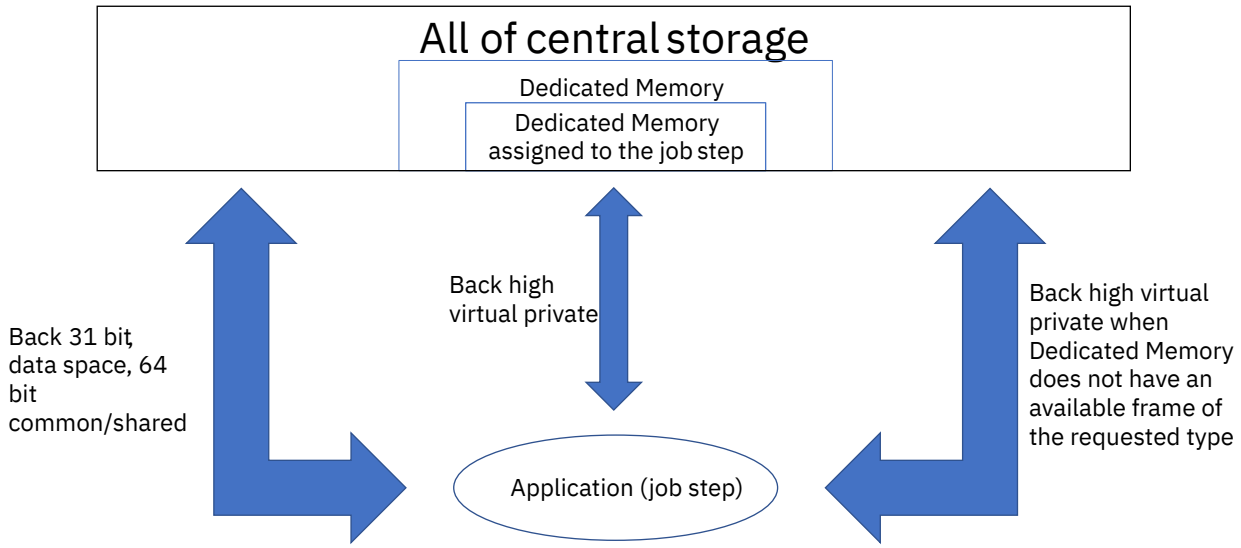


Figure 2. Dedicated Memory Usage Diagram

Memory pools

When you use a workload management (WLM) policy that classifies work into a WLM resource group with a real storage memory limit, you can explicitly restrict physical real memory consumption of work that is running in concurrent address spaces. An address space that is associated with the resource group through classification connects to the resource group's memory pool. All real storage that is used to manage the address space and any data space or hiperspace that is created by the address space, including the backing of its virtual storage, are counted towards the collective pool limit. However, auxiliary storage resources that are used for virtual storage paging, continue to be shared by all address spaces in the system.

A memory pool does not reserve real storage frames for its exclusive use. When unused frames are available in the system, the collection of address spaces is restricted to using up to the real memory limit of the memory pool. When a memory pool gets near its limit, the system starts to page-out pages from the memory pool to free frames. It also reduces any real storage that is kept for performance only reasons. The memory pool protects physical memory allocation from other units of work that are running on the system. If the system gets low on real storage, it initiates paging from any of the address spaces regardless of being in a memory pool. Address spaces that are not classified to a memory pool, connect to the global pool. Common storage and shared storage are always backed by frames from the global pool.

The relationship between virtual and real storage includes setting limits through a consideration of various resources. The amount of virtual storage that an address space can allocate (obtain), but not necessarily reference is controlled by limits that are not defined through WLM resource groups. Virtual limits are only defined at an address space basis where real storage limits cannot be controlled. Real storage limits can be defined only at the memory pool collective address space level.

The amount of virtual storage that an address space can obtain is limited by the following controls.

- Region size for 31-bit storage
- Memory limit (MEMLIMIT) for above the bar storage
- Program Properties Table (PPT)
- Installation exits.
- Data spaces and hiperspace region limits

For more information about virtual storage, see “Virtual storage overview” on page 14.

Allocated and referenced virtual storage can be backed by either real storage or paged-out to auxiliary storage. More virtual storage can be allocated than referenced and more storage cannot be referenced than allocated. An address space can never use more real storage than its virtual storage limit. However,

the real storage that is used for dynamic address translation (DAT) tables and other system-related control blocks are also used to manage virtual storage and *virtual I/O (VIO)*. Storage that is used by the system for performance reasons is also accumulated to real storage used by an address space. The additional real storage that the system uses makes it possible for real storage usage to be higher than the virtual storage limits and what is allocated for storage. In addition, all real storage that is associated with creating the address space is used to manage hiperspace, data spaces, and the common area data spaces.

An allocation service, such as STORAGE OBTAIN, GETMAIN, and IARV64, can reject a request when it causes the associated address spaces virtual storage limit to be exceeded. Various memory pool thresholds, which start with the OK threshold, are derived by the system from the memory pool's resource group memory limit and are used to control system behavior. The limit threshold attributes are defined in Table 2 on page 11.

Table 2. Threshold limits in ascending order

Threshold name	**Thresholds main attributes**
OK	Within normal limits.
Steal	The system initiates paging in an attempt to return to the OK threshold.
High	Selective real storage requesters are suspended until the limit falls below the high threshold. When the pool remains above the high threshold for the system defined period, pool members are subject to abend X'E22'.
Maximum	The memory pool limit that is defined by the WLM policy.

When the memory pool is over its high threshold, similar to virtual storage limits, system services might reject fixed storage allocation and page fix requests. The system might also take the following actions.

- Suspend users of services that require real storage and keep services suspended until the memory pool falls below its high threshold. For example, a unit of work that takes an enabled page fault is suspended when the memory pool is above the high threshold. When the pool falls below the high threshold, the unit of work resumes.
- Allow others to temporarily exceed the high threshold even when above the memory pool limit. For example, a unit of work, running with a program status word (PSW) that is disabled for interrupts, which page fixes a virtual page, can exceed the limit because it cannot be suspended.

To contain a pool within its memory pool limit and reduce the need for suspending requesters, the system begins paging the storage of memory pool member address spaces when the memory pool is at its steal threshold. Suspended units of work are resumed when the memory pool is below its high threshold. The system issues messages at different thresholds and might end the current job step of any pool member if the memory pool exceeds the threshold for the system defined time frame.

The following information summarizes the types of thresholds and actions.

When at the steal threshold

- The system issues message IAR055I to indicate that paging of memory pool members is started.
- The system starts to page out backed virtual storage from any number of members of the memory pool to bring the memory pools overall consumption down below the OK threshold. Only virtual storage for non-fixed pools and non-disabled reference (DREF) can be paged out.

When at or above the high threshold

- Message IAR052E is issued when the high threshold is reached. The system continues paging to reduce the memory pool real storage usage to the OK threshold, and then message IAR052E is deleted.

- Task processing that requires the system to back referenced virtual memory with real memory is suspended and not resumed until the memory pool falls below the high threshold. The suspension is done to prevent the pool from going over its limit. Requests can be in the form of page faults, STORAGE OBTAIN, GETMAIN, IARV64, and others. To accommodate system requesters that must not be suspended because of memory pool limits, the system allows certain requesters to increase the real storage usage above any threshold. The requesters include the following types.
  - Units of work that are:
    - Operating in service request block (SRB) mode
    - Running on behalf of DUMP processing
    - Disabled (and cannot be suspended)
    - Holding a system suspend lock (such as a local lock)
    - Going through termination
  - Other cases when the system code must obtain real storage.
- When real storage usage does not decrease below the high threshold within the system-defined time frame and the initial reason for reaching the high threshold is from reclassification, memory pool size reduction, or neither, the system action is explained below.
  - **Not from reclassification** of an address space into the memory pool **or from the reduction in the pool size.**

    The system might abnormally end the currently running job step of one or more memory pool classified address spaces. The job ends with an X'E22' abend code, which can or cannot terminate the job step. Real storage usage can decrease by either the system paging storage or when jobs within the memory pool release storage (including fixed storage that the system cannot page).
  - **From reclassification** of an address space into the memory pool **or the reduction in the pool size.**

    The system issues message IAR058E to indicate that the memory pool is over the limit. The system is not able to reduce the size below the high threshold within the system defined time frame. Unless a new steal reason is triggered, the system stops stealing pages from the memory pool. The memory pool is allowed to persist over the high threshold. However, you must take the appropriate action, to either increase the size of the memory pool or remove members from the pool. For more information about the appropriate actions, see message IAR058E in *z/OS MVS System Messages, Vol 6 (GOS-IEA)*.

When back at or below the OK threshold

- Message IAR054I is issued indicating that the memory pool is no longer approaching the memory pool limit.
- Any outstanding related memory pool messages are deleted.

For more information about all IAR messages, see the *IAR message* in *z/OS MVS System Messages, Vol 6 (GOS-IEA)*.

Things to consider when your installation uses memory pools

- Understand how workloads behave in a memory pool. For more information, see *z/OS MVS Planning: Workload Management*.
- IBM suggests that you use memory pools when you need to limit memory consumption for workloads. For example, Apache Spark provides guidance about how to operate workloads in a memory pool. For more information, see "Configuring z/OS workload management for Apache Spark" in IBM Documentation (www.ibm.com/docs/en)
- The WLM policy is sysplex wide, which results in resource groups that exist on each system in the sysplex, yet memory pools are managed on a per system basis. Therefore, a 100 GB memory pool in a

two-way sysplex might use nearly 200 GB of real storage, but only 100 GB per system. For the resource group memory pool definition and policy scope, review *z/OS MVS Planning: Workload Management*.

- Understand the difference between virtual storage and real storage limits. For more information, see "Virtual storage overview" on page 14.
- Dedicated memory, as described in "Dedicated Memory" on page 9, is not subject to memory pool constraints.
- **Paging to auxiliary storage**

  The system can start paging for one or more memory pools and all or some of pool members simultaneously. The system does not differentiate between auxiliary storage resources that are used for memory pools and the global pool. As such, reevaluate your real memory to auxiliary storage size requirements to ensure that auxiliary storage resources meet your paging needs.

  Understand memory pool storage requirements and appropriate classification rules for a workload before you run the workload in production. Review the related product documentation for guidance on memory pool usage. For example, putting all elements of a workload in the same memory pool cannot be correct for that workload.

  Memory pool paging uses system resources that might cause a performance impact on work that runs outside the memory pool. Evaluate frequent memory pool paging in the overall system context. Understand if paging is acceptable always, sometimes, a long period, at specific times, or never, and when you must intercede if an unacceptable condition occurs. For example, consider system automation that monitors memory pool-related paging messages and changes in paging, and raises alerts.
- Consider automation that can possibly increase the memory pool size to prevent unnecessary processing delays or termination. See the previous section for paging considerations and automation.
- Prevent contention with resources that are shared outside memory pool boundaries. As a memory pool is capping the amount of real storage that can be used by its members possibly resulting in execution threads or suspended work units when limits are reached, resources must not be shared across memory pool boundaries in a way that might block others outside the pool. This includes other memory pools and those running under the global pool. Doing so can result in unnecessary delays. For example, a memory pool member must not get exclusive ownership of a data set, file system that is used by a unit work that is running outside its memory pool.
- Any address space that runs a function that is considered important by the installation are not to be capped in general and must not be classified to a resource group with a memory limit.

Reducing memory pool thresholds and reclassifying pool members

Consider the following points when you want to reduce the memory limit of a resource group, reclassify address spaces from a memory pool to the global pool, or reclassify address spaces from the global pool to a memory pool.

The following actions can result in the memory pool possibly reaching thresholds that can have negative affects on what is running in the memory pool. The affects of reaching the various thresholds can vary by what is running in the memory pool and how the program reacts to the limitation. For more information, see the following sections:

To understand how the thresholds are defined, see Table 2 on page 11.
To understand the actions and subsequent implications, see "Things to consider when your installation uses memory pools" on page 12.

- If the steal threshold is reached, the system starts to steal frames, which causes paging in an attempt to enforce the memory pool threshold.
  - Long-term paging can have negative affects on other programs that are running outside the memory pool. For more information, see "Paging to auxiliary storage" in "Things to consider when your installation uses memory pools" on page 12.
  - If the system cannot reduce the memory pool real storage usage to an acceptable level, message IAR058E is issued. Message IAR058E indicates that the pool is over its limit, the system cannot

reduce it, and your installation must take further action. If reclassification or reduction in the memory pool caused the exceeded limit, the system cannot terminate any address spaces in the memory pool. For more information, see message IAR058E in *z/OS MVS System Messages, Vol 6 (GOS-IEA)*.

The reason the system cannot reduce real storage usage, with the previously larger pool or without the new reclassified address spaces, is that the collective memory pool address spaces were previously able to fix more storage than is allowed in the memory pool. Because the system cannot page fixed storage, it cannot reduce the current usage. Your installation must correct the condition.

Note: If the high threshold is reached, units of work in the address space can be suspended until the condition is fixed.

Virtual storage overview

While there is no theoretical limit to 64-bit virtual storage ranges, practical limits exist to the finite real storage frames (formally called *central storage*) and auxiliary storage pages (slots) that back the virtual storage. Thus, estimating and limiting the virtual storage allocated on a system is important primarily because some percentage, for instance 25 percent, of the virtual storage must be backed by real storage for the system to function and perform well. In cases where there is insufficient real storage to contain all the backed virtual storage of all the active address spaces, the system can, based on work importance, postpone new work, or page out the contents of pageable (not fixed) real storage frames to auxiliary storage, or both. Paging frees up the frames for use by more important work. A real storage memory pool limit can also cause the system to page out real frames of pool members even though there are plenty of available real frames on the system.

For information about estimating the amount of auxiliary storage your system will need, see the discussion of paging data space in Chapter 2, “Auxiliary storage management initialization,” on page 67.

Each installation can use virtual storage parameters to specify how certain virtual storage areas are to be allocated and limited. These parameters have an impact on real storage use and overall system performance. The following overview describes the function of each virtual storage area. For information about identifying problems with virtual storage requests, see “Identifying problems in virtual storage (DIAGxx parmlib member)” on page 47.

The virtual storage address space and ESA extensions

The long history of z/OS has resulted in the current 24-bit, 31-bit, and 64-bit virtual storage addressing modes (AMODE) and the Enterprise System Architecture (ESA) address space control (ASC) modes. These modes are used to enable and prevent virtual storage reference to the various address space virtual storage areas: the 64-bit high virtual areas, the 31-bit extended areas, and the 24-bit non-extended areas. For each virtual storage area, specific z/OS services and constructs define how the storage is allocated, deallocated, formatted, fixed, and so on.

Figure 3 on page 16 shows the layout of virtual storage for an address space and the various storage areas. The following information provides some necessary background about these areas. (For detailed information, see *z/OS MVS Programming: Assembler Services Guide* and *z/OS MVS Programming: Extended Addressability Guide*.)

- The 24-bit and 31-bit virtual storage areas are managed according to the concept of subpools, where each subpool has specific characteristics, such as authorization requirements, fixed or pageable storage, common (available to all address spaces) or private storage area, a system or application program area, and so on.
  - For each area below the 16 MB line, there is an equivalent area above the line. For example, the System Queue Area (SQA) is common fixed storage below the line, and Extended SQA is above the line but below the 2 GB bar.
  - Services such as GETMAIN, STORAGE OBTAIN, CPOOL are used to allocate, deallocate, and alter storage in these areas on a byte basis.
  - Specific virtual private ranges can be shared between address spaces.

  - Real frames backing these areas are restricted in size and, when fixed, must reside in a real storage area that meets the requirements of the virtual storage addressing mode.
- The 64-bit virtual storage areas are managed according to the concept of memory objects. Memory objects are allocated in megabyte increments and have specific attributes.
  - Services such as IARV64 and IARCP64 are used to allocate, deallocate, and alter storage in these areas.
  - Specific virtual storage ranges can be defined as common or private and, if private, can be shared among different address spaces.
  - Real frames backing the virtual storage in these areas, both pageable and fixed, can reside anywhere.
- Data spaces have the following characteristics:
  - They are limited to 2 GB and provide additional 31-bit addressable storage ranges for programs running in AR ASC mode.
  - The DSPSERV service is the primary means of allocating, deallocating, and manipulating data space storage.
  - Although data spaces are still prevalent, the support for 64-bit high virtual storage has removed their main benefit of providing additional virtual storage capacity.
  - Data spaces are owned by a specific address space, and an address space can own more than one data space.
  - Data spaces can be addressable by one or more address spaces.
  - Data spaces can have pageable or non-pageable storage.
  - Data spaces can be backed by various real storage frame sizes.
- Hiperspaces are similar to data spaces. See *z/OS MVS Programming: Extended Addressability Guide* for more information.

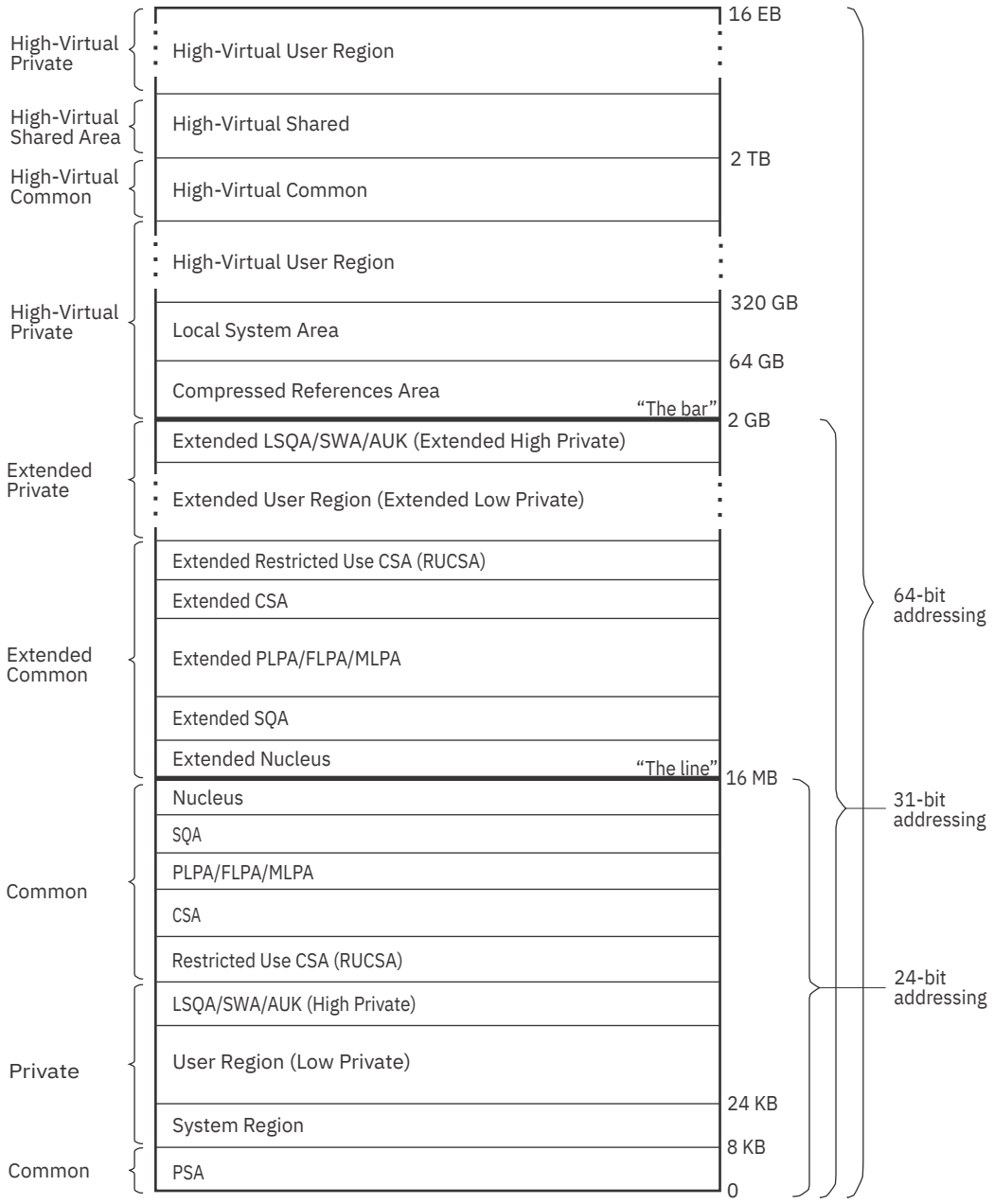


Figure 3. Virtual storage layout for a single address space (not drawn to scale)

Note: This topic does not cover 64-bit high virtual storage (above the 2-gigabyte address) in detail. For more information about 64-bit high virtual storage, see *z/OS MVS Programming: Extended Addressability Guide*.

The *common area* contains system control programs and control blocks. The following storage areas are located in the common area:

- Prefixed storage area (PSA)
- Common service area (CSA)
- Restricted use common service area (RUCSA)
- Pageable link pack area (PLPA)
- Fixed link pack area (FLPA)
- Modified link pack area (MLPA)

- System queue area (SQA)
- Nucleus, which is fixed and non-swappable.

Each storage area in the common area (below 16 megabytes) has a counterpart in the extended common area (above 16 megabytes) with the exception of the PSA.

Each address space uses the same common area. Portions of the common area are paged in and out as the demands of the system change and as new user jobs (batch or time-shared) start and old ones terminate.

The *private area* contains:

- A local system queue area (LSQA).
- A scheduler work area (SWA).
- The authorized user-key area (AUK).
- A 16K system region area.
- Either a V=V (virtual = virtual) or V=R (virtual = real) private user region for running programs and storing data.

Except for the 16K system region area and V=R user regions, each storage area in the private area below 16 megabytes has a counterpart in the extended private area above 16 megabytes.

Each address space has its own unique private area allocation. The private area (except LSQA) is pageable unless a user specifies a V=R region. If assigned as V=R, the actual V=R region area (excluding SWA, the 16K system region area, and AUK area) is fixed and non-swappable.

The 64-bit high virtual storage address space

See "The virtual storage address space and ESA extensions" on page 14 for a basic understanding of high virtual storage. See *z/OS MVS Programming: Extended Addressability Guide* for details.

There are controls similar to the 31-bit address space controls for limiting (MEMLIMIT) the amount of high virtual storage that an address space can consume. These are specifically described in Limiting the use of private memory objects in *z/OS MVS Programming: Extended Addressability Guide*.

General virtual storage allocation considerations

Virtual storage allocated in each address space is divided between the system's requirements and the user's requirements. The base system control programs require space from each of the basic areas.

Storage for SQA, CSA, LSQA, and SWA is assigned for either the system or a specific user. Generally, space is assigned to the system in SQA and CSA and, for users, in LSQA or SWA.

System Queue Area (SQA/Extended SQA)

This area contains tables and queues relating to the entire system. Its contents are highly dependent on configuration and job requirements at an installation. The total amount of virtual storage and number of private virtual storage address spaces are two of the factors that affect the system's use of SQA.

The SQA is allocated directly below the nucleus; the extended SQA is allocated directly above the extended nucleus.

The size of the SQA can be specified through the:

- SQA parameter in the IEASYSxx member of SYS1.PARMLIB
- NIP or operator's console.

If the specified amount of virtual storage is not available during initialization, a warning message will be issued. The SQA parameter may be respecified at that time from the operator's console.

Virtual SQA is allocated as a number of 64K blocks to be added to the minimum system requirements for SQA. If the SQA required by the system configuration exceeds the amount that has been reserved

through the SQA parameter, the system attempts to allocate additional virtual SQA from the CSA area. When certain storage thresholds are reached, as explained in “SQA/CSA thresholds” on page 18, the system stops creating new address spaces. When SQA is in use, it is fixed in central storage.

The size of the SQA cannot be increased or decreased by the operator during a restart that reuses the previously initialized PLPA (a quick start). The size will be the same as during the preceding IPL.

SQA/CSA thresholds

Ensuring the appropriate size of the extended SQA and extended CSA storage is critical to the long-term operation of the system.

If the size allocated for SQA is too large, the thresholds for the IRA100E and IRA101E messages will not be met and CSA can become exhausted and cause a system outage. One way to avoid this problem is to allocate the minimum SQA required or allow for some CSA conversion so that the storage thresholds that trigger the IRA100E and IRA101E messages are based only on the remaining CSA.

If the size allocated for extended SQA is too small or is used up very quickly, the system attempts to use extended CSA. When both extended SQA and extended CSA are used up, the system allocates space from SQA and CSA below 16 megabytes. The allocation of this storage could eventually lead to a system failure.

- When the size, in bytes, of combined total of free SQA + CSA pages falls below the "high insufficient" threshold, the system issues message IRA100E
- If the size, in bytes, of available SQA and CSA pages falls below the "low insufficient" threshold, the system issues message IRA101E
- If storage is freed such that the available SQA+CSA amount reaches the "high sufficient" threshold, the system issues message IRA102I

The following conditions may be responsible for a shortage of SQA/CSA:

- There has been storage growth beyond the previous normal range.
- Allocation of SQA and/or CSA is inadequate.
- The current thresholds at which the IRA100E and/or IRA101E messages are issued are too high for your installation.

Note: IBM recommends that you do not change the storage thresholds set by the system. Setting the thresholds too low can hamper the ability of the system to recover from storage shortages and may result in unscheduled system outages. Setting the thresholds too high can cause IRA100E, IRA101E, and IRA102I messages to be issued excessively. If you do change the storage thresholds, you should be sure that the threshold is not being reached because of a problem in the system or inadequate allocation of CSA/SQA.

The SQA/CSA threshold levels are contained in the IGVDCLIM CSECT in load module IEAIPL04. Table 3 on page 18 shows the offsets of the threshold values.

Table 3. Offsets for the SQA/CSA threshold levels

Offset	Default Value	System Enforced Minimum	Description	Comments
0000	x'41000'	x'9000'	Sufficient space high	IRA102I message issued at this threshold
0004	x'21000'	x'5000'	Sufficient space low	IRA102I message issued at this threshold
0008	x'40000'	x'8000'	Insufficent space high	IRA100E message issued at this threshold
000C	x'20000'	x'4000'	Insufficient space low	IRA101E message issued at this threshold

If you change the default thresholds, make sure that the new sufficient threshold values are at least x'1000' larger than the insufficient values. The new values become effective at the next IPL.

Pageable link pack area (PLPA/Extended PLPA)

This area contains SVC routines, access methods, and other read-only system programs along with any read-only reenterable user programs selected by an installation that can be shared among users of the system. Any module in the pageable link pack area will be treated by the system as though it came from an APF-authorized library. Ensure that you have properly protected SYS1.LPALIB and any library named in LPALSTxx or on an LPA statement in PROGxx to avoid system security and integrity exposures, just as you would protect any APF-authorized library.

It is desirable to place all frequently used refreshable SYS1.LINKLIB and SYS1.CMDLIB modules in the PLPA because of the following advantages:

- If possible, PLPA is backed by central storage above 16 megabytes; central storage below 16 megabytes is then available for other uses.
- The length of time that a page occupies central storage depends on its frequency of use. If the page is not used over a period of time, the system will reuse (steal) the central storage frame that the page occupies.
- The most frequently used PLPA modules in a time period will tend to remain in central storage.
- PLPA paged-in modules avoid program fetch overhead.
- Two or more programs that need the same PLPA module share the common PLPA code, thus reducing the demand for central storage.
- The main cost of unused PLPA modules is paging space, because only auxiliary storage is involved when modules are not being used.
- All modules in the PLPA are treated as refreshable, and are not paged-out. This action reduces the overall paging rate compared with modules in other libraries.

See "Placing modules in the system search order for programs" on page 19 for an alternative suggestion on the placement of some PLPA and SYS1.CMDLIB modules. Any installation may also specify that some reenterable modules from the LNKLST concatenation, SYS1.SVCLIB, and/or the LPALST concatenation be placed in a fixed extension to the link pack area (FLPA) to further improve performance (see "The fixed link pack area (FLPA)" on page 7).

Modules loaded into the PLPA are packed within page boundaries. Modules larger than 4K begin on a page boundary with smaller modules filling out. PLPA can be used more efficiently through use of the LPA packing list (IEAPAKxx). IEAPAKxx allows an installation to pack groups of related modules together, which can sharply reduce page faults. The total size of modules within a group should not exceed 4K, and the residence mode (RMODE) of the modules in a group should be the same. For more information about IEAPAKxx, see *z/OS MVS Initialization and Tuning Reference*.

Placing modules in the system search order for programs

Modules (programs), whether stored as load modules or program objects, must be loaded into both virtual storage and central storage before they can be run. When one module calls another module, either directly by asking for it to be run or indirectly by requesting a system service that uses it, it does not begin to run instantly. How long it takes before a requested module begins to run depends on where in its search order the system finds a usable copy and on how long it takes the system to make the copy it finds available.

You should consider these factors when deciding where to place individual modules or libraries containing multiple modules in the system-wide search order for modules:

- The search order the system uses for modules
- How placement affects virtual storage boundaries
- How placement affects system performance
- How placement affects application performance

Search order the system uses for programs

When a program is requested through a system service (like LINK, LOAD, XCTL, or ATTACH) using default options, the system searches for it in the following sequence:

1. Job pack area (JPA)

   A program in JPA has already been loaded in the requesting address space. If the copy in JPA can be used, it will be used. Otherwise, the system either searches for a new copy or defers the request until the copy in JPA becomes available. (For example, the system defers a request until a previous caller is finished before reusing a serially-reusable module that is already in JPA.)

2. TASKLIB

   A program can allocate one or more data sets to a TASKLIB concatenation. Data sets concatenated to TASKLIB are searched for after JPA but before any specified STEPLIB or JOBLIB. Modules loaded by unauthorized tasks that are found in TASKLIB must be brought into private area virtual storage before they can run. Modules that have previously been loaded in common area virtual storage (LPA modules or those loaded by an authorized program into CSA) must be loaded into common area virtual storage before they can run. For more information about TASKLIB, see *z/OS MVS Programming: Assembler Services Guide*.

3. STEPLIB or JOBLIB

   STEPLIB and JOBLIB are specific DD names that can be used to allocate data sets to be searched ahead of the default system search order for programs. Data sets can be allocated to both the STEPLIB and JOBLIB concatenations in JCL or by a program using dynamic allocation. However, only one or the other will be searched for modules. If both STEPLIB and JOBLIB are allocated for a particular jobstep, the system searches STEPLIB and ignores JOBLIB. Any data sets concatenated to STEPLIB or JOBLIB will be searched after any TASKLIB but before LPA. Modules found in STEPLIB or JOBLIB must be brought into private area virtual storage before they can run. Modules that have previously been loaded in common area virtual storage (LPA modules or those loaded by an authorized program into CSA) must be loaded into common area virtual storage before they can run. For more information about JOBLIB and STEPLIB, see *z/OS MVS JCL Reference*.

4. LPA, which is searched in this order:

   a. Dynamic LPA modules, as specified in PROGxx members

   b. Fixed LPA (FLPA) modules, as specified in IEAFIXxx members

   c. Modified LPA (MLPA) modules, as specified in IEALPAxx members

   d. Pageable LPA (PLPA) modules, loaded from libraries specified in LPALSTxx or PROGxx

   LPA modules are loaded in common storage, shared by all address spaces in the system. Because these modules are reentrant and are not self-modifying, each can be used by any number of tasks in any number of address spaces at the same time. Modules found in LPA do not need to be brought into virtual storage, because they are already in virtual storage.

5. Libraries in the linklist, as specified in PROGxx and LNKLSTxx.

   By default, the linklist begins with SYS1.LINKLIB, SYS1.MIGLIB, SYS1.CSSLIB, SYS1.SIEALNKE, and SYS1.SIEAMIGE. However, you can change this order using SYSLIB in PROGxx and add other libraries to the linklist concatenation. The system must bring modules found in the linklist into private area virtual storage before the programs can run.

Notes:

1. For more information about which system services load modules, see:
   - *z/OS MVS Programming: Assembler Services Guide*
   - *z/OS MVS Programming: Assembler Services Reference ABE-HSP*
   - *z/OS MVS Programming: Authorized Assembler Services Guide*
   - *z/OS MVS Programming: Authorized Assembler Services Reference ALE-DYN*
   - *z/OS MVS Programming: Authorized Assembler Services Reference EDT-IXG*

- *z/OS MVS Programming: Authorized Assembler Services Reference LLA-SDU*
- *z/OS MVS Programming: Authorized Assembler Services Reference SET-WTO*

2. The default search order can be changed by specifying certain options on the macros used to call programs. The parameters that affect the search order the system will use are EP, EPLOC, DE, DCB, and TASKLIB. For more information about these parameters, see the topic about the search for the load module in *z/OS MVS Programming: Assembler Services Guide*.
3. Some IBM subsystems (notably CICS® and IMS) and applications (such as ISPF) use the facilities described in the above note to establish other search orders for programs.
4. A copy of a module already loaded in virtual storage might not be accessible to another module that needs it. For example, the copy might reside in another address space, or might have been used or be in use and not be reusable or reentrant. Whenever an accessible copy is not available, any module to be used must be loaded. For more information about the system's search order for programs and when modules are usable or unusable, see the information on Program Management in *z/OS MVS Programming: Assembler Services Guide*.

Module placement effect on application performance

Modules begin to run most quickly when all these conditions are true:

- They are already loaded in virtual storage
- The virtual storage they are loaded into is accessible to the programs that call them
- The copy that is loaded is usable
- The virtual storage is backed by central storage (that is, the virtual storage pages containing the programs are not paged out).

Modules that are accessible and usable (and have already been loaded into virtual storage but not backed in central storage) must be returned to central storage from page data sets on DASD or SCM. Modules in the private area and those in LPA (other than in FLPA) can be in virtual storage without being backed by central storage. Because I/O is very slow compared to storage access, these modules will begin to run much faster when they are in central storage.

Modules placed anywhere in LPA are always in virtual storage, and modules placed in FLPA are also always in central storage. Whether modules in LPA, but outside FLPA, are in central storage depends on how often they are used by all the users of the system, and on how much central storage is available. The more often an LPA module is used, and the more central storage is available on the system, the more likely it is that the pages containing the copy of the module will be in central storage at any given time.

LPA pages are only stolen, and never paged out, because there are copies of all LPA pages in the LPA page data set. But the results of paging out and page stealing are usually the same; unless stolen pages are reclaimed before being used for something else, they will not be in central storage when the module they contain is called.

LPA modules must be referenced very often to prevent their pages from being stolen. When a page in LPA (other than in FLPA) is not continually referenced by multiple address spaces, it tends to be stolen. One reason these pages might be stolen is that address spaces often get swapped out (without the PLPA pages to which they refer), and a swapped-out address space cannot refer to a page in LPA.

When all the pages containing an LPA module (or its first page) are not in central storage when the module is called, the module will begin to run only after its first page has been brought into central storage.

Modules can also be loaded into CSA by authorized programs. When modules are loaded into CSA and shared by multiple address spaces, the performance considerations are similar to those for modules placed in LPA. (However, unlike LPA pages, CSA pages must be paged out when the system reclaims them.)

When a usable and accessible copy of a module cannot be found in virtual storage, either the request must be deferred or the module must be loaded. When the module must be loaded, it can be loaded from a VLF data space used by LLA, or from load libraries or PDSEs residing on DASD.

Modules not in LPA must always be loaded the first time they are used by an address space. How long this takes depends on:

- Whether the directory for the library in which the module resides is cached
- Whether the module itself is cached in storage
- The response time of the DASD subsystem on which the module resides at the time the I/O loads the module.

The LLA address space caches directory entries for all the modules in the data sets in the linklist concatenation (defined in PROGxx and LNKLSTxx) by default. Because the directory entries are cached, the system does not need to read the data set directory to find out where the module is before fetching it. This reduces I/O significantly. In addition, unless the system defaults are changed, LLA will use VLF to cache small, frequently-used load modules from the linklist. A module cached in VLF by LLA can be copied into its caller's virtual storage much more quickly than the module can be fetched from DASD.

You can control the amount of storage used by VLF by specifying the MAXVIRT parameter in a COFVLFxx member of PARMLIB. You can also define additional libraries to be managed by LLA and VLF. For more information about controlling VLF's use of storage and defining additional libraries, see *z/OS MVS Initialization and Tuning Reference*.

When a module is called and no accessible or usable copy of it exists in central storage, and it is not cached by LLA, the system must bring it in from DASD. Unless the directory entry for the module is cached, this involves at least two sets of I/O operations. The first reads the data set's directory to find out where the module is stored, and the second reads the member of the data set to load the module. The second I/O operation might be followed by additional I/O operations to finish loading the module when the module is large or when the system, channel subsystem, or DASD subsystem is heavily loaded.

How long it takes to complete these I/O operations depends on how busy all of the resources needed to complete them are. These resources include:

- The DASD volume
- The DASD controller
- The DASD control unit
- The channel path
- The channel subsystem
- The CPs enabled for I/O in the processor
- The number of SAPs (CMOS processors only).

In addition, if cached controllers are used, the reference patterns of the data on DASD will determine whether a module being fetched will be in the cache. Reading data from cache is much faster than reading it from the DASD volume itself. If the fetch time for the modules in a data set is important, you should try to place it on a volume, string, control unit, and channel path that are busy a small percentage of the time, and behind a cache controller with a high ratio of cache reads to DASD reads.

Finally, the time it takes to read a module from a load library (not a PDSE) on DASD is minimized when the modules are written to a data set by the binder, linkage editor, or an IEBCOPY COPYMOD operation when the data set has a block size equal to or greater than the size of the largest load module or, if the library contains load modules larger than 32 kilobytes, set to the maximum supported block size of 32760 bytes.

Access time for modules

From a performance standpoint, modules not already loaded in an address space will usually be available to a program in the least time when found at the beginning of the following list, and will take more time to be available when found later in the list. Remember that the system stops searching for a module once it has been found in the search order; so, if it is present in more than one place, only the first copy found will be used. The placement of the first copy in the search order will affect how long it takes the system to make the module available. Possible places are:

1. LPA

2. Link list concatenation (all directory entries and some modules cached automatically)
3. TASKLIB/STEPLIB/JOBLIB (with LLA caching of the library)
4. TASKLIB/STEPLIB/JOBLIB (without LLA caching of the library).

For best application performance, you should place as many frequently-used modules as high on this list as you can. However, the following system-wide factors must be considered when you decide how many load modules to place in LPA:

- Performance

  When central storage is not constrained, frequently-used LPA routines almost always reside in central storage, and access to these modules will be very fast.
- Virtual Storage

  How much virtual storage is available for address spaces that use the modules placed in LPA, and how much is available for address spaces that do not use the modules placed in LPA.

Module placement effect on system performance

Whether the placement of a module affects system performance depends on how many address spaces use the module and on how often the module is used. Placement of infrequently-used modules that are used by few address spaces have little effect on system-wide performance or on the performance of address spaces that do not use the modules. Placement of frequently-used modules used by a large number of address spaces, particularly those used by a number of address spaces at the same time, can substantially affect system performance.

Placement of modules in LPA

More central storage can be used when a large number of address spaces each load their own copy of a frequently-used module, because multiple copies are more likely to exist in central storage at any time. One possible consequence of increased central storage use is increased paging.

When frequently-used modules are placed in LPA, all address spaces share the same copy, and central storage usage tends to be reduced. The probability of reducing central storage usage increases with the number of address spaces using a module placed in LPA, with the number of those address spaces usually swapped in at any given time, and with how often the address spaces reference the module. You should consider placing in LPA modules used very often by a large number of address spaces.

By contrast, if few address spaces load a module, it is less likely that multiple copies of it will exist in central storage at any one time. The same is true if many address spaces load a module, run it once, and then never run it again, as might happen for those used only when initializing a function or an application. This is also true when many address spaces load a module but use it infrequently, even when a large number of these address spaces are often swapped in at one time; for example, some modules are used only when unusual circumstances arise within an application. Modules that fit these descriptions are seldom good candidates for placement in LPA.

You can add modules to LPA in these ways:

- Add the library containing the modules to the LPA list

  Any library containing only reentrant modules can be added to the LPA list, which places all its modules in LPA. Note that modules are added to LPA from the first place in the LPA list concatenation they are found.
- Add the modules to dynamic LPA

  Use this approach instead of placing modules in FLPA or MLPA whenever possible. Searches for modules in dynamic LPA are approximately as fast as those for modules in LPA, and placement of modules in dynamic LPA imposes no overhead on searches for other modules. Note that another LPA module whose address was stored before a module with the same name was loaded into dynamic LPA might continue to be used.
- Add the modules to FLPA

Modules in the fixed LPA list will be found very quickly, but at the expense of modules that must be found in the LPA directory. It is undesirable to make this list very long because searches for other modules will be prolonged. Note that modules placed in IEAFIXxx that reside in an LPA List data set will be placed in LPA twice, once in PLPA and once in FLPA.

- Add the modules to MLPA

  Like placement in FLPA, placing a large number of modules in MLPA causes searches for all modules to be delayed, and this should be avoided. The delay will be proportional to the number of modules placed in MLPA, and it can become significant if you place a large number of modules in MLPA.

Placement of modules outside LPA

In addition to the effects on central storage, channel subsystem and DASD subsystem load are increased when a module is fetched frequently from DASD. How much it increases depends on how many I/O operations are required to fetch the module and on the size of the module to be fetched.

Module placement effect on virtual storage

When a module is loaded into the private area for an address space, the region available for other things is reduced by the amount of storage used for the module. Modules loaded from anywhere other than LPA (FLPA, MLPA, dynamic LPA, or PLPA) will be loaded into individual address spaces or into CSA.

When a module is added to LPA below 16 megabytes, the size of the explicitly-allocated common area below 16 megabytes will be increased by the amount of storage used for the module. When the explicitly-allocated common area does not end on a segment boundary, IPL processing allocates additional CSA down to the next segment boundary. Therefore, which virtual storage boundaries change when modules are added to LPA depends on whether a segment boundary is crossed or not.

When modules are added to LPA below 16 megabytes, and this does **not** result in the expansion of explicitly-allocated common storage past a segment boundary, less virtual storage will be available for CSA (and SQA overflow) storage. The amounts of CSA and SQA specified in IEASYSxx will still be available, but the system will add less CSA to that specified during IPL.

When the addition of modules to LPA does not result in a reduction in the size of the private area below 16 megabytes, adding load modules to LPA increases the amount of private area available for address spaces that use those load modules. This is because the system uses the copy of the load module in LPA rather than loading copies into each address space using the load module. In this case, there is no change to the private area storage available to address spaces that do not use those load modules.

When modules are added to LPA below 16 megabytes, the growth in LPA can cause the common area below 16 megabytes to cross one or more segment boundaries, which will reduce the available private area below 16 megabytes by a corresponding amount; each time the common area crosses a segment boundary, the private area is reduced by the size of one segment. The segment size in z/OS is one megabyte.

When the size of the private area is reduced as a result of placing modules in LPA below 16 megabytes, the private area virtual storage available to address spaces that use these modules might or might not be changed. For example, if an address space uses 1.5 megabyte of modules, all of them are placed in LPA below 16 megabytes, and this causes the common area to expand across two segment boundaries, .5 megabytes less private area storage will be available for programs in that address space. But if adding the same 1.5 megabytes of modules causes only one segment boundary to be crossed, .5 megabytes more will be available, and adding exactly 1 megabytes of modules would cause no change in the amount of private area storage available to programs in that address space. (These examples assume that no other changes are made to other common virtual storage area allocations at the same time.)

When the size of the private area is reduced as a result of placing modules in LPA below 16 megabytes, less storage will be available to all address spaces that do **not** use those modules.

A process similar to the process described for LPA is used when ELPA, the other Extended common areas, and the Extended private area are built above 16 megabytes. The only difference is that common storage areas above 16 megabytes are built from 16 megabytes upward, while those below 16 megabytes are built from 16 megabytes downward.

Modules can also be loaded in CSA, and some subsystems (like IMS) make use of this facility to make programs available to multiple address spaces. The virtual storage considerations for these modules are similar to those for LPA.

Recommendations for Improving System Performance

The following recommendations should improve system performance. They assume that the system's default search order will be used to find modules. You should determine what search order will be used for programs running in each of your applications and modify these recommendations as appropriate when other search orders will be used to find modules.

- Determine how much private area, CSA, and SQA virtual storage are required to run your applications.
- Determine which modules or libraries are important to the applications you care most about. From this list, determine how many are reentrant to see which are able to be placed in LPA. Of the remaining candidates, determine which can be safely placed in LPA, considering security and system integrity.

  Note: All modules placed in LPA are assumed to be authorized. IBM publications identify libraries that can be placed in the LPA list safely, and many list modules you should consider placing in LPA to improve the performance of specific subsystems and applications.

  Note that the system will try to load RMODE(ANY) modules above 16 megabytes whenever possible. RMODE(24) modules will always be loaded below 16 megabytes.
- To the extent possible without compromising required virtual storage, security, or system integrity, place libraries containing a high percentage of frequently-used reentrant modules (and containing no modules that are not reentrant) in the LPA list. For example, if TSO/E response time is important and virtual storage considerations allow it, add the CMDLIB data set to the LPA list.
- To the extent possible without compromising available virtual storage, place frequently or moderately-used refreshable modules from other libraries (like the linklist concatenation) in LPA using dynamic LPA or MLPA. Make sure you do not inadvertently duplicate modules, module names, or aliases that already exist in LPA. For example, if TSO/E performance is important, but virtual storage considerations do not allow CMDLIB to be placed in the LPA list, place only the most frequently-used TSO/E modules on your system in dynamic LPA.

  Use dynamic LPA to do this rather than MLPA whenever possible. Modules that might be used by the system before a SET PROG command can be processed cannot be placed solely in dynamic LPA. If these modules are not required in LPA before a SET PROG command can be processed, the library in which they reside can be placed in the linklist so they are available before a SET PROG can be processed, but enjoy the performance advantages of LPA residency later. For example, Language Environment® runtime modules required by z/OS UNIX System Services initialization can be made available by placing the SCEERUN library in the linklist, and performance for applications using Language Environment (including z/OS UNIX System Services) can be improved by also placing selected modules from SCEERUN in dynamic LPA.

  For more information about dynamic LPA, see the information about PROGxx in *z/OS MVS Initialization and Tuning Reference*. For information about MLPA, see the information about IEALPAxx in *z/OS MVS Initialization and Tuning Reference*.

  To load modules in dynamic LPA, list them on an LPA ADD statement in a PROGxx member of PARMLIB. You can add or remove modules from dynamic LPA without an IPL using SET PROG=xx and SETPROG LPA operator commands. For more information, *z/OS MVS Initialization and Tuning Reference* and *z/OS MVS System Commands*.

  Note: If an application uses the CHKPT macro and dynamic LPA modules are in use at the time the checkpoint is taken, dynamic LPA must not be altered until you no longer intend to attempt a restart from the identified checkpoint. When a dynamic LPA module is detected on the load list, the CHKPT macro returns RC=16, RSN=64. If dynamic LPA was altered and the module has a different entry point address in dynamic LPA at deferred checkpoint restart, the restart fails with ABEND S13F. For more information about checkpoint/restart facilities, see *z/OS DFSMSdfp Checkpoint/Restart*.
- By contrast, do not place in LPA infrequently-used modules, those not important to critical applications (such as TSO/E command processors on a system where TSO/E response time is not important), and

low-use user programs when this placement would negatively affect critical applications. Virtual storage is a finite resource, and placement of modules in LPA should be prioritized when necessary. Leaving low-use modules from the linklist (such as those in CMDLIB on systems where TSO/E performance is not critical) and low-use application modules outside LPA so they are loaded into user subpools will affect the performance of address spaces that use them and cause them to be swapped in and out with those address spaces. However, this placement usually has little or no effect on other address spaces that do not use these modules.

- If other measures (like WLM policy changes, managing the content of LPA, and balancing central and expanded storage allocations) fail to control storage saturation, and paging and swapping begin to affect your critical workloads, the most effective way to fix the problem is to add storage to the system. Sometimes, this is as simple as changing the storage allocated to different LPARs on the same processor. You should consider other options only when you cannot add storage to the system. For additional paging flexibility and efficiency, you can add optional storage-class memory (SCM) on Flash Express® solid-state drives (SSD) or the Virtual Flash Memory (VFM) as a second type of auxiliary storage. DASD auxiliary storage is required. For details refer to “Using storage-class memory (SCM)” on page 51.

Modified link pack area (MLPA/Extended MLPA)

This area may be used to contain reenterable routines from either APF-authorized or non-APF-authorized libraries that are to be part of the pageable extension to the link pack area during the current IPL. Any module in the modified link pack area will be treated by the system as though it came from an APF-authorized library. Ensure that you have properly protected any library named in IEALPAxx to avoid system security and integrity exposures, just as you would protect any APF-authorized library.

The MLPA exists only for the duration of an IPL. Therefore, if an MLPA is desired, the modules in the MLPA must be specified for each IPL (including quick start and warm start IPLs).

The MLPA is allocated just below the FLPA (or the PLPA, if there is no FLPA); the extended MLPA is allocated above the extended FLPA (or the extended PLPA if there is no extended FLPA). When the system searches for a routine, the MLPA is searched before the PLPA.

Note: Loading a large number of modules in the MLPA can increase fetch time for modules that are not loaded in the LPA. This could affect system performance.

The MLPA can be used at IPL time to temporarily modify or update the PLPA with new or replacement modules. No actual modification is made to the quick start PLPA stored in the system's paging data sets. The MLPA is read-only, unless NOPROT is specified on the MLPA system parameter.

Specified by:

- Including a module list as an IEALPAxx member of SYS1.PARMLIB; where xx is the specific list.
- Including the MLPA system parameter in IEASYSxx or specifying MLPA from the operator's console during system initialization.

Common service area (CSA/Extended CSA)

This area contains pageable and fixed data areas that are addressable by all active virtual storage address spaces. CSA normally contains data referenced by a number of system address spaces, enabling address spaces to communicate by referencing the same piece of CSA data.

CSA is allocated directly below the MLPA; extended CSA is allocated directly above the extended MLPA. If the virtual SQA space is depleted, the system will allocate additional SQA space from the CSA.

Specified by:

- The SYSP parameter at the operator's console to specify an alternative system parameter list (IEASYSxx) that contains a CSA specification.
- The CSA parameter at the operator's console during system initialization. This value overrides the current system parameter value for CSA that was established by IEASYS00 or IEASYSxx.

Note: If the size allocated for extended SQA is too small or is used up very quickly, the system attempts to steal space from extended CSA. When both extended SQA and extended CSA are used up, the system allocates space from SQA and CSA below 16 megabytes. The allocation of this storage could eventually lead to a system failure. Ensuring the appropriate size of extended SQA and extended CSA storage is critical to the long-term operation of the system.

SQA/CSA shortage thresholds

Ensuring the appropriate size of extended SQA and extended CSA storage is critical to the long-term operation of the system. If the size allocated for extended SQA is too small or is used up very quickly, the system attempts to use extended CSA. When both extended SQA and extended CSA are used up, the system allocates space from SQA and CSA below 16 megabytes. The allocation of this storage could eventually lead to a system failure.

- When the size, in bytes, of combined total of free SQA + CSA pages falls below the "high insufficient" threshold, the system issues message IRA100E
- If the size, in bytes, of available SQA and SQA pages falls below the "low insufficient" threshold, the system issues message IRA101E

For more information about SQA shortages and the thresholds, see “SQA/CSA thresholds” on page 18.

Restricted use common service area (RUCSA/Extended RUCSA)

IBM recommends the elimination of all user-key (8 - 15) common storage, as it creates a security risk because the storage can be modified or referenced, even if fetch protected, by any unauthorized program from any address space.

For earlier releases, IBM has provided APARs OA53355, OA56180, and OA61368 to assist with identifying software programs that use user-key common storage. For those who cannot immediately eliminate all affected software programs or who need additional assistance in identifying all programs that reference user-key CSA storage, the more secure restricted use common service area (RUCSA) provided in earlier releases by the PTF for APAR OA56180 and as an optional, priced feature in z/OS V2R4 can be used. However, IBM still recommends the elimination of all user-key common storage.

RUCSA is a separate area from CSA that is situated between the CSA and PVT areas. Because it is a separate area, it can be managed as a secure resource. The security administrator can define, via the System Authorization Facility (SAF), which users have access to the RUCSA. Only those with SAF READ authority to the IARRSM.RUCSA profile in the FACILITY class can have access. Once defined, all requests for user key CSA storage will transparently obtain storage from the RUCSA. Application changes might not be necessary.

RUCSA (and extended RUCSA) are similar to CSA (and extended CSA) in that their storage ranges reduce the size of the 24-bit and 31-bit private area ranges for all address spaces. However, RUCSA (and extended RUCSA) differ in the following ways:

- Only required by installations that have programs that require them.
- Exclusively used as a user-key CSA area.
- Accessible only from address spaces that are running under user IDs that have SAF READ authority to the IARRSM.RUCSA profile in the FACILITY class, or, on z/OS V2R3 or earlier systems that have the VSM ALLOWUSERKEYCSA(YES) parameter specified in the DIAGxx member of parmlib. To ensure that a job has a consistent SAF authority while it is running, the authorization is checked at job start and not altered until the job ends. As such, a never-ending job that requires SAF authorization after it starts must be restarted.
- Never converted to SQA or extended SQA.

The RUCSA is allocated just below the CSA, and the extended RUCSA is allocated above the extended CSA. They are specified in the following ways:

- The SYSP parameter at the operator's console to specify an alternative system parameter list (IEASYSxx) that contains a RUCSA specification.

- The RUCSA parameter at the operator's console during system initialization. This value overrides the current system parameter value for RUCSA that was established in the IEASYSxx member.

Note: Unless otherwise noted in z/OS documentation, consider RUCSA and extended RUCSA areas to be part of the CSA and extended CSA areas.

Considerations for RUCSA

Consider the following points to evaluate whether RUCSA meets your needs:

- As RUCSA is one resource that is shared by multiple users, it does not prevent different SAF-authorized applications from altering or referencing each other's RUCSA storage.
- RUCSA storage can only be specified in 1 MB increments. There is an approximate total of 16 MB of storage in the non-extended area; therefore, when a non-extended RUCSA is required, adding 1 MB might not be acceptable, as it might not leave enough non-extended CSA for system-key users or non-extended private area for all address spaces.
- RUCSA is only used for user-key CSA storage.
  - Unlike CSA, SQA cannot overflow into RUCSA storage.
  - RUCSA cannot be used for system-key CSA.
  - System-key CSA cannot be used for RUCSA.
  - It is possible for you to define a larger overall common area in order to accommodate peaks in user-key CSA (now RUCSA) and CSA usage at different times. A larger common area reduces the size of all the private areas.
- RUCSA programming considerations:
  - RUCSA storage cannot be shared by using the IARVSERV SHARE service. RUCSA is common storage and does not need to be explicitly shared. Attempts to issue IARVSERV SHARE on RUCSA storage will be abended (ABEND-6C5 RSN-*xx*0132*xx*) regardless of having SAF READ authority to the IARRSM.RUCSA resource in the FACILITY class.
  - Unlike CSA storage which all address spaces share, first references to fixed RUCSA storage by instructions that do not cause a page fault (such as LRA, LRAY, and TPROT) from an address space other than the obtainer's address space will fail with a condition code indicating that the storage is not valid. Issuers of such instructions must ensure that a page fault has occurred on RUCSA storage from the referencing address space prior to issuing the instruction.

Migrating to RUCSA

IBM recommends the following steps to properly use RUCSA storage.

Procedure

1. Determine the maximum amount of user-key CSA storage that you want to define with the RUCSA parameter. Consider reducing the CSA size by what is required for RUCSA.

   Like all the common areas, ensuring the appropriate size of the RUCSA and extended RUCSA is critical to the long-term operation of the system. Regardless of whether or not a RUCSA is defined, you can determine the maximum required size by using the following resources:

   - The RMF Monitor I VSTOR (or equivalent) report. In the ALLOCATED CSA BY KEY section of the report, use the values under the **MAX** columns in the **8-F** row to size the (BELOW 16M) and EXTENDED (ABOVE 16M) RUCSA.
   - The following high-water marks:
     - GDA_RUCSA_HWM contains the high-water mark for use of non-extended user-key CSA storage.
     - GDA_ERUCSA_HWM contains the high-water mark for use of extended user-key CSA storage.

     You can view the high-water marks in IPCS from active system storage. Ensure that the dump source is set to ACTIVE and issue the CBF GDA command. Control block names RUCSAHWM and

ERUCSAHW contain the high-water marks for non-extended and extended user-key CSA storage, respectively.

2. Determine the effects on programs that are running in your environment.
   a) Apply any required monitoring program service and be prepared for any report differences.
   b) Ensure software that uses RUCSA abides by the programming interface differences outlined in "Considerations for RUCSA" on page 28.
3. Use SAF universal read access to authorize all users to have access to RUCSA.

   The following example shows the commands to define the resource profile and provide universal read access:

```
RDEFINE FACILITY IARRSM.RUCSA UACC(READ)
SETROPTS CLASSACT(FACILITY)
SETROPTS RACLIST(FACILITY) REFRESH
```

4. Follow the instructions in *z/OS Planning for Installation* to acquire licensing for the RUCSA feature and update the product enablement policy in the IFAPRDxx member of parmlib.

   Note: You must complete these actions *before* you proceed to the next step, as z/OS V2R4 requires a license to use the RUCSA feature.
5. With VSM ALLOWEARLYRUCSA(YES) specified in the DIAGxx member of parmlib, IPL the system with an IEASYSxx member that contains a RUCSA specification that matches the sizes determined in step "1" on page 28.

   Defining a RUCSA will cause allocations to come from RUCSA and allow the system to monitor user-key CSA storage usage. As long as all users have SAF READ access to RUCSA and any relevant issues that you identified in earlier steps have been mitigated, usage of user-key CSA storage is not restricted even though a RUCSA is defined.
6. Authorize those users who truly require access to user-key CSA storage.
   a) Determine who should be using user-key CSA storage.
      - The SMF30_UserKeyRucsaUsage field in SMF type 30 records reports all attempted uses of RUCSA storage. The SMF30_UserKeyCsaUsage field provided by APAR OA53355 only indicates attempts to obtain user-key CSA storage and is set whether or not a RUCSA is defined.
      - Using the SMF type 30 records, create a list of user IDs that truly require access to RUCSA.
   b) Authorize the required user IDs by permitting SAF READ authority to the IARRSM.RUCSA resource profile in the FACILITY class.

      These will be the only users that can obtain and access storage from RUCSA.

      The following example shows the commands to define the SAF resource profile and authorize a single user ID:

```
PERMIT IARRSM.RUCSA CLASS(FACILITY) ID(user1) ACCESS(READ)
SETROPTS CLASSACT(FACILITY)
SETROPTS RACLIST(FACILITY) REFRESH
```

   c) Authorization changes only take effect when a job starts; therefore, ensure that all users (jobs) identified as having used RUCSA have been recycled or ended to ensure that no unauthorized users still have access to RUCSA.
7. When you are comfortable with restricting non-SAF authorized user IDs from using RUCSA, change the SAF UACC value to NONE.
   The following example shows the commands to do this:

```
RALTER FACILITY IARRSM.RUCSA UACC(NONE)
SETROPTS RACLIST(FACILITY) REFRESH
```

   - Prior to removing SAF universal READ access, all user-key CSA creations, accesses, deletions, IARVSERV CHANGEACCESS requests, and PGSER PROTECT and UNPROTECT requests will be successful.

- After removing SAF universal READ access, only SAF-authorized user IDs can obtain or access user-key CSA storage. Other users will be abnormally ended, as follows:
  - ABEND-0C4 RSN-10 will be issued to indicate access failures.
  - ABEND-B*xx* RSN-5C will be issued to indicate obtain or release failures, where *xx* is 04, 05, 0A, or 78, depending on whether the request is from the GETMAIN, FREEMAIN, STORAGE, or CPOOL service.
  - ABEND-6C5 RSN-*xx*0340*xx* will be issued to indicate IARVSERV CHANGEACCESS failures.
  - ABEND-18A RSN-*xxxxxxxx* will be issued to indicate PGSER failures, where *xxxxxxxx* is *xx*0705*xx* or *xx*0805*xx*, depending on whether it is a PROTECT or UNPROTECT request.

8. Determine whether VSM AllowEarlyRucsa(YES) is required.

   The SMF30_RUCSAEarlyUsage field in SMF type 30 records reports all attempted uses of storage from RUCSA that were made by address spaces that were authorized for RUCSA during early IPL or early started task initialization due to the VSM AllowEarlyRucsa(Yes) statement in DIAGxx. If the SMF30_RUCSAEarlyUsage field has not been set in any SMF type 30 records, remove the VSM AllowEarlyRucsa statement from DIAGxx.

Local system queue area (LSQA/Extended LSQA)

Each virtual address space has an LSQA. The area contains tables and queues associated with the user's address space.

LSQA is intermixed with SWA and subpools 229, 230, and 249 downward from the bottom of the CSA into the unallocated portion of the private area, as needed. Extended LSQA is intermixed with SWA and subpools 229, 230, and 249 downward from 2 gigabytes into the unallocated portion of the extended private area, as needed. (See Figure 3 on page 16.) LSQA will not be taken from space below the top of the highest storage currently allocated to the private area user region. Any job will abnormally terminate unless there is enough space for allocating LSQA.

Address space layout randomization

Address space layout randomization (ASLR) is a technique that is used to increase the difficulty of performing a buffer overflow attack that requires the attacker to know the location of an executable in memory. A buffer overflow vulnerability is a flaw in software written in a memory-unsafe programming language, such as C. Such a flaw is characterized by a failure of an application to validate the size of user input data that is written to memory. An application can remedy this flaw by checking the length of the user input data and throwing an exception or issuing an error message if the actual length does not match the expected length.

z/OS provides options to enable ASLR for 24-bit and 31-bit low private storage as well as for 64-bit private storage. When enabled, the feature affects all storage allocations in the specified storage ranges (not just executables). Common storage, 24- and 31-bit high private storage (including LSQA), high virtual shared, the high virtual local system area and the 2G-64G area are unaffected.

Enabling ASLR

By default, ASLR is disabled. To enable ASLR, either IPL the system using a DIAGxx member that specifies the ASLR option or issue the SET DIAG=xx command after IPL. If you enable ASLR after IPL, only those jobs that are subsequently started and that are not exempt from ASLR will have ASLR enabled. The ASLR enablement options provide a way to restrict ASLR to subsets of address spaces where it is less likely to cause a storage constraint issue. For details about the options, see the description of the ASLR parameter in DIAGxx parmlib member in *z/OS MVS Initialization and Tuning Reference*.

Impact of ASLR on virtual storage

The decision to enable ASLR represents a tradeoff between enhanced security and a reduction in the amounts of available 24-bit, 31-bit, and 64-bit private storage. When enabled for 24- and 31-bit virtual storage, the size of available private storage will be reduced by up to 63 pages and 255 pages,

respectively. A job's requested region size must still be satisfied from within the reduced private area in order for the job to be started. Jobs whose region size cannot be satisfied will result in an ABEND 822. If a job's requested region size is satisfied, it is still possible that the reduced size of private storage prevents the job from completing, resulting in an ABEND 878.

One way to determine whether jobs would not be able to run under the constrained size of 24- or 31-bit private storage that would occur with ASLR enabled is to specify a larger value for the CSA parameter in parmlib. Increasing the sizes of both 24- and 31-bit CSA by 1M effectively reduces the sizes of 24- and 31-bit private storage by 1M, which is greater than the maximum reduction that would occur under ASLR.

Excluding individual address spaces from ASLR

When ASLR is enabled, you can use SAF authorization to exempt selected address spaces from ASLR. To do this, permit SAF READ authority to the IARRSM.EXEMPT.ASLR.*jobname* resource in the FACILITY class to fully exempt the job or to the IARRSM.EXEMPT.ASLR24.*jobname* resource to exempt the job from only 24-bit ASLR. The following example shows the set of commands to fully exempt a job from ASLR:

```
RDEFINE FACILITY IARRSM.EXEMPT.ASLR.jobname UACC(READ)
SETROPTS CLASSACT(FACILITY)
SETROPTS RACLIST(FACILITY) REFRESH
```

Certain system address spaces, such as MASTER, which initialize early during the IPL process are not randomized. High virtual storage may not be randomized in address spaces with initialization exits that obtain high virtual storage. Job steps that obtain high virtual storage and assign it to a task not within the program task tree of that job step limit the ability of the system to set up randomization for the next job step if the obtained storage persists across job steps.

Large pages and LFAREA

MVS supports the use of 1 MB and 2 GB large pages. Using large pages can improve performance for some applications by reducing the overhead of dynamic address translation. This is achieved by each large page requiring only one entry in the Translation Look-aside Buffer (TLB), as compared to the larger number of entries required for an equivalent number of 4 KB pages. A single TLB entry improves TLB coverage for exploiters of large pages by increasing the hit rate and decreasing the number of TLB misses that an application incurs.

Attention: Large pages are a performance improvement feature for some cases; switching to large pages is not recommended for all workloads.

Large pages provide performance value to a select set of applications that can generally be characterized as memory access-intensive and long-running. These applications meet the following criteria:

1. They must reference large ranges of memory.
2. They tend to exhaust the private storage areas available within the 2 GB address space (such as the IBM WebSphere® application), or they use private storage above the 2 GB address space (such as IBM DB2 software).

Displaying LFAREA information

You can use the MODIFY AXR,IAXDMEM command to display memory utilization statistics associated with the large frame area.

For more information, see Displaying real storage memory statistics in *z/OS MVS System Commands*.

LFAREA parameter

The IEASYSxx LFAREA parameter specifies the maximum amount of real storage that can be used to satisfy fixed 1 MB and 2 GB page requests below 4 TB. All fixed 1 MB and 2 GB pages are backed by contiguous 4 KB real storage frames. Specifying the LFAREA parameter for fixed 1 MB pages does not reserve an amount of real storage to be used exclusively for fixed 1 MB page requests. If the system becomes constrained by a lack of sufficient 4 KB frames to handle workload demand, it can use any

available real storage that is specified by the LFAREA 1M parameter to back 4 KB page requests, enabling the system to react dynamically to changing system storage frame requirements.

If 1 MB large frames are required but none are currently available, the system attempts to defragment real storage to form 1 MB large frames. However, frequent defragmentation attempts can indicate a system configuration and tuning issue. To resolve this issue, you can either decrease the size of the LFAREA to prevent 1 MB fixed usage, adjust the workload to reduce the demand for 4 KB frames, or add more real storage.

Because the IEASYSxx LFAREA parameter requires an IPL to change the LFAREA value, the following considerations apply when determining LFAREA:

- All online real storage above 4 TB is part of the 2 GB LFAREA in addition to any specified by the LFAREA keyword.
- If the value specified for LFAREA is too small, available 1 MB and 2 GB pages might not exist for applications that might benefit from large page usage.
- The LFAREA request can be specified with target and minimum values for 2 GB and 1 MB pages. Additionally, a NOPROMPT option allows the IPL to continue with no LFAREA with no operator intervention even if the minimum value is not met.
- Determine the total number of 1 MB and 2 GB pages that your applications require, and consider using this number as a starting point (minus any available 2 GB pages above 4 TB) for your LFAREA target values. Also, add a small amount of 1 MB pages to your starting point for use by the system. Use your existing performance monitor to determine how much 1 MB pages are used by your system.
- Use output from the MODIFY AXR,IAXDMEM system command as an estimate for the maximum number of 1 MB pages used on behalf of fixed 1 MB page requests. Use this estimate to determine if your LFAREA value is too small and see the IAR049I message in *z/OS MVS System Messages, Vol 6 (GOS-IEA)* for additional details.

See the following documentation for additional details about LFAREA:

- For specific details on specifying the IEASYSxx LFAREA parameter, see LFAREA parameter in IEASYSxx in *z/OS MVS Initialization and Tuning Reference*.
- For information on calculating the LFAREA value based on Db2 requests, see *IBM Db2 10 for z/OS Managing Performance*.
- For information about calculating the LFAREA value based on JAVA heaps, see *IBM SDK for z/OS, Java Technology Edition*.

LFAREA syntax and examples

Specifying the LFAREA parameter is done using one of the two separate and distinct syntax methods and percentage calculation formulas, which are shown in Table 4 on page 33. Following Table 4 on page 33, the subsequent tables show examples of specific LFAREA calculations and specifications.

Attention: All LFAREA calculation and specification examples are examples only, and are never to be used as a substitute for the specific calculations and specifications that are required for your z/OS system.

Table 4 on page 33 describes the two LFAREA syntax methods, and the formulas for the percentage specification.

Note: The LFAREA parameter is also described in *z/OS MVS Initialization and Tuning Reference*.

Table 4. The two supported LFAREA syntax methods

Syntax	Percentage formula (optional)	Usage notes
LFAREA=*x*M|*x*G|*x*T|*x*%	If a percentage (*x*%) is specified, the system calculates the requested maximum amount of real storage that can be used to satisfy fixed 1 MB page requests using the following formula: `Maximum number of 1 MB pages that can be used to satisfy fixed 1 MB page requests= (x% * online real storage at IPL in megabytes up to 4 TB) - 2048 MB` **Note:** The resulting 1 MB number of pages is rounded down to the next whole number of 1 MB pages (which is zero for values less than 1 MB).	Consider the following usage notes before using this syntax method: • The variable *x* specifies the LFAREA size in megabytes (M), gigabytes (G), or terabytes (T), or as a percentage (%). • This syntax method can only be used to specify the maximum amount of real storage that can be used to satisfy fixed 1 MB page requests. To specify the maximum amount of real storage up to 4 TB that can be used to satisfy 2 GB page requests, you must use the alternative syntax method which specifies 1M= and 2G= values.

LFAREA=(1M=(*target*[%],*minimum*[%]), 2G=(*target*[%],*minimum*[%]))	If percentages (*target*% and *minimum*%) are specified, the requested target and minimum number of 1 MB pages that can be used to satisfy fixed 1 MB page requests are calculated using the formula:
	`Maximum number of 1 MB pages to satisfy fixed 1 MB page requests = (target% or minimum%) * (online real storage at IPL in megabytes up to 4 TB - 4096 MB)`
	If percentages (*target*% and *minimum*%) are specified, the requested target or minimum number of 2 GB pages that can be used to satisfy 2 GB page requests is calculated using the formula:
	`number of 2 GB pages to reserve = target% or minimum% * (online real storage at IPL in 2 gigabytes up to 4 TB - 4 GB)`
	Note: The resulting 1 MB or 2 GB number of pages is rounded down to the next respective 1 MB or 2 GB whole number of pages (which is zero for values less than 1 MB or 2 GB).
		Consider the following usage notes before using this syntax method:
		• Any online real storage above 4 TB at IPL is 2 GB frames and not included in these calculations.
		• The *target* and *minimum* values specify either the number of pages or the percentage of online real storage at IPL as calculated using the percentage formula.
		• This syntax method can be used to specify the maximum amount of real storage to be used to satisfy both fixed 1 MB and 2 GB page requests.
		• You cannot combine both fixed and percentage *target* and *minimum* values within each 1 MB or 2 GB specification.

LFAREA calculation example 1

Table 5 on page 34 shows an example of the maximum amount of LFAREA that can be configured for various amounts of online real storage using the LFAREA=(1M=(*target*,*minimum*),2G=(*target*,*minimum*)) syntax. The maximum amount is calculated using the formula: 80% of (online storage at IPL up to 4 TB minus 4GB). Also shown are the smallest percentages that can be specified for an LFAREA request for 1 MB or 2 GB pages to satisfy at least one 1 MB page or one 2 GB page request, respectively.

For example, on a z/OS system with 16 GB of online storage at IPL, LFAREA=(2G=17%) must be specified to satisfy at least one 2 GB page request. This is calculated as *percentage* * (*online storage at IPL* – 4GB) = 0.17 * (16GB – 4GB) = 0.17 * 12GB = 2.04 GB, which is rounded down to the 2 GB boundary at 2 GB. To satisfy this request, the system must have 2 GB of contiguous real storage on a 2 GB boundary above the bar. The request is refused if there are offline storage increments that create discontiguous areas which prevent contiguous 2 GB pages from being formed.

Table 5. LFAREA calculation example 1

Amount of online real storage at IPL in gigabytes (GB)	**Maximum amount available for LFAREA in gigabytes (GB)**	**Minimum percentage request for at least one 1 MB page**	**Minimum percentage request for at least one 2 GB page**
4	0	Not applicable	Not applicable
4.25	0.2	1% (results in two 1 MB pages)	Not applicable
6.5	2	1% (results in 25 1 MB pages)	80%
6.75	2.2	1% (results in 28 1 MB pages)	79%
8	3.2	1% (results in 40 1 MB pages)	50%
16	9.6	1% (results in 122 1 MB pages)	17%
32	22.4	1% (results in 286 1 MB pages)	8%
64	48	1% (results in 614 1 MB pages)	4%
128	99.2	1% (results in 1269 1 MB pages)	2%
208	163.2	1% (results in 2088 1 MB pages)	1%
256	201.6	1% (results in 2580 1 MB pages)	1%
512	406.4	1% (results in 5201 1 MB pages)	1% (results in two 2 GB pages)
1024	816	1% (results in 10444 1 MB pages)	1% (results in five 2 GB pages)
2048	1635.2	1% (results in 20930 1 MB pages)	1% (results in ten 2 GB pages)
4096	3273.6	1% (results in 41902 1 MB pages)	1% (results in twenty 2 GB pages

Table 5. LFAREA calculation example 1 (continued)			
Amount of online real storage at IPL in gigabytes (GB)	**Maximum amount available for LFAREA in gigabytes (GB)**	**Minimum percentage request for at least one 1 MB page**	**Minimum percentage request for at least one 2 GB page**
8192	3273.6	1% (results in 41902 1 MB pages)	1% (results in twenty 2 GB pages below 4 TB plus 2048 2G pages in the area from 4 TB to 8 TB)

Note the following additional points regarding Table 5 on page 34:

- 4 GB is not enough online real storage to support fixed 1 MB or 2 GB page requests.
- 4.25 GB is the next incremental size up from 4096 MB (using an increment size of 256 MB) and the smallest amount of online real storage that can be configured for LFAREA=(1M=1%) to satisfy at least one 1 MB page request (in this case two 1 MB page requests).
- 6.5 GB is the smallest amount of online real storage that can be configured to satisfy at least one 2 GB page request. Note that LFAREA=(2G=80%) is required to satisfy that one 2 GB page request, leaving no storage available for 1 MB pages (because the sum of 1 MB and 2 GB pages must not exceed 80% of online real storage).
- 6.75 GB is the next incremental size up from 6656 MB (using an increment size of 256 MB) and the smallest amount of online real storage that can be configured for LFAREA=(1M=1%,2G=79%) to satisfy at least one 1 MB page and one 2 GB page request (in this case 28 1 MB page requests).
- 208 GB is the smallest amount of online real storage for LFAREA=(2G=1%) to satisfy one 2 GB page request (below 208 GB, a percentage higher than 1% is required to satisfy at least one 2 GB page request).
- The LFAREA specification only applies to storage up to 4 TB. In addition to any large frames that result due to the LFAREA specification, all storage above 4 TB is added to the 2 GB large frame area.

LFAREA calculation examples 2 and 3

The specific numbers of 1 MB pages and 2 GB pages that your system requires depend on multiple factors. One factor is the number of 1 MB pages and 2 GB pages that exploiting applications require for various workloads to gain a performance benefit. The following LFAREA specification examples illustrate how various amounts of online storage at IPL affect the resulting number of 1 MB and 2 GB pages.

Table 6 on page 35 and Table 7 on page 36 illustrate a simplified approach, using a LFAREA percentage specification, that works for any amount of online real storage without operator intervention. The specified *minimum* percentage of 0% allows the system to continue the IPL after setting the cap of the most 1 MB and 2 GB pages that can be used to satisfy both fixed 1 MB and 2 GB page requests, up to the specified *target* percentages. For these examples, some amount of real storage will always be used to satisfy fixed 1 MB page requests when the online real storage at IPL is above 4 GB. However, real storage will only be used for 2 GB page requests when the amount of online real storage at IPL is sufficiently large. Note that Table 7 on page 36 doubles the requested percentage, which provides a similar number of 1 MB and 2 GB pages at lower amounts of online storage.

Table 6. LFAREA calculation example 2		
LFAREA=(1M=(10%,0%),2G=(10%,0%))		
Amount of online real storage at IPL in gigabytes (GB)	**Resulting number of 1 MB pages**	**Resulting number of 2 GB pages**
2	0	0
4	0	0
8	409	0

Table 6. LFAREA calculation example 2 (continued)		
LFAREA=(1M=(10%,0%),2G=(10%,0%))		
Amount of online real storage at IPL in gigabytes (GB)	**Resulting number of 1 MB pages**	**Resulting number of 2 GB pages**
16	1228	0
32	2867	1
64	6144	3
128	12697	6
256	25804	12
512	52019	25
1024	104448	51
2048	209305	102
4096	419020	204
8192	419020	2252 (204 from below 4 TB and 2048 from above 4 TB)
16384	419020	6348 (204 from below 4 TB and 6144 from above 4 TB)

Table 7. LFAREA calculation example 3		
LFAREA=(1M=(20%,0%),2G=(20%,0%))		
Amount of online real storage at IPL in gigabytes (GB)	**Resulting number of 1 MB pages**	**Resulting number of 2 GB pages**
2	0	0
4	0	0
8	819	0
16	2457	1
32	5734	2
64	12288	6
128	25395	12
256	51609	25
512	104038	50
1024	208896	102
2048	418611	204
4096	838041	409
8192	838041	2457 (409 from below 4 TB and 2048 from above 4 TB)
16384	838041	6553 (409 from below 4 TB and 6144 from above 4 TB)

LFAREA calculation example 4

Table 8 on page 37 shows a series of LFAREA requests on a z/OS system with 64 GB of online real storage at IPL. The 1 MB pages are requested with *target* and *minimum* percentages of 40% and 20%, respectively, while the 2 GB pages are requested as a *target* numerical value that is increased with each request. This illustrates how the 1 MB pages are reduced toward the minimum to satisfy the requested target number of 2 GB pages. Table 8 on page 37 also shows that as the number of 2 GB pages reaches 19, the required reduction in 1 MB pages would be below the requested minimum. In this case, the system prompts to re-specify LFAREA. Also shown is a case where the requested number of 2 GB pages is 25, which by itself exceeds the 80% system limit and results in a prompt to re-specify LFAREA.

Table 8. LFAREA calculation example 4

LFAREA request with 64 GB of online real storage available at IPL	**Resulting number of 1 MB pages**	**Resulting number of 2 GB pages**
LFAREA=(1M=(40%,20%),2G=11	24576 (40%)	11
LFAREA=(1M=(40%,20%),2G=12	24576 (40%)	12
LFAREA=(1M=(40%,20%),2G=13	22118 (36%)	13
LFAREA=(1M=(40%,20%),2G=14	20275 (33%)	14
LFAREA=(1M=(40%,20%),2G=15	18432 (30%)	15
LFAREA=(1M=(40%,20%),2G=16	15974 (26%)	16
LFAREA=(1M=(40%,20%),2G=17	14131 (23%)	17
LFAREA=(1M=(40%,20%),2G=18	12288 (20%)	18
LFAREA=(1M=(40%,20%),2G=19	Unable to satisfy 1 MB minimum	Unable to satisfy 1 MB minimum
LFAREA=(1M=(40%,20%),2G=25	Above 80% limit	Above 80% limit

LFAREA calculation example 5

In comparison to Table 8 on page 37, Table 9 on page 37 also shows a series of LFAREA requests on a system with 64 GB of online real storage at IPL. The 1 MB pages are again requested with *target* and *minimum* percentages of 40% and 20%, respectively. The 2 GB pages, however, are requested as *minimum* and *target* numerical values that are increased with each request. This example illustrates how the 1 MB pages are again reduced toward the minimum to satisfy the target number of 2 GB pages, but when the 1 MB minimum cannot be reduced further, the 2 GB request is reduced toward its minimum. As shown in Table 9 on page 37, when the number of 2 GB pages reaches 19, the required reduction in 1 MB pages is below the requested minimum. At this point, the 2 GB request is reduced to 18, which is still above its minimum. When the 2 GB request reaches 21, it can no longer be satisfied by reducing either the 1 MB request or the 2 GB request, because either reduction would be below the requested minimum. At this point, a prompt to re-specify LFAREA is issued. Note also that this example illustrates how the system prioritizes the 2 GB request over the 1 MB request, provided that the request can be satisfied at or above the requested minimum.

Table 9. LFAREA calculation example 5

LFAREA request with 64 GB of online real storage available at IPL	**Resulting number of 1 MB pages**	**Resulting number of 2 GB pages**
LFAREA=(1M=(40%,20%),2G=(11, 9)	24576 (40%)	11
LFAREA=(1M=(40%,20%),2G=(12, 10)	24576 (40%)	12

Table 9. LFAREA calculation example 5 (continued)		
LFAREA request with 64 GB of online real storage available at IPL	**Resulting number of 1 MB pages**	**Resulting number of 2 GB pages**
LFAREA=(1M=(40%,20%),2G=(13,11)	22118 (36%)	13
LFAREA=(1M=(40%,20%),2G=(14,12)	20275 (33%)	14
LFAREA=(1M=(40%,20%),2G=(15,13)	18432 (30%)	15
LFAREA=(1M=(40%,20%),2G=(16,14)	15974 (26%)	16
LFAREA=(1M=(40%,20%),2G=(17,15)	14131 (23%)	17
LFAREA=(1M=(40%,20%),2G=(18,16)	12288 (20%)	18
LFAREA=(1M=(40%,20%),2G=(19,17)	12288 (20%)	18
LFAREA=(1M=(40%,20%),2G=(20,18)	12288 (20%)	18
LFAREA=(1M=(40%,20%),2G=(21,19)	Unable to satisfy minimum	Unable to satisfy minimum

LFAREA calculation example 6

Table 10 on page 38 shows a series of LFAREA requests on a z/OS® system with 16 TB of online real storage at IPL. In every case, there are 6144 2G pages from above 4 TB regardless of the LFAREA specification. The LFAREA specification then controls how many large pages result below 4 TB.

Table 10. LFAREA calculation example 6		
LFAREA requests with 16 TB of online real storage are available at IPL.		
LFAREA specification	**Resulting number of 1M page frames**	**Resulting number of 2G page frames (including those beyond 4 TB)**

`LFAREA=0M`
`or LFAREA=0%`
`or LFAREA=(1M=0,2G=0)`
`or LFAREA=(1M=0%,2G=0%)`
`or unspecified`
	0	6144

LFAREA=(1M=1,2G=1)	1	6145
LFAREA=(1M=20%,2G=20%)	838041	6553
LFAREA=(1M=0,2G=80%)	0	7782

Scheduler work area (SWA/Extended SWA)

This area contains control blocks that exist from task initiation to task termination. It consists of subpools 236 and 237. It includes control blocks and tables created during JCL interpretation and used by the initiator during job step scheduling. Each initiator has its own SWA within the user's private area.

To enable recovery, the SWA can be recorded on direct access storage in the JOB JOURNAL. SWA is allocated at the top of each private area intermixed with LSQA and AUK (subpools 229, 230, and 249).

Authorized user key (AUK) area

The authorized user key (AUK) area allows local storage within a virtual address space to be obtained in the requestor's storage protect key. The area is used for control blocks that can be obtained only by authorized programs having appropriate storage protect keys. These control blocks were placed in storage by system components on behalf of the user.

The AUK subpools are intermixed with LSQA and SWA. Subpool 229 is fetch protected; subpools 230 and 249 are not. All three subpools are pageable. Subpools 229 and 230 are freed automatically at task termination; subpool 249 is freed automatically at job step task termination.

System region

This area is reserved for GETMAINs by all system functions (for example, ERPs) running under the region control tasks. It comprises 16K (except in the master scheduler address space, in which it has a 200K maximum) of each private area immediately above the PSA. V=V region space allocated to user jobs is allocated upwards from the top of this area. This area is pageable and exists for the life of each address space.

The private area user region/extended private area user region

The portion of the user's private area within each virtual address space that is available to the user's problem programs is called the `user region`.

Types of user regions

There are two types of user regions: virtual (or V=V) and real (or V=R). Virtual and real regions are mutually exclusive; private areas can be assigned to V=R or V=V, but not to both. The installation determines the region to which jobs are assigned. Usually, V=R should be assigned to regions containing jobs that cannot run in the V=V environment, or that are not readily adaptable to it. Programs that require a one-to-one mapping from virtual to central storage, such as program control interruption (PCI) driven channel programs, are candidates for real regions.

Two significant differences between virtual and real regions are:

- How they affect an installation's central storage requirements
- How their virtual storage addresses relate to their central storage addresses.

For virtual regions, which are pageable and swappable, the system allocates only as many central storage frames as are needed to store the paged-in portion of the job (plus its LSQA). The processor translates the virtual addresses of programs running in virtual regions to locate their central storage equivalent.

For real regions, which are nonpageable and nonswappable, the system allocates and fixes as many central storage frames as are needed to contain the entire user region. The virtual addresses for real regions map one-for-one with central storage addresses.

Virtual regions

Virtual regions begin at the top of the system region (see Figure 3 on page 16) and are allocated upward through the user region to the bottom of the area containing the LSQA, SWA, and the user key area (subpools 229, 230, and 249). Virtual regions are allocated above 16 megabytes also, beginning at the top of the extended CSA, and upward to the bottom of the extended LSQA, SWA, and the user key area (subpools 229, 230, and 249).

As a portion of any V=V job is paged in, 4K multiples (each 4K multiple being one page) of central storage are allocated from the available central storage. Central storage is dynamically assigned and freed on a demand basis as the job executes. V=V region requests that specify a specific region start address, are

supported only for restart requests, and must specify an explicit size through JCL (see "Specifying region size" on page 41).

As a special optimization for IBM z13® and compatible hardware, the system resource manager (SRM) may decide to maintain the backing real storage after a page of low private (or region) storage is storage released (FREEMAINed). Low private storage, like all 31-bit virtual storage, is categorized into subpools 0-127, 129-132, 240, 244, 250, 251 and 252, which are described in *z/OS MVS Diagnosis: Reference*. When a page in one such subpool is backed by such a frame, the backing frame is called a *freemained frame*. FREEMAINed frames are still considered to be owned by the address space, but can be easily reclaimed if the system runs low on storage. On z14 compatible hardware, this feature is extended to high private storage. Thus when any 31 bit private storage is released (FREEMAINed) it can remain backed by a FREEMAINed frame. The contents of such frames are cleared or dirtied at Storage Release (FREEMAIN) time when it is possible for an unauthorized unit of work to access the data.The RAX_FREEMAINEDFRAMES field in the RAX data area contains the number of FREEMAINed frames owned by a given address space.

FREEMAINed frames have consequences both for the system and applications using 31-bit storage. Address spaces may appear to own more real storage than they are actually using, and the system may appear to have less available storage. Applications that issue the TPROT instruction or reference storage that was FREEMAINed will not always get a condition code of 3 or an 0C4 system abend as they would when this feature is disabled. Applications can either change their behavior or request that this feature be disabled, either within a set of address spaces or system wide.

- The VSMLOC and VSMLIST services can be used as alternatives to TPROT to determine whether areas of virtual storage are GETMAIN assigned. These services are described in *z/OS MVS Programming: Authorized Assembler Services Reference SET-WTO*.
- The IARBRVER and IARBRVEA services are higher performance callable services that provide the same results as TPROT while taking FREEMAINed frames into account. These services are described in *z/OS MVS Programming: Authorized Assembler Services Reference EDT-IXG*.
- The FREEMAINEDFRAMES and FF31HIGH parameters in the DIAG*xx* parmlib member provides a way to deactivate these features either system wide or per address space. See *z/OS MVS Initialization and Tuning Reference* for more information.

When the RCEOA46291APPLIED bit is set (b'1') in the RCE data area, application programs can use the RAX_FREEMAINEDFRAMES value along with the RAX_PARMLIBSAYSKEEPFREEMAINEDFRAMES flag to determine whether the FREEMAINed frames feature is disabled for an address space. For instance, if RAX_FREEMAINEDFRAMES equals 0 and RAX_PARMLIBSAYSKEEPFREEMAINEDFRAMES equals b'0', the feature is disabled for that address space. Similarly when RCEOA51864 applied bit is set to b'1' in the RCE data area, application programs can use Rax64_FFHighFrames value along with the value of Rax_ParmlibSaysKeepHighFreemainedFrames flag to determine whether the high FREEMAINed frames feature is disabled for an address space.

Note: When Rax_ParmlibSaysKeepFreemainedFrames is b'0', Rax_ParmlibSaysKeepHighFreemainedFrames will also be b'0''. Rax_FreemainedFrames includes all FREEMAINed Frames that back low and high private.

Real regions

Real regions begin at the top of the system region (see Figure 3 on page 16) and are allocated upward to the bottom of the area containing LSQA, SWA, and the user key area (subpools 229, 230, and 249). Unlike virtual regions, real regions are only allocated below 16 megabytes.

The system assigns real regions to a virtual space within the private area that maps one-for-one with the real addresses in central storage below 16 megabytes. Central storage for the entire region is allocated and fixed when the region is first created.

Specifying region type

A user can assign jobs (or job steps) to virtual or real regions by coding a value of VIRT or REAL on the ADDRSPC parameter on the job's JOB or EXEC statement. For more information on coding the ADDRSPC parameter, see *z/OS MVS JCL Reference*.

The system uses ADDRSPC with the REGION parameter. The relationship between the ADDRSPC and REGION parameter is explained in *z/OS MVS JCL User's Guide*.

Region size and region limit

What is region size?

The region size is the amount of storage in the user region available to the job, started task, or TSO/E user. The system uses region size to determine the amount of storage that can be allocated to a job or step when a request is made using the STORAGE or GETMAIN macros and a variable length is requested. The region size minus the amount of storage currently allocated, determines the maximum amount of storage that can be allocated to a job by any single variable-length GETMAIN request.

What is region limit?

The region limit is the maximum total storage that can be allocated to a job by any combination of requests for additional storage using the GETMAIN or STORAGE macros. It is, in effect, a second limit on the size of the user's private area, imposed when the region size is exceeded.

Specifying region size

Users can specify a job's region size by coding the REGION parameter on the JOB or EXEC statement. The system rounds all region sizes to a 4K multiple.

The region size value should be less than the region limit value to protect against programs that issue variable length GETMAINs with very large maximums, and then do not immediately free part of that space or free such a small amount that a later GETMAIN (possibly issued by a system service) causes the job to fail.

For V=V jobs, the region size can be as large as the entire private area, minus the size of LSQA/SWA/user key area (subpools 229, 230, and 249) and the system region (see Figure 3 on page 16).

For V=R jobs, the REGION parameter value cannot be greater than the value of the REAL system parameter specified at IPL. If the user does not explicitly specify a V=R job's region size in the job's JCL, the system uses the VRREGN system parameter value in the IEASYS00 member of SYS1.PARMLIB.

For more information on coding the REGION parameter, see *z/OS MVS JCL Reference*.

Note: VRREGN should not be confused with the REAL system parameter. REAL specifies the total amount of central storage that is to be reserved for running all active V=R regions. VRREGN specifies the default subset of that total space that is required for an individual job that does not have a region size specified in its JCL.

An installation can override the VRREGN default value in IEASYS00 by:

- Using an alternate system parameter list (IEASYSxx) that contains the desired VRREGN parameter value.
- Specifying the desired VRREGN value at the operator's console during system initialization. This value overrides the value for VRREGN that was specified in IEASYS00 or IEASYSxx.

For V=R requests, if contiguous storage of at least the size of the REGION parameter or the system default is not available in virtual or central storage, a request for a V=R region is placed on a wait queue until space becomes available.

Note that V=R regions must be mapped one for one into central storage. Therefore, they do not have their entire virtual storage private area at their disposal; they can use only that portion of their private area having addresses that correspond to the contiguous central storage area assigned to their region, and to LSQA, SWA, and subpools 229, 230, and 249.

Limiting user region size

Why control region size or region limit?

Placing controls on a program's maximum region size or region limit is optional. The region size allowed to users can affect performance of the entire system. When altering the region size for an address space, you would also consider how it is related to any WLM resource group memory limit (memory pool) that the job might run under. Increasing the region size for instance, might require an increase in the memory pool limit to prevent unwanted paging. You might want to decrease the memory pool size for the same reasons you are decreasing the region size for address spaces classified to the resource group. For more information about memory pools and WLM resource groups see the "Processor storage overview" on page 6 section and the *z/OS MVS Planning: Workload Management*.

When there is no limit on region size and the system uses its default values, users might obtain so much space within a region (by repeated requests for small amounts of storage or a single request for a large amount) that no space would remain in the private area for the system to use. This situation is likely to occur when a program issues a request for storage and specifies a variable length with such a large maximum value that most or all of the space remaining in the private area is allocated to the request. If this program actively uses this large amount of space (to write tables, for example), it can affect central storage (also known as real storage) and thus impact performance.

To avoid an unexpected out-of-space condition, the installation should require the specification of some region size on the REGION parameter of JOB or EXEC JCL statements. Also, the installation can set system-wide defaults for region size through the job entry subsystem (JES) or with MVS installation exit routines. The installation can control user region limits for varying circumstances by selecting values and associating them with job classes or accounting information.

Using JES to limit region size

After interpreting the user-coded JCL, JES will pass to MVS either the value requested by the user on the REGION parameter, or the installation-defined JES defaults. JES determines the REGION value based on various factors, including the source of the job, or the class of the job, or both.

The limit for the user region size below 16 megabytes equals (1) the region size that is specified in the JCL, plus 64K, or (2) the JES default, plus 64K, if no region size is specified in the JCL. The IBM default limit for the user region size above 16 megabytes is 32 megabytes.

Note: If the region size specified in the JCL is zero, the region will be limited by the size of the private area.

For more information on JES defaults, see *z/OS JES2 Initialization and Tuning Reference*.

Using exit routines to limit region size

Two MVS installation exits are available to override the region values that JES passes to MVS. The exit routines are called IEFUSI and IEALIMIT and are described in detail in *z/OS MVS Installation Exits*.

The installation can use either IEFUSI or IEALIMIT to change the job's limit for the region size below 16 megabytes of virtual storage. However, to change the limit for the region size above 16 megabytes, the installation must code the IEFUSI installation exit.

If your installation does not supply an IEFUSI exit routine, the system uses the default values in the IBM-supplied IEALIMIT exit routine to determine region size. To determine region limit, the system adds 64K to the default region size.

Region and MEMLIMIT values and limits set by the IEFUSI exit will not be honored for programs with the NOHONORIEFUSIREGION program property table (PPT) attribute specified. The NOHONORIEFUSIREGION PPT attribute can be specified in the SCHEDxx member of SYS1.PARMLIB, or as an IBM supplied PPT default. This PPT attribute is used to bypass IEFUSI region controls for specific programs that require a larger than normal region in order to successfully execute.

Using SMFLIMxx to control the REGION and MEMLIMIT

In today's systems, above the bar storage amounts are vastly larger than the amount of private storage in the first 2 GB of an address space. This reduces the effectiveness of instituting controls over how the

private region storage is used. However, ensuring private storage availability for system-key functions remains a high priority for system programmers. Establishing a MEMLIMIT to limit the amount of 64-bit storage that a program can use is also a high priority. In addition, trying to maintain controls over these storage amounts with an assembler module (installation exit IEFUSI) causes intolerable overhead for many of today's clients.

The SMFLIMxx parmlib member addresses that problem by allowing the system programmer to set rules for storage usage in a member in parmlib. The SMFLIMxx parmlib member reduces or may even eliminate the need to update assembler code to establish or change limits on how much storage a user key job step program can use.

Note: This parmlib member is powerful and its effects can be far reaching. You must select values and filter keywords carefully and test them thoroughly before the member is put into production. The values in this parmlib member require as much thought, consideration, and testing as the CSA and SQA values in IEASYSxx.

Overview of the SMFLIM keywords

An SMFLIMxx parmlib member consists of an ordered set of rules, each starting with the word REGION to begin the rule. The keywords that follow the REGION statement are composed of filters and attributes.

Filter keywords are used by the system to match the jobstep being initiated to the rule. If any filter does not match, the attributes are not used and processing continues with the next rule. Filter keywords allow up to eight values to be specified for each, applied in an OR fashion. For example, a REGION statement with SUBSYS(JES*,STC) would match any jobstep that started as an STC or a JES-initiated job (provided all other filter keywords match as well). Each filter keyword that is used must have at least one matching string for the rule to match. In other words, the filter keywords are applied in an AND fashion. For example, a rule that consists of the following keywords:

```
REGION
JOBNAME(ABC) SUBSYS(STC,JES2)
MEMLIMIT(32T)
```

would match a jobname of ABC, started as either a started task or a job under JES2, but would not match a started task of XYZ.

The filter keywords allow for wildcard characters to help match rules generically. The * character is defined to match 0 or more characters, while the ? character matches exactly one character. The statements are ordered; subsequent statements with matching filters that appear later in the parmlib member or in a subsequent parmlib member override the values of statements that appeared earlier.

The following are the filter keywords:

JOBNAME
Matches a REGION statement to the jobname specified in the JCL.

JOBCLASS
Matches a REGION statement to the JES jobclass for the job.

JOBACCT
Matches the accounting information that is specified on the JCL JOB statement.

STEPNAME
Matches a REGION statement to the stepname specified in the JCL.

STEPACCT
Matches the accounting information that is specified on the JCL EXEC statement.

PGMNAME
Matches the jobstep program that is specified on the PGM= keyword of the EXEC.

USER
Matches the userid that is associated with the job, either specified on the USER keyword on the JOB statement, or the user that submitted the job. For more information about the USER keyword, see the *z/OS MVS JCL User's Guide*.

SUBSYS
Matches the subsystem that is associated with the job. This subsystem is often JES2 or JES3, but it might also be STC or some other subsystem name.

SYSNAME
Matches the name of the system. The SYSNAME keyword is handled differently than the other filter keywords, as detailed in the *z/OS MVS Initialization and Tuning Reference*.

See the *z/OS MVS Initialization and Tuning Reference* for the complete syntax and examples.

The other part of a REGION statement is the attributes to apply. These attributes consist of the following keywords:

SYSRESVABOVE
SYSRESVBELOW
Use these keywords to set aside part of the private area for system-key functions and services, which prevents the user-key jobstep program from obtaining all available private storage and causing system functions to fail.

REGIONABOVE
REGIONBELOW
Use these keywords to override the JCL REGION or REGIONX specification on a job to increase or decrease the job step's requested region size, up to the region limit set by the SYSRESVBELOW and SYSRESVABOVE keywords.

In today's systems with large amounts of real storage, it is often less important to enforce a restrictive REGION size on a jobstep. However, it might be useful to code REGIONABOVE(NOLIMIT) REGIONBELOW(NOLIMIT) to easily give the jobstep program access to the entire below-the-bar private area without altering the default for the jobclass or altering the REGION statements of many JCL jobs.

Another use might be to reduce the storage available to a job step. Consider this SMFLIMxx statement:

```
   REGION JOBNAME(*) SUBSYS(JES*,STC)
   REGIONABOVE(1G) REGIONBELOW(5M)
   SYSRESVABOVE(50M) SYSRESVBELOW(512K)
```

With SYSRESVABOVE(50M), the region limit is likely to be more than 1.5 GB, but the REGIONABOVE further reduces the region available to only 1 GB, perhaps to enforce some service level agreement or avoid some installation constraint.

Note: Coding REGIONABOVE(NOLIMIT) REGIONBELOW(NOLIMIT) (or using large specific values) is still subject to the SYSRESVABOVE and SYSRESVBELOW values that ensure that private storage is still available to system-key functions. For example, coding a rule such as:

```
REGION  JOBNAME(*) SUBSYS(JES*,STC)
   REGIONABOVE(NOLIMIT) REGIONBELOW(NOLIMIT)
   SYSRESVABOVE(50M)  SYSRESVBELOW(512K)
```

can meet the same goal as the coded IEFUSI exit used by your system for many years. In this example, the system will ensure that 50M of above-the-line storage and 512K of below the line storage is available to system-key functions, and the remaining above-the-line and below-the-line storage is available to the user program.

MEMLIMIT
Use the MEMLIMIT keyword to override the MEMLIMIT value that is provided as a default in SMFPRMxx or with the JCL MEMLIMIT= keyword. NOLIMIT is accepted for MEMLIMIT. Using NOLIMIT might leave your system exposed to storage shortages if the jobstep program attempts to obtain and use all of the above-the-bar virtual storage in its address space.

Establishing SMFLIM rules in SMFLIMxx

1. Determine the type of work to affect: all jobs, all started tasks, all job classes, jobs that are associated with one particular userid, and so on. Carefully plan what job values to override: execution status (EXECUTE keyword), private region storage (REGIONBELOW and REGIONABOVE keywords), system reserved storage (SYSRESVBELOW and SYSRESVABOVE keywords), or overall MEMLIMIT for the step (MEMLIMIT keyword). Code only the keywords for things you want to change about the unit of work. For example, by default, a given job step will be executed; that is why it was submitted. Therefore, you do not need to code EXECUTE(YES), unless some prior matching rule coded EXECUTE(CANCEL). See "SMFLIM examples" on page 46 for details on the interaction between rules.
2. Determine the attributes that you want to establish:
   a. If you want to restrict the user-key job step program from getting all available private storage, code the SYSRESVABOVE and SYSRESVBELOW keywords, with a reasonable amount of storage for your installation. A good value to choose might be the value that is formerly used in an IEFUSI exit. They are often coded such that the REGION LIMIT is reduced by some factor like 50M and 512K, respectively. If you don't have any values that are established in the IEFUSI exit, select some values that seem reasonable, such as SYSRESVABOVE(50M) and SYSRESVBELOW(512K). Then, test the values off-shift or on a test system that has a similar workload on it and similar storage amounts.

      Reminder: The more storage that is reserved with the SYSRESVxxxxx keywords, the less storage is available for user-key storage GETMAIN and STORAGE OBTAIN activity. This is a similar trade-off to selecting CSA and SQA storage values in IEASYSxx.

      If you code these keywords, IBM recommends that you reserve a minimum of 512K for SYSRESVBELOW and 50M for SYSRESVABOVE, ensuring you reserve some of your available extended private area for system use.
   b. If you want to override the REGION setting of the work unit, code the REGIONBELOW and REGIONABOVE keywords. If you are satisfied with the job using the REGION or REGIONX it coded, do not code the REGIONxxxxx keywords. Consider the following examples:
      - If you want to override the REGION setting and give a program access to all private storage, because you know that you established some reserved storage for system-key programs with the SYSRESVxxxxx keywords, you can code REGIONBELOW(NOLIMIT) and REGIONABOVE(NOLIMIT).
      - If you have a series of jobs that code REGION=200M but 200M is no longer sufficient, you can add a rule that overrides the 200M value to increase storage.
   c. If you want to override the MEMLIMIT value, code the MEMLIMIT keyword with the desired value.

      Reminder: Started tasks are often long running subsystems, such as LLA or DB2, providing services to other jobs. If you choose to set the MEMLIMIT with SMFLIMxx, you would likely use larger values for long running server address spaces than for batch jobs (where each step likely needs less storage than a server address space).
3. Examine the rules that are already created in the active SMFLIMxx parmlib members to see how these attributes are different.
   a. If you want to use the same attributes, you can update the filter keywords for that rule to include that additional type of work.
   b. If you want to use different attributes, select a set of filter keywords that the system can use to determine that the attributes should be applied:
      - The available filters are JOBNAME, STEPNAME, PGMNAME, JOBACCT, STEPACCT, USER, JOBCLASS, and SUBSYS. See the *z/OS MVS Initialization and Tuning Reference* for details on these keywords
      - The two accounting filters, JOBACCT and STEPACCT, are the most powerful because they let you code rules for job steps across all of the other keywords. For example, two jobs that have accounting data of "(D2405X,NY,POK)" could be treated the same, despite running in different job classes, with different job names and assigned user IDs, and so on. Given that

accounting information is only defined by the installation, these keywords provide the most end-user-oriented way of selecting job step attributes.

c. IBM recommends that you code separate rules to cover jobs and started tasks by using the SUBSYS keyword. In addition, by using the SUBSYS keyword, you can automatically exclude spawned z/OS UNIX work. For example, use SUBSYS(JES*,STC) for a rule that covers only JES initiated job steps and started tasks.

What happens when there are multiple rules, or if there is an SMFLIMxx active and an IEFUSI exit active

The system does the following in preparation for running the job-step program:

- Call the current IEFUSI exit and apply the region size and region limit set by the exit.
- Process the set of filters on the REGION statements in each active SMFLIMxx parmlib member.
  - If the filters on the REGION statement match the job step, the specified attributes are updated.
  - Processing continues with each subsequent REGION statement in each subsequent SMFLIMxx member, and the specified attributes are updated when the filters match the job step.
- Upon completion of IEFUSI and SMFLIMxx processing, the resulting attribute settings are a compendium of values set by the IEFUSI exit and the results of the matching REGION statements from the SMFLIMxx parmlib members.
- If both IEFUSI and SMFLIMxx REGION rules modify the same value or attribute, the REGION rules will take precedence over values set by IEFUSI. When multiple REGION rules modify the same value or attribute, REGION rules that are specified later in one or more SMFLIMxx parmlib members will take precedence over rules that are specified earlier in one or more parmlib members.

Note: If the IEFUSI exit sets bit 4 of word 5, subword 1 of its parameter list, the SMFLIMxx parmlib settings do not override the exit's settings. This applies to all SMFLIMxx settings, not just region size and region limit information.

SMFLIM examples

1. The following example sets a limit on the amount of storage that is used by all JES-initiated programs, but it does not change how the program obtains its private storage:

```
REGION JOBNAME(*) SUBSYS(JES*)
SYSRESVABOVE(50M) SYSRESVBELOW(512K)
MEMLIMIT(10T)
```

2. The following example sets a larger limit on the amount of storage that is used by started tasks (including those tasks that are started with SUB=MSTR). This example also overrides the REGION (or REGIONX) specification to allow access to all storage that is not otherwise reserved for the system:

```
REGION JOBNAME(*) SUBSYS(STC)
REGIONABOVE(NOLIMIT) REGIONBELOW(NOLIMIT)
SYSRESVABOVE(50M) SYSRESVBELOW(512K)
MEMLIMIT(NOLIMIT)
```

3. The following example allows user SBJ access to all available 64-bit storage when a large DFSORT job named SBJSORT is running. This example also cancels any other user that attempts to run DFSORT (this example requires two REGION statements):

```
REGION  JOBNAME(*) USER(*) PGMNAME(ICEMAN)
EXECUTE(CANCEL)
REGION JOBNAME(SBJSORT) USER(SBJ) PGMNAME(ICEMAN) SUBSYS(JES*)
/* do not change SYSRESV values, prior rule establishes the correct amounts */
MEMLIMIT(NOLIMIT) EXECUTE(YES)
```

4. The following example cancels a job that did not code any accounting data, or coded null accounting data:

```
REGION SUBSYS(JES*)
JOBACCT(,()) EXECUTE(CANCEL)
```

Because these examples really affect different work, they would be coded in the same parmlib member, as follows:

```
REGION JOBNAME(*) SUBSYS(JES*)
   SYSRESVABOVE(50M) SYSRESVBELOW(512K)
   MEMLIMIT(10T)
REGION JOBNAME(*) SUBSYS(STC)
   REGIONABOVE(NOLIMIT) REGIONBELOW(NOLIMIT)
   SYSRESVABOVE(50M) SYSRESVBELOW(512K)
   MEMLIMIT(NOLIMIT)
REGION  JOBNAME(*) USER(*) PGMNAME(ICEMAN)
   EXECUTE(CANCEL)
REGION JOBNAME(SBJSORT) USER(SBJ) PGMNAME(ICEMAN) SUBSYS(JES*)
REGION SUBSYS(JES*)
   JOBACCT(,())
```

Notes:

- A job that is running under JES2 with jobname BDOALLOC would have 50 MB and 512 KB reserved for system-key storage, but it would run with whatever REGION was specified in its jobclass or on its JCL.
- The same job that is started as a started task would have unlimited extended and non-extended private region, regardless of what it specified in its JCL.
- Any job with program name ICEMAN would be cancelled, unless it had JOBNAME SBJSORT, and was running under JES2 or JES3, with user SBJ assigned.
- Any job without JOB accounting information would be cancelled.

Identifying problems in virtual storage (DIAGxx parmlib member)

This section describes functions you can use to identify problems with virtual storage requests (such as excessive use of common storage, or storage that is not freed at the end of a job or address space). The functions are as follows; you can control their status in the DIAGxx parmlib member.:

- Common storage tracking
- GETMAIN/FREEMAIN/STORAGE (GFS) trace.

Using the common storage tracking function

Common storage tracking collects data about requests to obtain or free storage in CSA, ECSA, SQA, and ESQA. You can use this data to identify jobs or address spaces that use up an excessive amount of common storage or have ended without freeing common storage. If those jobs or address spaces have code to free that storage when they are canceled, you might relieve the shortage and avoid an IPL if you cancel those jobs or address spaces using an operator command.

You can use Resource Measurement Facility (RMF) or a compatible monitor program to display the data that the storage tracking function collects. You can also format storage tracking data in a dump using interactive problem control system (IPCS). For information on how to use IPCS to format common storage tracking data, see the description of the VERBEXIT VSMDATA subcommand in *z/OS MVS IPCS Commands*.

The OWNER parameter on the CPOOL BUILD, GETMAIN, and STORAGE OBTAIN macros assigns ownership of the obtained CSA, ECSA, SQA, or ESQA storage to a particular address space or the system. To get the most value from the common storage tracking function, ensure that authorized programs specify the OWNER parameter on all CPOOL BUILD, GETMAIN, and STORAGE OBTAIN macros that:

- Request storage in CSA, ECSA, SQA, or ESQA, *and*
- Have an owning address space that is *not* the home address space.

IBM recommends that common storage tracking always be activated.

You can turn storage tracking on by activating a DIAGxx parmlib member, either at IPL (specify DIAG=xx as a system parameter) or during normal processing (enter a SET DIAG=xx command). For more information about using SET DIAG=xx, see *z/OS MVS System Commands*.

If you do not specify a DIAGxx parmlib member at IPL, the system processes the default member DIAG00. If DIAG00 does not exist, common storage tracking will not be turned on. Common storage

remains active until turned off through a SET DIAG=*xx* command. DIAG00 also turns off the GFS trace function, which is described in "Using GETMAIN/FREEMAIN/STORAGE (GFS) trace" on page 48.

IBM also provides the DIAG01 parmlib member, which turns the common storage tracking function on, and DIAG02, which turns the common storage tracking function off. Your installation must create any additional DIAGxx parmlib members.

Using GETMAIN/FREEMAIN/STORAGE (GFS) trace

GFS trace helps you identify problems related to requests to obtain or free storage. GFS trace uses the generalized trace facility (GTF) to collect data about storage requests, and places this data in USRF65 user trace records in the GTF trace. You can use IPCS to format the GTF trace records.

To turn GFS trace on or off, activate a DIAGxx parmlib member, either at IPL (specify DIAG=xx as a system parameter) or during normal processing (enter a SET DIAG=xx operator command). If you do not specify a DIAGxx parmlib member at IPL, the system processes the default member DIAG00, which turns GFS trace off. See *z/OS MVS System Commands* for more information about using SET DIAG=xx.

The DIAGxx parmlib member also allows you to specify:

- GFS trace filtering data, which limits GFS trace output according to:
  - Address space identifier (ASID)
  - Storage subpool
  - The length of the storage specified on a request
  - Storage key
- GFS trace record data, which determines the data to be placed in each record, according to one or more data items specified on the DATA keyword.

GFS trace degrades performance to a degree that depends on the current workload in the system. Therefore, it is a good idea to activate GFS trace only when you know there is a problem. For information about a GFS trace, see *z/OS MVS Diagnosis: Tools and Service Aids*.

Auxiliary storage overview

Auxiliary storage DASD hard disk drives are required on z/OS systems for storing all system data sets. For additional paging flexibility and efficiency, you can add optional storage-class memory (SCM) on Flash Express solid-state drives or the Virtual Flash Memory (VFM) as a second type of auxiliary storage.

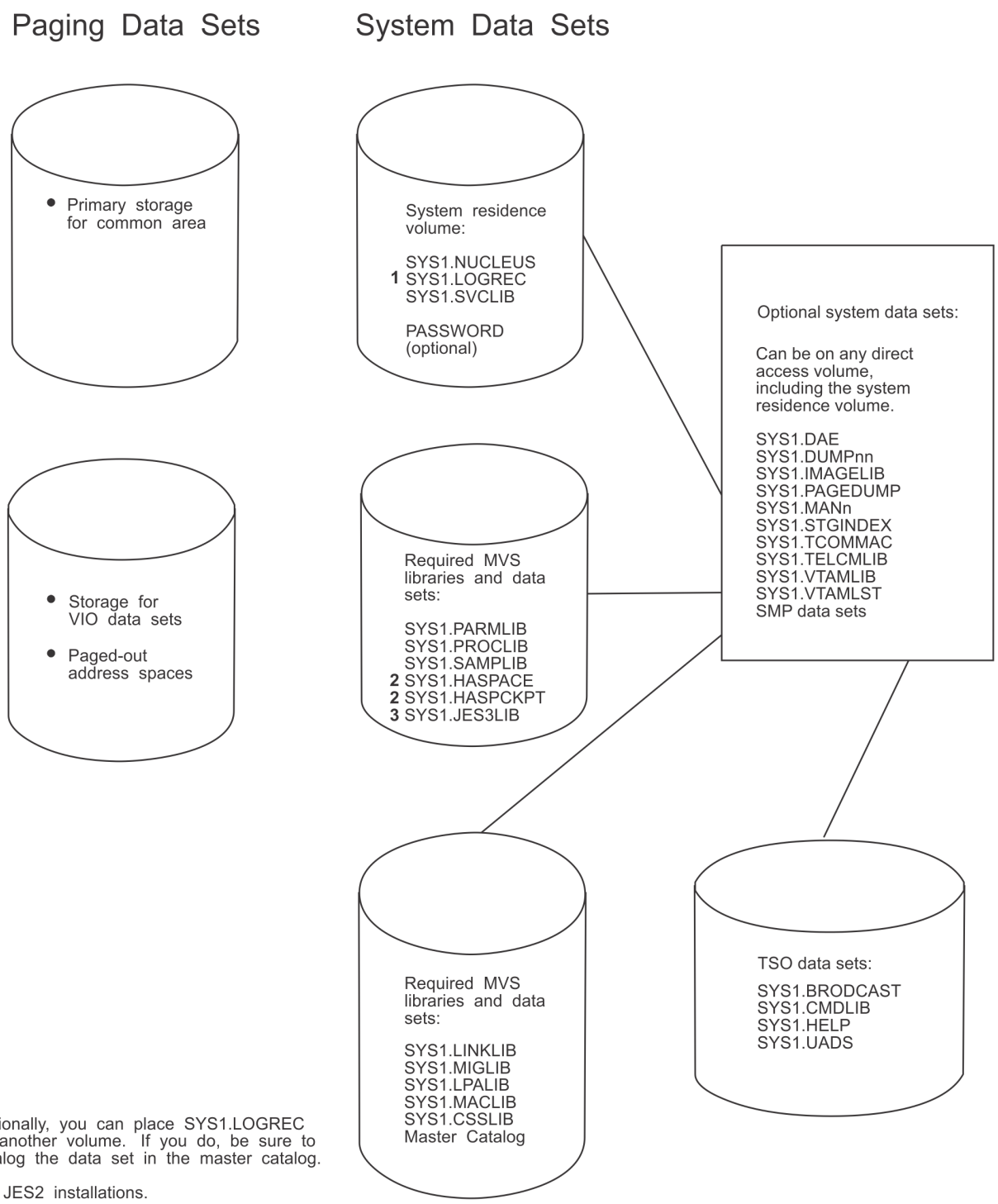
You must have enough auxiliary storage available to store the programs and data that comprise your z/OS system. This auxiliary storage that supports basic system requirements has three logical areas (see Figure 4 on page 49):

- System data set storage area.
- Paging data sets for backup of all pageable address spaces.
- TSO data sets.

If your auxiliary storage includes SCM on Flash drives, you must be aware of the data storage requirements and limitations for each storage medium, which are described in the following sections. Refer to Chapter 2, "Auxiliary storage management initialization," on page 67, and to the PAGE, PAGESCM, NONVIO and PAGTOTL parameters (IEASYSxx PARMLIB member) in *z/OS MVS Initialization and Tuning Reference*.

System data sets

During z/OS system installation, you must allocate space for the required system data sets on appropriate direct access devices. The DEFINE function of access method services defines both the storage requirements and the volume for each system data set. Some data sets must be allocated on the system residence volume, while others can be placed on other direct access volumes.



1 Optionally, you can place SYS1.LOGREC on another volume. If you do, be sure to catalog the data set in the master catalog.

2 For JES2 installations.

3 For JES3 installations.

Note that auxiliary storage need not be organized as shown above. This figure summarizes the makeup of auxiliary storage.

Figure 4. Auxiliary storage requirement overview

Paging data sets

When you are creating WLM resource groups with a memory limit, you need to consider the impact that potential paging of the memory pool might have on system paging resources. For more information see the section on “Memory pools” on page 10.

Paging data sets and optional storage-class memory (SCM), contain the paged-out portions of all virtual storage address spaces. In addition, output to virtual I/O devices might be stored in the paging data sets. VIO data cannot be stored on SCM because SCM does not currently support persistence across IPLs. Before the first IPL, you must allocate sufficient space to back up the following virtual storage areas:

- Primary storage for the pageable portions of the common area
- Secondary storage for duplicate copies of the pageable common area
- The paged-out portions of all swapped-in address spaces - both system and installation
- Space for all address spaces that are, or were, swapped out
- VIO data sets that are backed by auxiliary storage.

Note: VIO data must remain on page data sets even when SCM is installed. All other data types can be paged to page data sets or to SCM.

Paging data sets are specified in IEASYSxx members of SYS1.PARMLIB. When this is done, any PAGE parameter in an IEASYSxx specified during IPL overrides any PAGE parameter in IEASYS00. For paging to SCM, use the PAGESCM IEASYSxx member.

To add paging space to the system after system installation, the installation must use the access method services to define and format the new paging data sets. To add the new data sets to the existing paging space, either use the PAGEADD operator command or re-IPL the system, issuing the PAGE parameter at the operator's console. To delete paging space, use the PAGEDEL operator command. To bring additional SCM online after an IPL, use the `CONFIG ONLINE` command.

During initialization, paging spaces are set up by merging the selected IEASYSxx parmlib member list of data set names with any names that are provided by the PAGE parameter (issued at the operator console) and any names from the previous IPL.

Directed VIO

When backed by auxiliary storage, VIO data set pages can be directed to a subset of the local paging data sets through *directed VIO*, which allows the installation to direct VIO activity away from selected local paging data sets and use these data sets only for non-VIO paging. With directed VIO, faster paging devices can be reserved for paging where good response time is important. The NONVIO system parameter, with the PAGE parameter in the IEASYSxx parmlib member, allows the installation to define those local paging data sets that are not to be used for VIO, leaving the rest available for VIO activity. However, if space is depleted on the paging data sets that were made available for VIO paging, the non-VIO paging data sets will be used for VIO paging.

The installation uses the DVIO keyword in the IEAOPTxx parmlib member to either activate or deactivate directed VIO. To activate directed VIO, the operator issues a SET OPT=xx command where the xx specifies the IEAOPTxx parmlib member that contains the DVIO=YES keyword; to deactivate directed VIO, xx specifies an IEAOPTxx parmlib member that contains the DVIO=NO keyword. The NONVIO parameter of IEASYSxx and the DVIO keyword of IEAOPTxx is explained more fully in *z/OS MVS Initialization and Tuning Reference*.

Primary and secondary PLPA

During initialization, both primary and secondary PLPAs are established. The PLPA is established initially from the LPALST concatenation.

On the PLPA page dataset, ASM maintains records that have pointers to the PLPA and extended PLPA pages. During quick start and warm start IPLs, the system uses the pointers to locate the PLPA and extended PLPA pages. The system then rebuilds the PLPA and extended PLPA page and segment tables, and uses them for the current IPL.

If CLPA is specified at the operator's console, indicating a cold start is to be performed, the PLPA storage is deleted and made available for system paging use. A new PLPA and extended PLPA is then loaded, and pointers to the PLPA and extended PLPA pages are recorded on the PLPA page dataset. CLPA also implies CVIO.

If storage-class memory (SCM) is installed, ASM pages PLPA to the PLPA data set and also to SCM. ASM then uses the PLPA data set for warm starts, and the PLPA on SCM for resolving page faults.

Virtual I/O storage space

Virtual I/O operations may send VIO dataset pages to the local paging dataset space. During a quick start, the installation uses the CVIO parameter to purge VIO dataset pages. During a warm start, the system can recover the VIO dataset pages from the paging space. If an installation wants to delete VIO page space reserved during the warm start, it issues the CVIO system parameter at the operator's console. CVIO applies only to the VIO dataset pages that are associated with journaled job classes. (The VIO journaling dataset contains entries for the VIO datasets associated with journaled job classes.) If there are no journaled job classes or no VIO journaling dataset, there is no recovery of VIO dataset pages. Instead, all VIO dataset pages are purged and the warm start is effectively a quick start.

If the SPACE parameter for a DD statement having a UNIT parameter, associated with a UNITNAME that was defined with having VIO=YES, is not specified, the default size is 10 primary and 50 secondary blocks with an average block length of 1000 bytes.

The cumulative number of page datasets must not exceed 256. This maximum number of 256 page data sets should follow these guidelines:

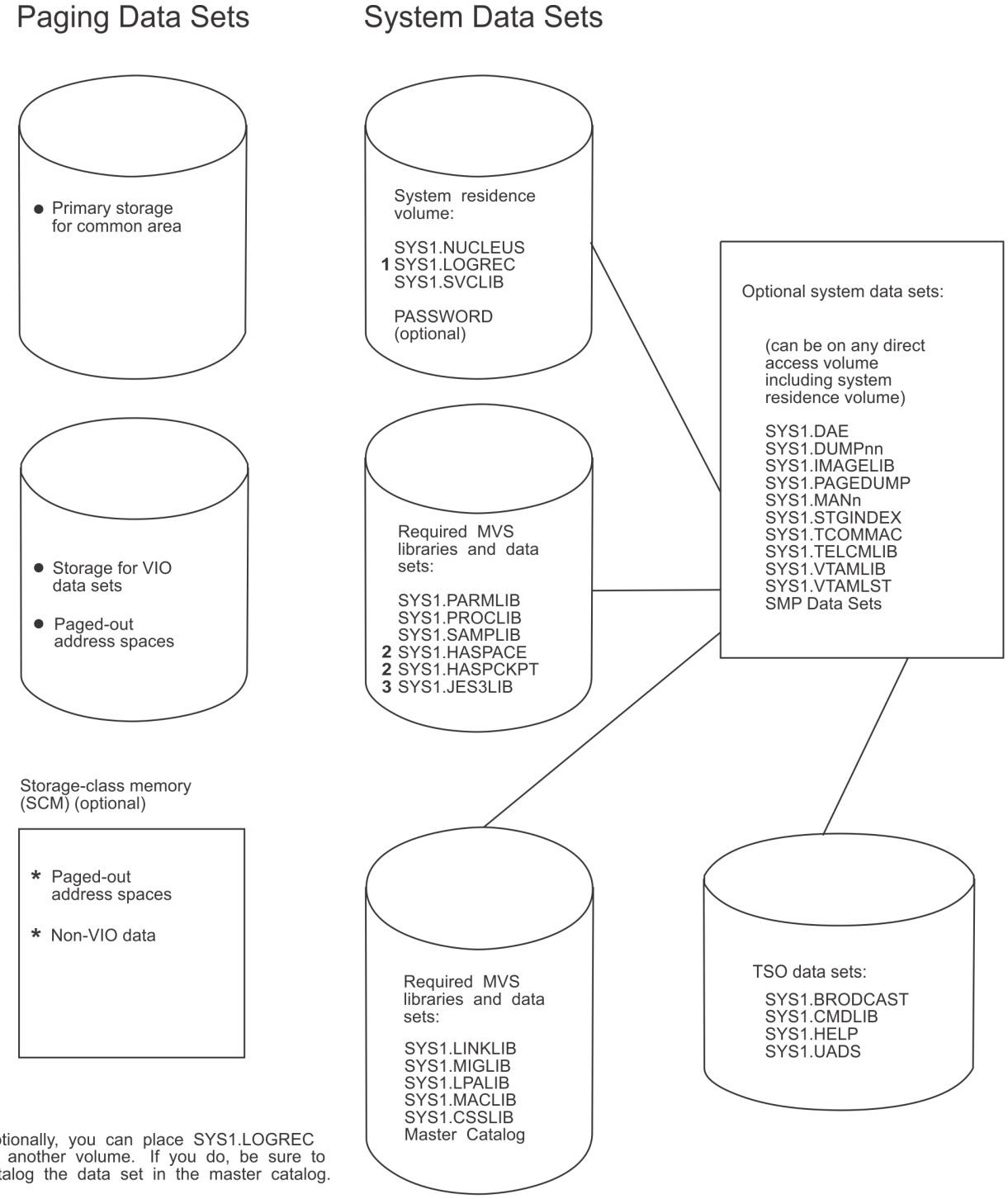
- There must either be one PLPA page data set or *NONE* must be specified to indicate that SCM is to be substituted. In either case, the specification counts toward the 256 maximum.
- There must either be one common page data set or *NONE* must be specified to indicate that SCM is to be substituted. In either case, the specification counts toward the 256 maximum.
- There must be at least one *local* page data set, but no more than 253.

The actual number of pages required in paging data sets depends on the system load, including the size of the VIO data sets being created and the rates at which they are created and deleted.

Using storage-class memory (SCM)

Adding optional storage-class memory (SCM) on Flash Express cards, including Virtual Flash Memory (VFM), to your auxiliary storage can increase paging performance and flexibility. Even with SCM usage, page data sets on DASD are required for auxiliary storage. With the exception of VIO data, which must remain on page data sets, all other data types can be paged out to SCM. ASM pages out from main memory to either storage medium based on the response times and on data set characteristics, critical paging requirements, disk availability (during a HyperSwap® failover, for example) and available storage space. For additional information, refer to Chapter 2, “Auxiliary storage management initialization,” on page 67.

Figure 5 on page 52 depicts the required auxiliary storage management page data sets with the addition of optional SCM. For additional information on using SCM, refer to Chapter 2, “Auxiliary storage management initialization,” on page 67 and the PAGE, PAGESCM, NONVIO and PAGTOTL parameters of parmlib member IEASYSxx in *z/OS MVS Initialization and Tuning Reference*.



1 Optionally, you can place SYS1.LOGREC on another volume. If you do, be sure to catalog the data set in the master catalog.

2 For JES2 installations.

3 For JES3 installations

Note that auxiliary storage need not be organized as shown. This figure summarizes the makeup of auxiliary storage.

Figure 5. Auxiliary storage diagram with SCM

Improving module fetch performance with LLA

You can improve the performance of module fetching on your system by allowing library lookaside (LLA) to manage your production load libraries. LLA reduces the amount of I/O needed to locate and fetch modules from DASD storage.

Specifically, LLA improves module fetch performance in the following ways:

- By maintaining (in the LLA address space) copies of the library directories the system uses to locate load modules. The system can quickly search the LLA copy of a directory in virtual storage instead of using costly I/O to search the directories on DASD.
- By placing (or staging) copies of selected modules in a virtual lookaside facility (VLF) data space (when you define the LLA class to VLF, and start VLF). The system can quickly fetch modules from virtual storage, rather than using slower I/O to fetch the modules from DASD.

  LLA determines which modules, if staged, would provide the most benefit to module fetch performance. LLA evaluates modules as candidates for staging based on statistics it collects about the members of the libraries it manages (such as module size, frequency of fetches per module (fetch count), and the time required to fetch a particular module).

  If necessary, you can directly influence LLA staging decisions through installation exit routines (CSVLLIX1 and CSVLLIX2). For information about coding these exit routines, see *z/OS MVS Installation Exits*.

LLA and module search order

When a program requests a module, the system searches for the requested module in various system areas and libraries, in the following order:

1. Modules that were loaded under the current task (LLEs)
2. The job pack area (JPA)
3. Tasklib, steplib, joblib, or any libraries that were indicated by a DCB specified as an input parameter to the macro used to request the module (LINK, LINKX, LOAD, ATTACH, ATTACHX, XCTL or XCTLX).
4. Active link pack area (LPA), which contains the FLPA and MLPA
5. Pageable link pack area (PLPA)
6. SYS1.LINKLIB and libraries concatenated to it through the LNKLSTxx member of parmlib. (“Placing modules in the system search order for programs” on page 19 explains the performance improvements that can be achieved by moving modules from the LNKLST concatenation to LPA.)

When searching TASKLIBs, STEPLIBs, JOBLIBs, a specified DCB, or the LNKLST concatenation for a module, the system searches each data set directory for the first directory entry that matches the name of the module. The directory is located on DASD with the data set, and is updated whenever the module is changed. The directory entry contains information about the module and where it is located in storage. (For more information, see the "Program Management" topic in the *z/OS MVS Programming: Assembler Services Guide*.)

As mentioned previously, LLA caches, in its address space, a copy of the directory entries for the libraries it manages. For modules that reside in LLA-managed libraries, the system can quickly search the directories in virtual storage instead of using I/O to search the directories on DASD.

Planning to use LLA

Using LLA will involve the following activities:

1. Determining which libraries LLA is to manage
2. Coding members of parmlib; see “Coding the required members of parmlib” on page 54

3. Controlling LLA through operator commands; see "Controlling LLA and VLF through operator commands" on page 56.

When determining which data sets LLA is to manage, try to limit these choices to production load libraries that are rarely changed. Because LLA manages LNKLST libraries by default, you need only determine which non-LNKLST libraries LLA is to manage. If, for some reason, you do not want LLA to manage particular LNKLST libraries, you must explicitly remove the libraries from LLA management, as described in "Removing libraries from LLA management" on page 58.

Securing LLA

For information about protecting the use of LLA, see the following topics:

- Protecting LLA-managed data sets in *z/OS Security Server RACF Security Administrator's Guide*
- LLA parmlib data sets and LLA-managed data sets in *z/OS Planning for Multilevel Security and the Common Criteria*.

Using VLF with LLA

Because you obtain the most benefit from LLA when you have both LLA and VLF functioning, you should plan to use both. When used with VLF, LLA reduces the I/O required to fetch modules from DASD by causing selected modules to be staged in VLF data spaces. LLA does not use VLF to manage library directories. When used without VLF, LLA eliminates only the I/O the system would use in searching library directories on DASD.

LLA notes

1. All LLA-managed libraries must be cataloged. This includes LNKLST libraries. A library must remain cataloged for the entire time it is managed by LLA. Please see "Recataloging LLA-managed data sets while LLA is active" on page 60 for additional information about recataloging LLA-managed libraries.
2. The benefits of LLA load module caching applies only to modules that are retrieved through the following macros: LINK, LINKX, LOAD, ATTACH, ATTACHX, XCTL and XCTLX.
3. LLA does not stage load modules in overlay format. LLA manages the directory entries of overlay format modules, but the modules themselves are provided through program fetch. If you want to make overlay format modules eligible for staging, you must re-linkedit the modules as non-overlay format. These reformatted modules might occupy more storage when they execute and, if LLA does not stage them, might take longer to be fetched.
4. LLA is set to service class SYSTEM, independent of any user-defined classification rules.

Coding the required members of parmlib

LLA and VLF are associated with parmlib members, as follows:

- Use the CSVLLAxx member to identify the libraries that LLA is to manage.
- Use the COFVLFxx member to extend VLF services to LLA.

This information provides guidance information for coding the keywords of CSVLLAxx. For information about the required syntax and rules for coding CSVLLAxx and COFVLFxx, see *z/OS MVS Initialization and Tuning Reference*.

Coding CSVLLAxx

Use CSVLLAxx to specify which libraries LLA is to manage and how it is to manage them.

IBM does not provide a default CSVLLAxx member (such as CSVLLA00). If you do not supply a CSVLLAxx member by the LLA=xx parameter of the LLA procedure on the first starting of LLA for this IPL, or if you specify a parameter of LLA=NONE, LLA by default manages only the libraries that are accessed through

the LNKLST concatenation. If you have started LLA successfully with a CSVLLAxx member and then stop LLA, a subsequent start of LLA uses that CSVLLAxx member unless you supply another CSVLLAxx member. If you want to return to the default settings, specify LLA=NONE LLA=00.

Using multiple CSVLLAxx members

To use more than one CSVLLAxx member at a time, specify the additional members to be used on the PARMLIB and SUFFIX keywords in the original CSVLLAxx member. The CSVLLAxx members pointed to by the PARMLIB and SUFFIX keywords must not point back to the original member, nor to each other.

You can use the PARMLIB and SUFFIX keywords to specify CSVLLAxx members that reside in data sets other than PARMLIB. You can then control LLA's specifications without having update access to PARMLIB.

You can also use the PARMSUFFIX parameter of the CSVLLAxx parmlib member to specify additional CSVLLAxx members. PARMSUFFIX differs from the PARMLIB(dsn) SUFFIX(xx) statement in that no data set name is specified. PARMSUFFIX searches the logical parmlib for the CSVLLAxx member.

Example of multiple CSVLLAxx members

Let's say you want two parmlibs (IMS.PARMLLA and AMVS.LLAPARMS) so that a TSO REXX exec can automatically activate a new module in LNKLST when it has copied the new module into a LNKLST library.

Do the following; START LLA,LLA=IM,SUB=MSTR with CSVLLAIM as shown:

```
FREEZE(-LNKLST-)
PARMLIB(IMS.PARMLLA)
SUFFIX(IA)
PARMLIB(AMVS.LLAPARMS)
SUFFIX(RX)
```

where AMVS.LLAPARMS (CSVLLARX) would contain the latest update requested by the REXX exec, such as:

```
REMOVE(...)/LIBRARIES(...)MEMBERS...
                or
PARMSUFFIX(...)
                or
PARMLIB(...) SUFFIX(...)
```

The REXX exec can either use multiple members and use the PARMSUFFIX to identify them, or move the old CSVLLARX to CSVLLARn before building the new one.

Storing program objects in PDSEs

With SMS active, you can produce a program object, and executable program unit that can be stored in a partitioned data set extended (PDSE) program library. Program objects resemble load modules in function, but have fewer restrictions and are stored in PDSE libraries instead of PDS libraries. PDSE libraries that are to be managed by LLA must contain program objects. LLA manages both load and program libraries.

Coding COFVLFxx

To have VLF stage load modules from LLA-managed libraries, you can use the default COFVLFxx member that is shipped in parmlib (COFVLF00), or, optionally, an installation-supplied COFVLFxx member. The installation-supplied member must contain a CLASS statement for LLA (see COFVLF00 for an example).

If you modify the COFVLFxx parmlib member, you must stop and restart VLF to make the changes effective.

If you want to use an installation-supplied COFVLFxx member instead of COFVLF00, do the following:

- Specify a CLASS statement for LLA in the alternative COFVLFxx member
- Specify the suffix of the alternative COFVLFxx member on the START VLF command. Otherwise, the system uses COFVLF00 by default.

For information about the required syntax and rules for coding the COFVLFxx member, see *z/OS MVS Initialization and Tuning Reference*.

Controlling LLA and VLF through operator commands

Use the following commands to control LLA:

- START LLA and START VLF
- STOP LLA and STOP VLF
- MODIFY LLA.

This information explains how to use commands to control LLA. For information about the required syntax and rules for entering commands, see *z/OS MVS System Commands*.

Starting LLA

The START LLA command is included in the IBM-supplied IEACMD00 member of parmlib. Therefore, the system automatically starts LLA when it reads the IEACMD00 member during system initialization.

Although the system automatically starts LLA, it does not, by default, activate a CSVLLAxx member. To activate a CSVLLAxx member at system initialization, specify the suffix (xx) of the CSVLLAxx member in either of the following places:

- In the LLA procedure in SYS1.PROCLIB, as shown, where 'xx' matches the suffix on the CSVLLAxx member to be used:

```
//LLA PROC LLA=xx
//LLA EXEC PGM=CSVLLCRE,PARM='LLA=&LLA'
```

- On the START LLA command in the IEACMD00 member, as shown, where 'xx' matches the suffix on the CSVLLAxx member to be used:

```
COM='START LLA,SUB=MSTR,LLA=xx'
```

You should not set a region limit for LLA, either by setting a region size or by an IEFUSI exit. This will avoid storage exhaustion abends in the LLA address space.

If you do not supply a CSVLLAxx member by the LLA=xx parameter of the LLA procedure on the first starting of LLA for this IPL, or if you specify a parameter of LLA=NONE, LLA will by default manage only the libraries that are accessed through the LNKLST concatenation. If you have started LLA successfully with a CSVLLAxx member and then stop LLA, a subsequent start of LLA will use that CSVLLAxx member unless you supply another CSVLLAxx member. If you want to get back to the default settings, specify LLA=NONE.

When started, LLA manages both the libraries specified in the CSVLLAxx member as well as the libraries in the LNKLST concatenation.

If you wish to use a data set other than the parmlib concatenation for LLA, specify the data set that LLA is to use on an IEFPARM DD statement in the LLA procedure in PROCLIB. Note that specifying a data set on the IEFPARM DD statement will result in an ENQ on that data set for the duration of the LLA started task. The ENQ can only be removed by stopping LLA.

Starting LLA after system initialization

IBM recommends that you specify SUB=MSTR on the START LLA command to prevent LLA from failing if JES fails. For example, in the following command, 'xx' matches the suffix on the CSVLLAxx member to be used, if any:

```
START LLA,SUB=MSTR,LLA=xx
```

Starting VLF

LLA provides better performance when VLF services are available, so it is better (although not necessary) to start VLF before LLA. However, the operation of LLA does not depend on VLF.

To allow VLF to be started through the START command, create a VLF procedure, or use the following procedure, which resides in SYS1.PROCLIB:

```
//VLF     PROC  NN=00
//VLF     EXEC  PGM=COFMINIT,PARM='NN=&NN'
```

When you issue the START VLF command, the VLF procedure activates the IBM-supplied COFVLF00 member, which contains a CLASS statement for LLA.

Stopping LLA and VLF

You can stop LLA and VLF through STOP commands. Note that stopping VLF purges all the data in VLF data spaces for all classes defined to VLF (including LLA), and will slow performance.

If LLA or VLF is stopped (either by a STOP command or because of a system failure), you can use a START command to reactivate the function.

Modifying LLA

You can use the MODIFY LLA command to change LLA dynamically, in either of the following ways:

- MODIFY LLA,REFRESH. Rebuilds LLA's directory for the entire set of libraries managed by LLA. This action is often called a complete refresh of LLA.
- MODIFY LLA,UPDATE=xx. Rebuilds LLA's directory only for specified libraries or modules. *xx* identifies the CSVLLAxx member that contains the names of the libraries for which directory information is to be refreshed. This action is often called a *selective refresh* of LLA. (For details, see “Identifying members for selective refreshes” on page 57.)

When an LLA-managed library is updated, the version of a module that is located by a directory entry saved in LLA will differ from the version located by the current directory entry on DASD for that module. If you update a load module in a library that LLA manages, it is a good idea to follow the update by issuing the appropriate form of the MODIFY LLA command to refresh LLA's cache with the latest version of the directory information from DASD. Otherwise, the system will continue to use an older version of a load module.

Notes:

1. Applications can use the LLACOPY macro to refresh LLA's directory information. For information about the LLACOPY macro, see *z/OS MVS Programming: Authorized Assembler Services Guide*.
2. You can specify up to 255 concurrent LLA modify commands before the system reports that LLA is busy.

Identifying members for selective refreshes

In CSVLLAxx, specify the MEMBERS keyword to identify members of LLA-managed libraries for which LLA-cached directory entries are to be refreshed during selective refreshes of LLA. If you issue the MODIFY LLA,UPDATE=xx command to select a CSVLLAxx member that has libraries specified on the MEMBERS keyword, LLA will update its directory for each of the members listed on the MEMBERS keyword.

Selectively refreshing LLA directory allows updated LLA-managed modules to be activated without activating other changed LLA-managed modules. Selective LLA refresh also avoids the purging and restaging of modules that have not changed. When a staged module is refreshed in the LLA directory, LLA purges the copy of the module in the virtual lookaside facility (VLF) data space and may then restage the module into VLF.

For more information about specifying the MEMBERS keyword, see the description of the CSVLLAxx member in *z/OS MVS Initialization and Tuning Reference*.

Removing libraries from LLA management

In CSVLLAxx, specify the REMOVE keyword for libraries that are to be removed dynamically from LLA management. If you issue the MODIFY LLA,UPDATE=xx command (selective refresh) to select a CSVLLAxx member that lists libraries on the REMOVE keyword, LLA no longer provides directory entries or staged modules for these libraries, regardless of whether the libraries are included in the LNKLST concatenation. (Note that you cannot use REMOVE to change the order or contents of the LNKLST concatenation itself.)

You can also use the MODIFY LLA,REFRESH command (complete refresh) to remove libraries from LLA management. This command rebuilds the entire LLA directory, rather than specified entries in LLA's directory. To limit the adverse effects on performance caused by an LLA refresh, whenever possible, use a selective refresh of LLA instead of a complete refresh, or stopping and restarting LLA.

In any case, when LLA directory entries are refreshed, LLA discards directory information of the associated module and causes VLF (if active) to remove the module from the VLF cache. This reduces LLA's performance benefit until LLA stages them again. Because LLA stages modules using the cached directory entries, you should refresh LLA whenever a change is made to an LLA-managed data set.

The MODIFY LLA command does not reload (or refresh) modules that are already loaded into virtual storage, such as modules in long-running or never-ending tasks.

For more information about specifying the REMOVE keyword, see the description of the CSVLLAxx member in *z/OS MVS Initialization and Tuning Reference*.

Modifying shared data sets

You can allow two or more systems to share access to the same library directory. When modifying or stopping LLA in this case, your changes must take effect simultaneously on all systems that share access to the directory.

In cases where you simply want to update an LLA-managed data set, it is easier to remove the data set from LLA management and update it, rather than stopping LLA on all systems. To do so, enter a MODIFY LLA,UPDATE=xx command on each system that shares access to the data set, where 'xx' identifies a CSVLLAxx member that specifies, on the REMOVE keyword, the data set to be removed from LLA management.

When you have completed updating the data set, enter the MODIFY LLA,UPDATE=xx command again, this time specifying a CSVLLAxx parmlib member in which the keyword LIBRARIES specifies the name of the data set.

In any case, whenever multiple systems share access to an LLA-managed data set, STOP, START, or MODIFY commands must be entered for LLA on all the systems.

Using the FREEZE|NOFREEZE option

For an LLA-managed library, use the FREEZE|NOFREEZE option to indicate whether the system is to search the LLA-cached or DASD copy of a library directory. With FREEZE|NOFREEZE, which you code in the CSVLLAxx member, you specify on a library-by-library basis which directory copy the system is to search, as follows:

- If you specify FREEZE, the system uses the copy of the directory that is maintained in the LLA address space (the library is "frozen").
- If you specify NOFREEZE, the system searches the directory that resides in DASD storage.

The system always treats libraries in the LNKLST concatenation as frozen. Therefore, you need only specify the FREEZE option for non-LNKLST libraries, or for LNKLST libraries that are referenced through TASKLIBs, STEPLIBs, or JOBLIBs.

When an LLA-managed library is frozen, the following is true:

- Users of the library always receive versions of the library's modules that are pointed to by the LLA-cached directory.

- Users do not see any updates to the library until LLA is refreshed. If a user does multiple linkedits to a member in a FREEZE data set, the base for each subsequent linkedit does not include the previous linkedits; the base is the LLA version of the member.
- If the version of a requested module matches the version of the module in VLF, the users receive the module from VLF. Otherwise, users receive the module from DASD.

To take full advantage of LLA's elimination of I/O for directory search, specify FREEZE for as many read-only or infrequently updated libraries as appropriate.

Having NOFREEZE in effect for an LLA-managed library means that your installation does not eliminate I/O while searching the library's directory. However, LLA can still improve performance when the system fetches load modules from the VLF data space instead of DASD storage.

Table 11 on page 59 summarizes the affects of the FREEZE|NOFREEZE option on directory search and module fetch.

Table 11. FREEZE|NOFREEZE processing

Action	FREEZE or NOFREEZE	LNKLST Libraries Accessed From LNKLST	LNKLST Libraries Accessed Outside LNKLST	Other Non-LNKLST Libraries
Directory Search	FREEZE	LLA directory is used.	LLA directory is used.	LLA directory is used.
	NOFREEZE	LLA directory is used.	DASD directory is used (I/O occurs)	DASD directory is used (I/O occurs)
Module Fetch	FREEZE	Requestor receives LLA version of module (from VLF data space or DASD).	Requestor receives LLA version of module (from VLF data space or DASD).	Requestor receives LLA version of module (from VLF data space or DASD).
	NOFREEZE	Requestor receives LLA version of module (from VLF data space or DASD).	Requestor receives most current version of module (from DASD or VLF data space, if staged).	Requestor receives most current version of module (from DASD or VLF data space, if staged).

You can change the FREEZE|NOFREEZE status of an LLA-managed library at any time through the MODIFY LLA command. Changing a library from NOFREEZE to FREEZE also causes a refresh of the directory information for that library (note that when a library is refreshed, all of its modules are destaged from the VLF data space, which will slow performance until new versions are staged).

For more information about specifying the FREEZE|NOFREEZE option, see the description of the CSVLLAxx member in *z/OS MVS Initialization and Tuning Reference*.

Changing LLA-managed libraries

After changing a module in, or adding a module to, an LLA-managed library, IBM recommends that you refresh LLA for the library. A module that is changed and has a new location is not considered for staging until that member is refreshed.

The recommended way to make updates in the production system is to use IEBCOPY under FREEZE mode. The member to be updated should be copied to another data set, the linkedits run against the second data set and then the updated member can be copied back to the LLA-managed data set. If LLA-managed production libraries must be updated directly, LLA should be refreshed to manage the data set in NOFREEZE mode.

LLA ENQ consideration

By default, LLA allocates the libraries it manages as DISP=SHR. This means that if a job attempts to allocate an LLA-managed library as DISP=OLD, the job is enqueued until LLA is stopped or the library is removed from LLA management. Before adding a library to the libraries that LLA manages, review and, if necessary, change the jobs that reference the library.

Using the GET_LIB_ENQ keyword

The GET_LIB_ENQ keyword in the CSVLLAxx member allows you to prevent LLA from obtaining an exclusive enqueue on the libraries it manages. If you specify GET_LIB_ENQ (NO), your installation's jobs can update, move, or delete LLA-managed libraries while other users are accessing the libraries. GET_LIB_ENQ (NO) is generally not recommended, however, because of the risks it poses to data set integrity.

Compressing LLA-managed libraries

If you compress an LLA-managed library, LLA continues to provide the obsolete directory entries. For members that have been staged to the VLF data space, the system will operate successfully. If the member is not currently staged, however, the cached obsolete directory entry can be used to fetch the member at the old TTR location from DASD.

Because using obsolete directory entries can cause such problems as abends, breaches of system integrity, and system failures, use the following procedure to compress LLA-managed libraries:

1. Issue a MODIFY LLA,UPDATE=xx command, where the CSVLLAxx parmlib member includes a REMOVE statement identifying the library that needs to be compressed.
2. Compress the library
3. Issue a MODIFY LLA,UPDATE=xx command, where the CSVLLAxx parmlib member includes a LIBRARIES statement to return the compressed library to LLA management.

This procedure causes all members of that library to be discarded from the VLF data space. The members are then eligible to be staged again.

Recataloging LLA-managed data sets while LLA is active

LLA dynamically allocates the library data sets it manages. To re-catalog an LLA-managed library data set while LLA is active, do the following:

1. Remove the library data set from LLA. (Issue a MODIFY LLA,UPDATE=xx command, where *xx* identifies the suffix of the CSVLLAxx parmlib member that includes a REMOVE statement that identifies the library data set to be removed from LLA management.)
2. Recatalog the library data set.
3. Return the library data set to LLA. (Issue a MODIFY LLA,UPDATE=xx command, where *xx* identifies the suffix of the CSVLLAxx parmlib member that includes a LIBRARIES statement that identifies the recataloged library data set to be returned to LLA management.) Recataloged LNKLST libraries cannot be put back into LLA management. This causes fetch failures.

Allocation considerations

Before a job can execute, the operating system must set aside the devices, and space on the devices, for the data that is to be read, merged, sorted, stored, punched, or printed. In MVS, the "setting-aside" process is called allocation.

MVS assigns units (devices), volumes (space for data sets), and data sets (space for collections of data) according to the *data definition* (DD) and *data control block* (DCB) information included in the JCL for the job step.

When the data definition or DCB information is in the form of text units, the allocation of resources is said to be *dynamic*. Dynamic allocation means you are requesting the system to allocate and/or deallocate resources for a job step while it is executing.For details on the use of dynamic allocation, see *z/OS MVS Programming: Authorized Assembler Services Guide*.

Serialization of resources during allocation

When the system is setting aside non-sharable devices, volumes and data sets for a job or a step, it must prevent any other job from using those resources during the allocation process. To prevent a resource from changing status while it is being allocated to a job, the system uses serialization. Serialization during allocation causes jobs to wait for the resources and can have a major impact on system performance. Therefore, the system attempts to minimize the amount of time lost to serialization by providing a specific order of allocation processing. See Table 12 on page 61.

Knowing the order in which the system chooses devices, you can improve system performance by making sure your installation's jobs request resources that require the least possible serialization.

The system processes resource allocation requests in the order shown in Table 12 on page 61. As you move down the list, the degree of serialization – and processing time – increases.

Table 12. Processing order for allocation requests requiring serialization

Kinds of allocation requests	**Serialization required**
Requests requiring no specific units or volumes; for example, DUMMY, VIO, and subsystem data sets.	No serialization.
Requests for sharable units: DASD that have permanently resident or reserved volumes mounted on them.	No serialization.
Teleprocessing devices.	Serialized on the requested devices.
Pre-mounted volumes, and devices that do not need volumes.	Serialized on the group(s) of devices eligible to satisfy the request. A single generic device type is serialized at a time.
Online, nonallocated devices that need the operator to mount volumes.	Serialized on the group(s) of devices eligible to satisfy the request. A single generic device type is serialized at a time.
All other requests: offline devices, nonsharable devices already allocated to other jobs.	Serialized on one or more groups of devices eligible to satisfy the request. A single generic device type is serialized at a time.

Improving allocation performance

You can contribute to the efficiency of allocation processing throughout your installation in several ways:

- For devices:
  - Use the device preference table, specified through the Hardware Configuration Definition (HCD) to set up the order for device allocation. See the *z/OS MVS Device Validation Support* appendix for a listing of the device preference table installation's devices as esoteric groups, and to group them for selection by allocation processing.
  - Use the eligible device table (EDT) to identify the I/O devices that you want to include in the esoteric groups.

    For more information on the device preference table and the EDT, see *z/OS HCD Planning*.
- For volumes, use the VATLSTxx members of SYS1.PARMLIB to specify volume attributes at IPL.
- For data sets, you can specify the JCL used for your installation's applications according to the device selection criteria you have set up through the HCD process and IPL.

The volume attribute list

In MVS, each online device in the installation has a mount attribute and each mounted volume has a use attribute.

The mount attributes determine when the volume on that device should be removed. This information is needed when selecting a device and during step unallocation processing. The mount attributes are:

- Permanently resident
- Reserved
- Removable.

The use attributes determine the type of nonspecific requests eligible to that volume. The use attributes are:

- Public
- Private
- Storage.

Allocation routines use the volumes' mount and use attributes in selecting devices to satisfy allocation requests. Thoughtful selection of the use and mount attributes is important to the efficiency of your installation. For example, during allocation, data sets on volumes marked permanently resident or reserved are selected first because they require no serialization, thus minimizing processing time.

During system initialization, you can assign volumeattributes to direct access volumes by means of the volume attribute list. The volume attribute list is defined at IPL time using the VATLSTxx member of SYS1.PARMLIB.

After an IPL, you can assign volume attributes to both direct access and tape volumes using the MOUNT command. The USE= parameter on the MOUNT command defines the use attribute the volume is to have; a mount attribute of reserved is automatic. For the details of using the MOUNT command, see *z/OS MVS System Commands*.

When volumes are not specifically assigned a use attribute in the VATLSTxx member or in a MOUNT command, the system assigns a default. You can specify this default using the VATDEF statement in VATLSTxx. If you do not specify VATDEF, the system assigns a default of public. For details on including a volume attribute list in IEASYSxx,and on coding the VATLSTxx parmlib member itself, see *z/OS MVS Initialization and Tuning Reference*.

Use and mount attributes

Every volume is assigned use and mount attributes through an entry in the VATLSTxx member at IPL, a MOUNT command, or by the system in response to a DD statement.

The relationships between use and mount attributes are complex, but logical. The kinds of devices available in an installation, the kinds of data sets that will reside on a volume, and the kinds of uses the data sets will be put to, all have a bearing on the attributes assigned to a volume. Generally, the operating system establishes and treats volume attributes as outlined in the following sections.

Use attributes

- Private — meaning the volume can only be allocated when its volume serial number is explicitly or implicitly specified.
- Public — meaning the volume is eligible for allocation to a temporary data set, provided the request is not for a specific volume and PRIVATE has not been specified on the VOLUME parameter of the DD statement.

  Both tape and direct access volumes are given the public use attribute.

  A public volume may also be allocated when its volume serial number is specified on the request.
- Storage — meaning the volume is eligible for allocation to both temporary and non-temporary data sets, when no specific volume is requested and PRIVATE is not specified. Storage volumes usually contain non-temporary data sets, but temporary data sets that cannot be assigned to public volumes are also assigned to storage volumes.

Mount attributes

- Permanently resident — meaning thevolume cannot be demounted. Only direct access volumes can be permanently resident. The following volumes are always permanently resident:
  - All volumes that cannot be physically demounted, such as drum storage and fixed disk volumes
  - The IPL volume
  - The volume containing the system data sets

  In the VATLSTxx member, you can assign a permanently-residentvolume any of the three use attributes. If you do not assign a use attribute to a permanently-resident volume, the default is public.
- Reserved — meaning the volume is to remain mounted until the operator issues an UNLOAD command.

  Both direct access and tape volumes can be reserved because of the MOUNT command; only DASD volumes can be reserved through the VATLSTxx member.

  The reserved attribute is usually assigned to a volume that will be used by many jobs to avoid repeated mounting and demounting.

  You can assign a reserved *direct access* volume any of the three use attributes, through the USE parameter of the MOUNT command or the VATLSTxx member, whichever is used to reserve the volume.

  A reserved *tape* volume can only be assigned the use attributes of private or public.
- Removable — meaning that the volume is not permanently resident or reserved. A removable volume can be demounted either after the end of the job in which it is last used, or when the unit it is mounted on is needed for another volume.

  You can assign the use attributes of private or public to a removable *direct access* volume, depending on whether VOLUME=PRIVATE is coded on the DD statement: if this subparameter is coded, the use attribute is private; if not, it is public.

  You can assign the use attributes of private or public to a removable *tape* volume under one of the following conditions:

  - Private
    - The PRIVATE subparameter is coded on the DD statement.
    - The request is for a specific volume.
    - The data set is nontemporary (not a system-generated data set name, and a disposition other than DELETE).

    Note: The request must be for a **tape only** data set. If, for example, an esoteric group name includes both tape and direct access devices, a volume allocated to it will be assigned a use attribute of public.
  - Public
    - The PRIVATE subparameter is not coded on the DD statement.
    - A nonspecific volume request is being made.
    - The data set is temporary (a system-generated data set name, or a disposition of DELETE).

Table 13 on page 63 summarizes the mount and use attributes and how they are related to allocation requests.

Table 13. Summary of mount and use attribute combinations

Volume State	Temporary Data Set	Nontemporary data set	How Assigned	How Unmounted
	Type of Volume Request			
Public and Permanently Resident (see note)	Nonspecific or Specific	Specific	VATLSTxx entry or by default	Always mounted

Table 13. Summary of mount and use attribute combinations (continued)

Volume State	Temporary Data Set	Nontemporary data set	How Assigned	How Unmounted
	Type of Volume Request			
Private and Permanently Resident (see note)	Specific	Specific	VATLSTxx entry	Always mounted
Storage and Permanently Resident (see note)	Nonspecific or Specific	Nonspecific or Specific	VATLSTxx entry	Always mounted
Public and Reserved (Tape and direct access)	Nonspecific or Specific	Specific	Direct access: VATLSTxx entry or MOUNT command; Tape: MOUNT command	UNLOAD or VARY OFFLINE commands
Private and Reserved (Tape and direct access)	Specific	Specific	Direct access: VATLSTxx entry or MOUNT command ;Tape: MOUNT command	UNLOAD or VARY OFFLINE commands
Storage and Reserved (see note)	Nonspecific or Specific	Nonspecific or Specific	VATLSTxx entry or MOUNT command	UNLOAD or VARY OFFLINE commands
Public and Removable (Tape and direct access)	Nonspecific or Specific	Specific	VOLUME=PRIVATE is not coded on the DD statement. (For tape, nonspecific volume request and a temporary data set also cause this assignment.)	When unit is required by another volume; or by UNLOAD or VARY OFFLINE commands.
Private and Removable (Tape and direct access)	Specific	Specific	VOLUME=PRIVATE is coded on the DD statement. (For tape, a specific volume request or a nontemporary data set also cause this assignment).	At job termination for direct access; at step termination or dynamic unallocation for tape (unless VOL=RETAIN or a disposition of PASS was specified); or when the unit is required by another volume.

The nonsharable attribute

Some allocation requests imply the exclusive use of a direct access device while the volume is mounted or unmounted. The system assigns the **nonsharable attribute** to volumes that might require demounting during step execution.

When a volume is thus made non-sharable, it cannot be assigned to any other data set until the non-sharable attribute is removed at the end of step execution.

The following types of requests cause the system to automatically assign the nonsharable attribute to a volume:

- A specific volume request that specifies more volumes than devices.
- A nonspecific request for a *private* volume that specifies more volumes than devices.
- A volume request that includes a request for unit affinity to a preceding DD statement, but does not specify the same volume for the data set. For more information, see the discussion of unit affinity in *z/OS MVS JCL Reference*.
- A request for deferred mounting of the volume on which a requested data set resides.

Except for one situation, the system will not assign the non-sharable attribute to a permanently-resident or reserved volume. The exception occurs when the allocation request is for more volumes than units, and one of the volumes is reserved. The reserved volume is to share a unit with one or more *removable* volumes, which precede it in the list of volume serial numbers.

Consider the following example, where volume A is removable and volume B is reserved. In this example, *both* volumes are assigned the non-sharable attribute; neither of them can be used in another job at the same time. To avoid this situation, do one of the following:

- Specify the same number of volumes as units
- Specify parallel mounting
- Set the mount attribute of volume A as resident or reserved.

```
DSN=BCA.ABC,VOL=SER=(A,B),UNIT=DISK
```

System action

Table 14 on page 65 shows the system action for sharable and non-sharable requests.

Table 14. Sharable and nonsharable volume requests

The Request is:	**The Volume is Allocated:**	
	Sharable	**Nonsharable**
Sharable	allocate the volume	wait (see note)
Nonsharable	wait (see note)	wait (see note)
Note: The operator has the option of failing the request. The request will always fail if waiting is not allowed.		

For more detailed information on how an application's JCL influences the processing of allocation requests, see *z/OS MVS JCL Reference*.

For details on how dynamic allocation affects the use attributes of the volumes in your installation, see *z/OS MVS Programming: Assembler Services Guide*.

Chapter 2. Auxiliary storage management initialization

This topic describes the effective initialization and use of paging, which can use page data sets only, or page data sets in addition to optional storage-class memory (SCM).

Paging is the process that z/OS uses to select which pages to move from central storage to auxiliary storage. Auxiliary storage requires page data sets on DASD, and can also include the optional SCM on Flash Express (SSD) cards or the Virtual Flash Memory (VFM). If SCM is available and online, ASM uses both SCM and page data sets for auxiliary storage by paging data to the preferred storage medium, based on response times and additional factors.

Adding SCM to your system can increase the flexibility and performance of your paging operations. However, because SCM is not persistent across IPLs, SCM cannot be used for paging Virtual I/O (VIO) data or PLPA data for warm starts.

For additional information on ASM, refer to "Auxiliary storage overview" on page 48, and the PAGE, PAGESCM, NONVIO and PAGTOTL parameters of parmlib member IEASYSxx in *z/OS MVS Initialization and Tuning Reference*.

Page operations

Auxiliary storage manager (ASM) paging controllers attempt to maximize I/O efficiency by incorporating a set of algorithms to distribute the I/O load as evenly as is practical. In addition, priority is given to keeping the system operable in situations where a shortage of a specific type of page space exists.

If you are using optional storage-class memory (SCM) in addition to the required page data sets, ASM selects the optimal paging medium by comparing the observed latency times for I/O (response times). To ASM, the response time is the average time that it takes to complete an I/O request divided by the number of pages that are serviced by a request. ASM also analyzes data characteristics, critical paging requirements, availability of the DASD (during a HyperSwap failover, for example) and available storage space before selecting a paging target.

A HyperSwap failover is a data availability mechanism that permits replacing a DASD with a backup DASD. During a HyperSwap failover the DASD is not available for paging, so address spaces in main memory, some of which are critical, cannot be paged out to the disk. In this case, SCM can be used for critical address space paging to reduce the risk of critical data loss.

Paging operations and algorithms

To page efficiently and expediently, ASM divides z/OS system pages into classes, namely PLPA, common and local. Contention is reduced when these classes of pages are placed on different physical devices. Multiple local page data sets are recommended. Although the system requires only one local page data set, performance can be improved when local page data sets are distributed across multiple devices, even if one device is large enough to hold the entire amount of required page space.

The PLPA and common page data sets are optional if storage-class memory (SCM) is available (specify *NONE* to use SCM), but there can be only one of each. Spillage back and forth between the PLPA and common page data sets is permissible, but in the interest of performance, only spilling from PLPA to common should be permitted.

The general intent of the ASM algorithms for page data set selection construction is to:

- *Use all available local page data sets:* When ASM writes a group of data, it selects a local page data set in a circular order within each type of device, considering the availability of free space and device response time.

When ASM selects a data set, the paging data sets that reside on Parallel Access Volume (PAV) devices are examined first because of reliability and performance characteristics. Because preference is given to PAV devices, it is normal to have a higher usage of PAV data sets as compared to non-PAV data sets.

- *Write group requests to contiguous slots:* ASM selects contiguous space in local page data sets on moveable-head devices to receive group write requests. For certain types of requests, ASM's slot allocation algorithm tries to select sequential (contiguous) slots within a cylinder. The reason for doing this is to shorten the I/O access time needed to read or write a group of requests. For other types of requests (such as an individual write request), or if there are no sequential slots, ASM selects any available slots.
- *Limit the apparent size of local page data sets to reduce seek time:* If possible, ASM concentrates group requests and individual requests that are within a subset of the space allocated to a local page data set.

Paging operations and algorithms for storage-class memory (SCM)

On IBM zEnterprise® EC12 (zEC12), BC12 (zBC12) and later processors, storage-class memory (SCM) implemented through the optional Flash Express feature or the Virtual Flash Memory (VFM) can be used for auxiliary storage in conjunction with page data sets.

Once page data sets on DASD are selected by ASM as the preferred storage medium, all of the factors in “Paging operations and algorithms” on page 67 remain applicable. However, if ASM selects SCM as the preferred storage medium, then additional operations and algorithms apply.

Because SCM does not support persistence of data across IPLs, VIO data can only be paged out to DASD. Therefore, even when SCM is installed you must still maintain a minimum amount of storage that supports paging for all of your VIO data, and a minimum amount of local paging data sets. All other data types can be paged out to SCM.

With the use of SCM, all PLPA pages can be stored on both SCM and optionally, on the PLPA data set. If the PLPA page data set exists, it is used during warm starts, and the PLPA on SCM is used to resolve any page faults. Resolving PLPA page faults on SCM provides system resiliency, particularly during HyperSwap failovers.

Table 15 on page 68 summarizes the criteria that ASM uses to determine which storage medium to use for paging to auxiliary storage from central storage.

Table 15. ASM criteria for paging to storage-class memory (SCM) or page data sets

Main memory data type	**ASM selection criteria for paging to SCM or DASD**
PLPA	At IPL/NIP, PLPA pages can be paged to both SCM and the PLPA data set. If the PLPA data set exists, it is used for warm starts, and SCM is used to resolve PLPA page faults.
VIO (Virtual I/O)	VIO data is paged to local page data sets only; first to VIO-accepting data sets, and then to any overflow to non-VIO data sets.
HyperSwap Critical Address Space data	If SCM space is available, all pages that are assigned to a HyperSwap Critical Address Space are paged to SCM. If SCM space is not available, HyperSwap pages are kept in main memory and are only paged to page data sets if real storage becomes constrained and no other alternative exists.
Pageable large pages	If contiguous SCM space is available, pageable large pages are paged to SCM. If contiguous SCM is not available, large pages are demoted to SCM or DASD as 4 K page data sets, depending on service request response times. If SCM is selected, the large page is demoted to 256 4-K blocks and stored across noncontiguous SCM. If paging data sets is selected, the large page is also demoted to 256 4-K blocks and stored across noncontiguous 4 K blocks on paging data sets.

Table 15. ASM criteria for paging to storage-class memory (SCM) or page data sets (continued)	
Main memory data type	**ASM selection criteria for paging to SCM or DASD**
All other data types, including Common, Local, and Private Area data	If available space exists on both page data sets and on SCM, the system allocates data to the preferred storage medium based on response times. If the CriticalPaging function is active, data in Common and address spaces are subject to critical paging.

When using or planning to use SCM, the PAGESCM IEASYSxx parameter specifies the amount of SCM that is to be reserved for auxiliary storage.

When setting the PAGESCM value, you need to take some items into consideration.

When PAGESCM=ALL (default) or some value other than NONE:

- Should be specified if SCM is available and are used for auxiliary storage. If SCM is not currently available but is planned to be used at some time during this IPL, specifying a PAGESCM value allows SCM to be subsequently brought online and used for auxiliary storage without the need for an IPL
- Pageable large pages are defined, regardless of whether SCM is installed. Pageable large pages are requested via the IARV64, DSPSERV, CPOOL and STORAGE OBTAIN services.

  If SCM is not in use and the need arises to page out pageable large page, the page will be paged-out as 256 4-K pages thus losing some of benefits that are provided by the large page.
- When specifying a value other than NONE, it is recommended to specify ALL (or take the default of ALL) since auxiliary storage is currently the only exploiter of SCM in z/OS. This value will need to be reexamined if other exploiters are introduced.

When PAGESCM=NONE:

- Indicates that SCM will not be used during this IPL. Subsequent use of SCM will require an IPL with a PAGESCM value other than none.

Configuring storage-class memory (SCM)

The amount of storage-class memory (SCM) that is available to a z/OS system is specified using the Manage Flash Allocation dialog from the SE/HMC Configuration menu. This dialog also specifies the amount of SCM that is initially brought online to the LPAR, after which additional SCM can be brought online using the `CONFIG ONLINE` command. For complete command usage information refer to *z/OS MVS System Commands*.

z/OS currently supports up to 16 TB of SCM within a single system image, but the maximum amount of SCM that is available to the central-processing complex (CPC) is limited by the hardware model. SCM and the data that resides on it can only be accessed from the partition on which the SCM was defined.

Page data set sizes

The size of a page data set can affect system performance. The maximum number of slots that a page data set can be is 16,777,215. However, the amount of available free space on the volume that the page data set is allocated to limits the size that can be allocated. A 3390 device with 65,520 cylinders contains 11,793,600 slots.

Note: If you are using storage-class memory (SCM), the size of your page data sets can be reduced, assuming that SCM has demonstrated faster I/O response times.

Note the following recommendations:

- *PLPA data set.* The total combined size of the PLPA page data set and common page data set cannot exceed the size of the PLPA (and extended PLPA) plus the amount that is specified for the CSA and the size of the MLPA. In defining the size of these data sets, a reasonable starting value might be four megabytes for PLPA and 20 megabytes for common, as spilling occurs if the PLPA data set becomes full.

RMF reports can be used to determine the size requirements for these data sets. During system initialization, check the size of the page data sets in the page data set activity report. If the values of the used slots are very close to the values of the allocated slots, the size should be enlarged. For more information, see the activity report of the page data set in *z/OS Resource Measurement Facility Report Analysis*.

- *Common page data set.* The common page data set should be large enough to contain all of the common area pages plus room for any expected PLPA spill. Although it is possible for the common page data set to spill to the PLPA page data set, this situation should not be allowed to occur because it might heavily affect performance. As noted for the PLPA data set, a reasonable starting size for the common page data set might be twenty megabytes.
- *Local page data sets.* The local page data sets must be large enough to hold the private area and VIO pages that cannot be contained in real storage. To ensure an even distribution of paging requests across as many data sets as possible, all local paging data sets should be approximately the same size.

  Note: When all local paging data sets are not the same size, the smaller data sets might reach a full condition sooner than the large data sets. This reduces the number of data sets that are available to ASM for subsequent paging requests. Depending on the paging configuration, this situation could degrade paging performance because the I/O workload and contention for the remaining available data sets might increase.

  To minimize the path length in the ASM slot selection code (bitmap search), plan for local page data sets to not exceed 30% of their capacity under normal system workloads.

Storage requirements for page data sets

ASM allocates storage in extended system queue area (ESQA) for every page data set that is in use. For data sets that are defined during IPL, this storage is obtained during IPL. For data sets that are added dynamically after IPL, this storage is obtained during the processing of the PAGEADD command.

Regardless of the page data set size, the ESQA consists of the following:

- A fixed amount of bytes (approximately 32,000 bytes)
- A variable amount that is determined by the size of the data set (24 bytes for each cylinder)

Page data set protection

The page data set protection feature was introduced in z/OS V1R3 to help guard you from unintentionally IPLing with a page data set that is already in use. It does this by formatting and maintaining a status information record at the beginning of each page data set and by using an ENQ to serialize usage of the data sets.

The page data set protection feature prevents two systems from accidentally using the same physical data set. However, it is not possible to prevent the same data sets from being used when:

- the request to use the data set comes from a system outside of the GRS Ring/Star configuration.
- the installation has excluded the data sets from multi-system serialization. See *z/OS MVS Planning: Global Resource Serialization* for more information on SYSTEMS Exclusion RNL.

Page data sets are protected by a two-tier mechanism using:

- “SYSTEMS level ENQ” on page 70 .
- “Status information record” on page 71 .

SYSTEMS level ENQ

Page data sets are protected with a SYSTEMS level ENQ that contains the name of the page data set and the volser it resides on. The qname used on the ENQ is SYSZILRD, and the rname used on the ENQ is the dsname + volser. This ENQ is obtained during master scheduler initialization for the IPL-specified page data sets. For example, the ENQ would be obtained from the definitions in the IEASYSxx parmlib member. The ENQ is also obtained whenever a page data set is added or replaced after IPL.

A warning-level message is issued if the ENQ cannot be obtained successfully during IPL for the IPL-specified page data sets. Processing for the command is terminated if the ENQ cannot be obtained for a page data set that is being added or replaced with the PAGEADD or PAGEDEL command.

Status information record

A data set status information record is written to every page data set. The status record identifies the system using the data set.

The status record is validated during IPL. If the record indicates that the data set is in use by another system, message ILR030A is issued and the system waits for an operator response to allow or disallow use of the data set.

Note that this feature does not offer full protection in the case of a page data set that was defined on, or was last used by a pre-z/OS V1R3 environment. This is because the status record used to perform this check did not exist prior to V1R3. The z/OS V1.3 system will format the status record, issue ILR029I as an informational message and continue to use the data set (along with the other system).

Once the IPL is complete, the status record is validated on a regular interval. If concurrent use of a data set is detected, both systems will be terminated with a 02E wait state code. Catalog information will also be validated with the status record. If the data set is deleted or key catalog information changes, the system will be terminated with a 02E wait state code.

Space calculation examples

Table 16 on page 71 shows the values for page data sets. The examples following these figures show how to apply their tabular information to typical initialization considerations.

Note: After the system is running, you can use RMF reports to determine the sizes of page data sets. RMF reports provide the minimum, maximum, and average number of slots in use for page data sets. Thus, you can use the reports to adjust data set sizes, as needed.

Table 16. Page data set values

Device Type	Slots/Cyl	Cyl/Meg
3380 3390	150 180	1.7 1.42

Example 1: Sizing the PLPA page data set, size of the PLPA and extended PLPA unknown

Define the PLPA page data set to hold four megabytes; if that amount of space is exceeded, the remainder can be placed on the common page data set until the PLPA value is determined exactly.

Therefore: From the tables, 7 cylinders on a 3380 are needed for the 4-megabyte PLPA page data set. For the 3390, 6 cylinders are needed for the 4-megabyte PLPA page data set.

Example 2: Sizing the PLPA page data set, size of the PLPA and extended PLPA known

Assume the PLPA size is known to be 10 megabytes. Define the PLPA page data set to hold 10 megabytes plus 5%, or 10.5 megabytes. (The extra 5% allows for loss of space as a result of permanent I/O errors.)

Therefore: From the tables, 18 cylinders on a 3380 are needed for the 10.5-megabyte PLPA page data set. For the 3390, 15 cylinders are needed for the 10.5-megabyte PLPA page data set.

Note: This example provides no warm start capability. If installed, SCM can provide warm start capability.

Example 3: Sizing the common page data set

Use the size of the virtual common service area (CSA) and extended CSA, defined by the CSA= parameter in the IEASYSxx parmlib member, as the minimum size for the common page data set. If the system is IPLed with a specification of CSA=(3000,80000), then the total CSA specified is 83000 kilobytes (approximately 81 megabytes). However, CSA and extended CSA always end on a segment boundary, so the system may round the size up by as much as 1023 kilobytes each. That rounding could make the CSA size as large as 83 megabytes with the CSA=(3000,80000) specification. After the system is running, you can use RMF reports to determine how much of the common page data set is being used and adjust the size of the data set accordingly.

Therefore: From the tables, 118 cylinders on a 3390 are needed to start with an 83-megabyte common page data set.

Note: If you are using storage-class memory (SCM), the size of your common page data set can be reduced, assuming that SCM has demonstrated faster I/O response times.

Note: This example provides no warm start capability. If installed, SCM can provide warm start capability.

Example 4: Sizing local page data sets

Assume that the master scheduler address space and JES address space can each use about eight megabytes of private area storage. Next, determine the number of address spaces that will be used for subsystem programs such as VTAM and the system component address spaces, and allow eight megabytes of private area storage for each. To determine the amount of space necessary for batch address spaces, multiply the maximum number of batch address spaces that will be allowed to be active at once by the average size of a private area (calculated by the installation or approximated at eight megabytes).

Note: If you are using storage-class memory (SCM), the size of your local page data sets can be reduced, assuming that SCM has demonstrated faster I/O response times.

To determine the amount of space necessary for TSO/E, multiply the maximum number of TSO/E address spaces allowed on the system at once by the average size of a private area (calculated by the installation or approximated at eight megabytes).

Next, determine the amount of space required for any large swappable applications which run concurrently. Use the allocated region size for the calculation.

Finally, estimate the space requirements for VIO data sets. Approximate this requirement by multiplying the expected number of VIO data sets used by the entire system by the average size of a VIO data set for the installation. After the system is fully loaded, you can use RMF reports to evaluate the estimates.

Note: If your local DASD storage is not large enough to contain your VIO data, VIO data will not be paged out to SCM.

For example purposes, assume that the total space necessary for local page data sets is:

- 8 megabytes for the master scheduler address space
- 8 megabytes for the PC/AUTH address space
- 8 megabytes for the system trace address space
- 8 megabytes for the global resource serialization address space
- 8 megabytes for the allocation address space
- 8 megabytes for the communications task address space
- 8 megabytes for the dumping services address space
- 8 megabytes for the system management facilities address space
- 8 megabytes for the VTAM address space
- 8 megabytes for the JES address space

8 megabytes for the JES3AUX address space if JES3 is used

10 megabytes for the batch address space (10 batches x 1 megabyte each)

50 megabytes for TSO/E address spaces (50 TSO/E users x 1 megabyte each)

102 megabytes for large swappable application

+ 40 megabytes for VIO data sets (200 data sets x 0.2 megabyte each)

290 megabytes total + 15 meg (approx. 5%) buffer = 305 megabytes

Therefore: From the tables, 549 cylinders on 3380 type devices are necessary. For the 3390, 434 cylinders are necessary.

Note: Even when the local page data sets will fit on one 3390, you should spread them across more than one device. See performance recommendation number 5. If large swappable jobs or large VIO users are started and there is insufficient allocation of space on local page data sets, a system wait X'03C' could result.

The installation should also consider the extent of the use of data-in-virtual when calculating paging data set requirements. Users of data-in virtual may use sizable amounts of virtual storage which may put additional requirements on paging data sets.

The calculations shown here will provide enough local page space for the system to run. However, if a program continually requests virtual storage until the available local page data set space is constrained, this will not be enough space to prevent an auxiliary storage shortage.

Auxiliary storage shortages can cause severe performance degradation, and if all local page data set space is exhausted the system might fail. To avoid this, you can do one or more of the following:

- Use the IEFUSI user exit to set REGION limits for most address spaces and data spaces. If you have sufficient DASD, add enough extra local page data set space to accommodate one address space or data space of the size you allow in addition to the local page data set space the system usually requires, multiply the sum by 1.43, and allocate that amount of local page data set space. For more information about IEFUSI, see *z/OS MVS Installation Exits*.
- Overallocate local page data set space by 300% if you have sufficient DASD. This is enough auxiliary storage to prevent the system from reaching the auxiliary storage shortage threshold when the maximum amount of storage is obtained in a single address space or data space.

Example 5: Sizing page data sets when using storage-class memory (SCM)

You can replace some of your DASD paging space with SCM. VIO paging, however, must still be paged to DASD so adequate DASD storage is required to avoid local storage overruns.

For example, if your VIO local page data sets require a given amount of storage, but you have only one local storage DASD allocated for VIO, local storage could become critical. In this case, allocating three times the amount of VIO space required across several DASDs can provide capacity for local storage overruns and also for disk failover.

When migrating to SCM, be aware of the following recommendations:

- Maintain your original paging data set configuration while you integrate SCM into your system configuration.
- Once SCM has become fully integrated into your system configuration, you can choose to reduce the number of local paging data sets.
- Maintain sufficient local paging data space to accommodate VIO pages, because VIO pages are not written to SCM.

Performance recommendations

The following recommendations can improve system performance through the careful use of paging data sets and devices.

1. Allocate only one paging data set per device. Doing this reduces contention among more than one data set for the use of the device. If you do define more than one paging data set for each device, the use of Parallel Access Volume (PAV) devices can help to reduce contention for a device.

   Reason: If there is more than one paging data set on the same device, a significant seek penalty is incurred. Additionally, if the data sets are local page data sets, placing more than one on a device can cause the data sets to be selected less frequently to fulfill write requests.

   Comments: You might, however, place low-activity non-paging data sets on a device that holds a page data set and check for device contention by executing RMF to obtain direct access device activity reports during various time intervals. The RMF data on device activity count, percent busy, and average queue length should suggest whether device contention is a problem. RMF data for devices that contain only a page data set can be used as a comparison base.

2. Over-specify space for all page data sets.

   Reason: Over-specifying space allows for the creation of additional address spaces before the deletion of current ones and permits some reasonable increase in the number of concurrent VIO data sets which may be backed by auxiliary storage. VIO data set growth might become a problem because there is no simple way to limit the total number of VIO data sets used by multiple jobs and TSO/E sessions. VIO data set paging can be controlled by restricting it to certain page data sets through the use of directed VIO.

   VIO data set pages can be purged by a reIPL specifying CVIO (or CLPA). CVIO indicates that the system is to ignore any VIO pages that exist on the page data sets and treat the page data sets initially as if there is no valid data on them (that is, there are no allocated slots). Thus, specifying CVIO prevents the warm start of jobs that use VIO data sets because the VIO pages have been purged. (For additional space considerations, see the guideline for estimating the total size of paging data sets in "Estimating total size of paging data sets" on page 75.)

   In all cases, ASM avoids scattering requests across large, over-specified, page data sets by concentrating its activity to a subset of the space allocated.

3. Use more than one local page data set, each on a unique device, even if the total required space is containable on one device.

   Reason: When ASM uses more than one data set, it can page concurrently on more than one device. This is especially important during peak loads.

4. Distribute ASM data sets among channel paths and control units.

   Reason: Although ASM attempts to use more than one data set concurrently, the request remains in the channel subsystem queues if the channel path or control unit is busy.

5. Dedicate channel paths and control units to paging devices.

   Reason: In heavy paging environments, ASM can use the path to the paging devices exclusively for page-ins and page-outs and avoid interference with other users, such as IMS.

6. Make a page data set the only data set on the device.

   Reason: Making a paging data set the only data set on the device enables ASM to avoid contention. ASM can monopolize the device to its best performance advantage by controlling its own I/O processing of that data set. ASM does not have to perform the additional processing that it would otherwise have to perform if I/O for any other data set, especially another page data set, were on the same device. If another data set must be placed on the device, select a low-use data set to minimize contention with the page data set.

7. Do not share volumes that contain page data sets among multiple systems.

   Reason: While page data sets may be defined on volumes that contain shared non-paging data sets, they cannot be shared between systems.

8. Take one of the following actions to control the risk of auxiliary storage shortages. Auxiliary storage shortages have severe effects on system performance when they occur, and can also cause the system to fail. When a shortage occurs, the system rejects LOGON, MOUNT, and START commands and keeps

address spaces with rapidly increasing auxiliary storage requirements from running until the shortage is relieved.

a. Use the SMF Step Initiation Exit, IEFUSI, to limit the sizes of most address spaces and data spaces.

b. If you do not establish limits using IEFUSI, consider over-allocating local page space by an amount sufficient to allow a single address space or data space to reach the virtual storage limit (as might happen if a program looped obtaining storage) without exhausting virtual storage shortage.

See "Example 4: Sizing local page data sets" on page 72 for more information about calculating local page data set space requirements.

Estimating total size of paging data sets

You can obtain a general estimate of the total size of all paging data sets by considering the following space factors.

1. The space needed for the common areas of virtual storage (PLPA and extended PLPA, MLPA, and CSA).
2. The space needed for areas of virtual storage: private areas of concurrent address spaces, and concurrently existing VIO page data sets. (The system portion of concurrent address spaces needs to be calculated only once, because it represents the same system modules.)
3. If you add storage-class memory (SCM) to your system, the required sizes of your paging data sets for VIO and local page data sets must be large enough to accommodate VIO workload, plus additional space for paging spikes beyond what SCM can accommodate.

Using measurement facilities

You can possibly simplify the space estimate for the private areas mentioned in "Estimating total size of paging data sets" on page 75 by picking an arbitrary value. Set up this amount of paging space, and then run the system with some typical job loads. To determine the accuracy of your estimate, start RMF while the jobs are executing. The paging activity report of the measurement program gives data on the number of unused 4K slots, the number of VIO data set pages backed by auxiliary storage, address space pages, and the number of unavailable (defective) slots.

The RMF report also contains the average values based on a number of samples taken during the report interval. These average values are in a portion of the report entitled "Local Page data set Slot Counts". They are somewhat more representative of actual slot use because slot use is likely to vary during the report interval. The values from the paging activity report should enable you to adjust your original space estimate as necessary.

Adding paging space

To add paging space for page data sets on DASD HDDs, you must use the DEFINE PAGESPACE command of Access Method Services to pre-format and catalog each new page data set. To add the page data set, you can use the PAGEADD operator command, or specify the data set at the next IPL on the PAGE parameter.

To dynamically add paging space to storage-class memory (SCM) on Flash Express SSDs or the Virtual Flash Memory (VFM), use the `CONFIG ONLINE` command. For complete command usage information refer to *z/OS MVS System Commands*.

For more information refer to the following information:

- See *z/OS DFSMS Access Method Services Commands* for information about the DEFINE PAGESPACE command, and on the related commands - ALTER and DELETE - used for the handling of VSAM data sets.
- See *z/OS MVS System Commands* for a description of the PAGEADD command.
- See the description of the PAGE parameter in parmlib member IEASYSxx in *z/OS MVS Initialization and Tuning Reference*.

Deleting, replacing or draining page data sets

You might need to remove a local page data set from the system for any of the following reasons:

- The hardware is being reconfigured.
- The hardware is generating I/O errors.
- The configuration of the page data set is being changed.
- System tuning requires the change.

To remove local page data sets or SCM from your system, use the CONFIG OFFLINE command.

The PAGEDEL command allows you to delete, replace or drain local page data set without an IPL, though the command can be disruptive and must be used judiciously. See *z/OS MVS System Commands* for the description of the PAGEDEL command.

ASM will reject a PAGEDEL command that will decrease the amount of auxiliary storage below an acceptable limit. ASM determines what is acceptable by examining SRM's auxiliary storage threshold constant, MCCASMTI. If it is determined that too much storage would be deleted by the PAGEDEL command, ASM will fail the page delete request.

Questions and answers

The following questions and answers describe ASM functionality:

Q:
Does ASM use I/O load balancing?

A:
Yes, ASM does its own I/O load balancing.

When selecting a local page data set to fulfill a write request, ASM attempts to avoid overloading page data sets. ASM also attempts to favor those devices or channel paths that are providing the best service. If SCM demonstrates a performance advantage, then SCM is selected over any page data set.

Q:
How does the auxiliary storage shortage prevention algorithm in SRM prevent shortages?

A:
It does so by swapping out address spaces that are accumulating paging space at a rapid rate. Page space is not immediately freed, but another job or TSO/E session (still executing) will eventually complete and free page space. SRM also prevents the creation of new address spaces and informs the operator of the shortage so that he can optionally cancel a job.

Q:
Is running out of auxiliary storage (paging space) catastrophic?

A:
No, not necessarily; it might be possible to add more page data sets with the PAGEADD operator command, optionally specifying the NONVIO system parameter. It may be necessary to reIPL to specify an additional pre-formatted and cataloged page data set. (See the description of the PAGE parameter of the IEASYSxx member in *z/OS MVS Initialization and Tuning Reference*.)

Q:
Can we dynamically allocate more paging space?

A:
Yes. Additional paging space may be added with the PAGEADD operator command if the PAGTOTL parameter allowed for expansion (see the description of the PAGTOTL parameter of the IEASYSxx member in *z/OS MVS Initialization and Tuning Reference* and the PAGEADD command in *z/OS MVS System Commands*).

If you are using storage-class memory (SCM), you can dynamically allocate additional paging space to SCM using the `CONFIG ONLINE` command.

Q:

Can we remove paging space from system use?

A:

Yes. Use the PAGEDEL command for local page data sets.

Q:

How does ASM select slots?

A:

ASM selects slots when writing out pages to page data sets based on whether the write request is an individual request or a group request. For an individual write request, such as a request to write stolen pages (those pages changed since they were last read from the page data set), ASM selects any available slots. For a group write request, such as a request that results from a VIO move-out of groups of pages to page data sets, ASM attempts to select available slots that are contiguous. ASM also attempts to avoid scattering requests across large page data sets.

If you are using storage-class memory (SCM), ASM pages to contiguous blocks of SCM, if available.

Q:

How does ASM select a local page data set for a page-out?

A:

ASM selects a local page data set for page-out from its available page data sets. ASM selects these data sets in a circular order within each type of data set, subject to the availability of free space and the device response time.

If you are using storage-class memory (SCM), ASM selects paging space first from available contiguous blocks of SCM, and then from available noncontiguous blocks of SCM.

Q:

What factors should I consider when allocating storage-class memory (SCM) to a partition?

A:

1. Continue to define page data sets on DASD, which provides improved availability compared to failure scenarios that could consume all of your paging space.
2. Configure approximately the same amount of paging space for storage-class memory (SCM) on Flash Express cards or the Virtual Flash Memory (VFM) feature as you have defined for page data sets on DASD. For many configurations, a single pair of Flash Express cards or the Virtual Flash Memory (VFM) feature provides enough paging space for an entire z/OS partition.
3. Using 1 MB pageable large pages with SCM can improve system performance by paging a smaller number of larger pages to SCM than would be paged to 4 KB page data sets on DASD. If contiguous space is not available on SCM, 1 MB large pages are demoted to 256 4 KB blocks and paged to either 4 KB page data sets or to SCM, based on response time.
4. Because SCM is not persistent across IPLs, PLPA data is also required for warm starts. The PLPA copy on page data sets is used for warm starts, and the PLPA copy on SCM is used for resolving page faults. In addition, local page data sets must accommodate all VIO paging.
5. For additional SCM configuration options, refer to “Space calculation examples” on page 71, “Estimating total size of paging data sets” on page 75 and the IEASYSxx system parameter list in *z/OS MVS Initialization and Tuning Reference*.

Q:

Will data-in-virtual users increase the need for paging data sets?

A:

Data-in-virtual does provide applications with functions that would encourage extensive use of virtual storage. Depending on the extent of the usage of data-in-virtual, paging data set requirements may increase.

Chapter 3. The system resources manager

To a large degree, an installation's control over the performance of the system is exercised through the system resources manager (SRM).

"Section 1: Description of the system resources manager (SRM)" on page 79 discusses the types of control available through SRM, the functions used to implement these controls, and the concepts inherent in the use of SRM parameters. The parameters themselves are described in *z/OS MVS Initialization and Tuning Reference*.

"Section 2: Basic SRM parameter concepts" on page 87 discusses some basic OPT parameters. *z/OS MVS Initialization and Tuning Reference* provides descriptions of the OPT parameters and syntax rules.

"Section 3: Advanced SRM parameter concepts" on page 89 discusses some more advanced topics.

"Section 4: Guidelines" on page 90 provides some guidelines for defining installation requirements and preparing an initial OPT.

"Section 5: Installation management controls" on page 93 contains information about commands for SRM-related functions.

System tuning and SRM

The task of tuning a system is an iterative and continuous process. The controls offered by SRM and workload management (WLM) are only one aspect of this process. Initial tuning consists of selecting appropriate parameters for various system components and subsystems. Once the system is operational and criteria have been established for the selection of work for execution, SRM and WLM control the distribution of available resources according to the parameters specified by the installation.

However SRM and WLM, can only deal with available resources. If these are inadequate to meet the needs of the installation, even optimal distribution may not be the answer — other areas of the system should be examined to determine the possibility of increasing available resources. Therefore, installations must perform regular capacity planning as outlined by the information available at IBM Techdocs (www.ibm.com/support/techdocs/atsmastr.nsf/Web/TechDocs).

When requirements for the system increase and it becomes necessary to shift priorities or acquire additional resources, such as a larger processor, more storage, or more terminals, the SRM and WLM parameters might have to be adjusted to reflect changed conditions.

Section 1: Description of the system resources manager (SRM)

SRM is a component of the system control program. It determines which address spaces, of all active address spaces, should be given access to system resources and the rate at which each address space is allowed to consume these resources.

Before an installation turns to SRM, it should be aware of the response time and throughput requirements for the various types of work that will be performed on its system. Questions similar to the following should be considered:

- How important is turnaround time for batch work, and are there distinct types of batch work with differing turnaround requirements?
- Should subsystems such as IMS and CICS be controlled at all, or should they receive as much service as they request? That is, should they be allowed unlimited access to resources without regard to the impact this would have on other types of work?
- What is acceptable TSO/E response time for various types of commands?
- What is acceptable response time for compiles, sorts, or other batch-like work executed from a terminal?

Guidelines for defining installation requirements are discussed in “Section 4: Guidelines” on page 90.

Once these questions have been answered and, whenever possible, quantified, and the installation is reasonably confident that its requirements do not exceed the physical capacity of its hardware, it should then turn to SRM to specify the desired degree of control.

Controlling SRM

You can control the system resources manager (SRM) through workload management (WLM).

With WLM, you specify performance goals for work, and SRM adapts the system resources to meet the goals.

While most information about how to use WLM is in *z/OS MVS Planning: Workload Management*, many of the concepts that SRM uses are explained in this publication, including the IEAOPTxx parameters.

Objectives

SRM bases its decision on two fundamental objectives:

1. To distribute system resources among individual address spaces in accordance with the installation's response, turnaround, and work priority requirements.
2. To achieve optimal use of system resources as seen from the viewpoint of system throughput.

SRM attempts to ensure optimal use of system resources by periodically monitoring and balancing resource utilization. If resources are under-utilized, SRM will attempt to increase the system load. If resources are overutilized, SRM will attempt to reduce the system load.

Types of control

SRM offers three distinct types of control to an installation:

- Service classes
- Dispatching control, for all address spaces
- Period

SRM sets the values of controls dynamically based on the performance goals for work defined in a service policy. The remainder of this section describes these types of controls and the functions that SRM uses to implement them.

Dispatching control

Dispatching priorities control the rate at which address spaces are allowed to consume resources after they have been given access to these resources. This form of competition takes place outside the sphere of domain control, that is, all address spaces compete with all other address spaces with regard to dispatching priorities.

Functions

This topic discusses the functions used by SRM to implement the controls described in “Dispatching control” on page 80. The functions are as follows:

- Swapping (see “Swapping” on page 81)
- Dispatching of work (see “Dispatching of work” on page 82)
- Resource use functions (see “Resource use functions” on page 82)
- Enqueue delay minimization (see “Enqueue delay minimization” on page 84)
- I/O priority queueing (see “I/O priority queueing” on page 84)
- DASD device allocation (see “DASD device allocation” on page 84)

- Prevention of storage shortages (see “Prevention of storage shortages” on page 85)
- Pageable frame stealing (see “Pageable frame stealing” on page 86).

Swapping

Swapping is the primary function used by SRM to exercise control over distribution of resources and system throughput. Using system status information that is periodically monitored, SRM determines which address spaces should have access to system resources.

In addition to the swapping controls described in the following text, SRM also provides an optional swap-in delay to limit the response time of TSO/E transactions.

There are several reasons for swapping. Some swaps are used for control of domains and the competition for resources between individual address spaces within a domain, while others provide control over system-wide performance and help increase the throughput.

Domain-related swaps

- *Unilateral swap in:* If the number of a domain's address spaces that are in the multiprogramming set (MPS) is less than the number SRM has set for the swap-in target, SRM swaps in additional address spaces for that domain, if possible.
- *Unilateral swap out:* If the number of a domain's address spaces that are in the multiprogramming set is greater than the number SRM has set for the swap-out target, SRM swaps out address spaces from that domain.
- *Exchange swap:* All address spaces of a domain compete with one another for system resources. When an address space in the multiprogramming set has exceeded its allotted portion of resources, relative to an address space of the same domain waiting to be swapped in, SRM performs an exchange swap. That is, the address space in the multiprogramming set is swapped out and the other address space is swapped in. (The multiprogramming set consists of those address spaces that are in central storage and are eligible for access to the processor.) This competition between address spaces is described in detail in “Section 2: Basic SRM parameter concepts” on page 87.

System-related swaps

- *Swaps due to storage shortages:* Two types of shortages cause swaps: auxiliary storage shortages and pageable frame shortages. If the number of available auxiliary storage slots is low, SRM will swap out the address space that is acquiring auxiliary storage at the fastest rate. For a shortage of pageable frames, if the number of fixed frames is very high, SRM will swap out the address space that acquired the greatest number of fixed frames. This process continues until the number of available slots rises above a fixed target, or until the number of fixed frames falls below a fixed target.
- *Swaps to improve central storage usage:* The system will swap out an address space when the system determines that the current mix of address spaces is not best utilizing central storage. The system swaps out address spaces to create a positive effect on system paging and swap costs.
- *Swap out an address space to make room for an address space:* The system will swap in an address space when the system determines that it has been out longer than its recommendation value would dictate. See “Working set management” on page 83 for information about the recommendation value.
- *Swaps due to wait states:* In certain cases, such as a batch job going into a long wait state (LONG option specified on the WAIT SVC, an STIMER wait specification of greater than or equal to 0.5 seconds, an ENQ for a resource held by a swapped out user), the address space will itself signal SRM to be swapped out in order to release storage for the use of other address spaces. Another example would be a time sharing user's address space that is waiting for input from the terminal after a transaction has completed processing. SRM also detects address spaces in a wait state. That is, address spaces in central storage that are not executable for a fixed interval will be swapped. (See “Logical swapping” on page 83.)
- *Request Swap:* The system may request that an address space be swapped out. For example, the CONFIG STOR, OFFLINE command requests the swap out of address spaces that occupy frames in the storage unit to be taken offline.

- *Transition Swap:* A transition swap occurs when the status of an address space changes from swappable to nonswappable. For example, the system performs a transition swap out before a nonswappable program or V=R step gets control. This special swap prevents the job step from improperly using reconfigurable storage.

Swap recommendation value

SRM calculates a swap recommendation value to determine which address spaces to use in an exchange or unilateral swap. A high swap recommendation value indicates that the address space is more likely to be swapped in. As the swap recommendation value decreases, that address space is more likely to be swapped out.

The swap recommendation value for an address space that is swapped in ranges from 100 to -999 as service is accumulated. An address space must accumulate enough CPU service to justify the cost of a swap out. Once the swap recommendation value goes below 0, the address space is ready to be swapped out in an exchange swap.

The swap recommendation value for an address space that is swapped out and ready to come in ranges from 0 to 998 as the address space remains out. Once the swap recommendation value goes above 100, the address space has been out long enough to justify the cost of the exchange swap.

For an address space that is swapped out but not ready to come in, the swap recommendation value as reported by the RMF Monitor II ASD report is meaningless. The swap recommendation value for an address space that is out too long is reported as 999. Also, if an address space has been assigned long-term storage protection (as described in the "Storage Protection" section of the "Workload Management Participants" chapter in *z/OS MVS Planning: Workload Management*), then the swap recommendation value is 999.

For monitored address spaces, SRM calculates a working set manager recommendation value. See "Working set management" on page 83 for information about the working set manager recommendation value.

Dispatching of work

Dispatching of work is done on a priority basis. That is, the ready work with the highest priority is dispatched first. The total range of priorities is from 191 to 255.

Note: Certain system address spaces execute at the highest priority and are exempt from installation prioritization decisions.

Resource use functions

The resource use functions of SRM attempt to optimize the use of system resources on a system-wide basis, rather than on an individual address space basis. The functions are as follows:

- Logical swapping - SRM automatically performs logical swapping when sufficient central storage is available.
- Working set management - SRM automatically determines the best mix of work in the multiprogramming set (MPS) and the most productive amount of central storage to allocate to each address space.

Multiprogramming level adjusting

SRM monitors system-wide utilization of resources, such as the CPU and paging subsystem, and seeks to alleviate imbalances, that is, over-utilization or under-utilization. This is accomplished by periodically adjusting the number of address spaces that are allowed in central storage and ready to be dispatched for appropriate domains (multiprogramming set).

When system contention factors indicate that the system is not being fully utilized, SRM will select a domain and increase the number of address spaces allowed access to the processor for that domain, thereby increasing utilization of the system.

Logical swapping

To use central storage more effectively and reduce processor and channel subsystem overhead, the SRM logical swap function attempts to prevent the automatic physical swapping of address spaces. Unlike a physically swapped address space, where the LSQA, fixed frames, and recently referenced frames are placed on auxiliary storage, SRM keeps the frames that belong to a logically swapped address space in central storage.

Address spaces swapped for wait states (for example, TSO/E terminal waits) are eligible to be logically swapped out whenever the think time associated with the address space is less than the system threshold value. SRM adjusts this threshold value according to the demand for central storage. SRM uses the unreferenced interval count (UIC) to measure this demand for central storage. As the demand for central storage increases, SRM reduces the system threshold value; as the demand decreases, SRM increases the system threshold value.

The system threshold value for think time fluctuates between low and high boundary values. The installation can change these boundary values in the IEAOPTxx parmlib member. The installation can also set threshold values for the UIC; setting these threshold values affects how SRM measures the demand for central storage.

SRM logically swaps address spaces if the real frames they own are not required to immediately replenish the supply of available real frames. Any address space subject to a swap out can become logically swapped as long as there is enough room in the system. Address spaces with pending request swaps or those marked as swaps due to storage shortages are the only address spaces that are physically swapped immediately.

Large address spaces that have been selected to be swapped to replenish central storage are trimmed before they are swapped. The trimming is done in stages and only to the degree necessary for the address space to be swapped. In some cases, it might be necessary to trim pages that have been recently referenced in order to reduce the address space to a swappable size.

SRM's logical swapping function periodically checks all logically swapped out address spaces to determine how long they've been logically swapped out. SRM physically swaps out those address spaces that have been logically swapped out for a period greater than the system threshold value for think time only when it is necessary to replenish the supply of available frames.

Working set management

SRM automatically determines the best mix of work in the multiprogramming set (MPS) and the most productive amount of central storage to allocate to each address space within MPL constraints.

To achieve this, SRM monitors the system paging, page movement, and swapping rates and productive CPU service for all address spaces to detect when the system might run more efficiently with selected address space working sets managed individually. If the system is spending a significant amount of resources for paging, SRM will start monitoring the central storage of selected address spaces.

After SRM decides that an address space should be monitored, SRM collects additional address space data. Based on this data, SRM might discontinue implicit block paging.

For monitored address spaces, SRM calculates a working set manager recommendation value that can override the swap recommendation value. See “Swap recommendation value” on page 82 for information about the swap recommendation value. The working set manager recommendation value measures the value of adding the address space to the current mix of work in the system. Even when the swap recommendation value indicates that a specific address space should be swapped in next, the working set manager recommendation value might indicate that the address space should be bypassed. To ensure that no address space is repeatedly bypassed, the system swaps in a TSO/E user 30 seconds after being bypassed. For all other types of address spaces, the system will swap in the address space 10 minutes after being bypassed.

If a monitored address space is paging heavily, SRM might manage its central storage usage. When an address space is managed, SRM imposes a central storage target (implicit dynamic central storage isolation maximum working set) on an address space.

Enqueue delay minimization

This function deals with the treatment of address spaces enqueued upon system resources that are in demand by other address spaces or resources for which a RESERVE has been issued and the device is shared. If an address space controlling an enqueued resource is swapped out and that resource is required by another address space, SRM will ensure that the holder of the resource is swapped in again as soon as possible.

Once in central storage, a swap out of the controlling address space would increase the duration of the enqueue bottleneck. Therefore, the controlling address space is given a period of CPU service during which it will not be swapped due to service considerations (discussed in "Section 2: Basic SRM parameter concepts" on page 87.) The length of this period is specified by the installation by means of a tuning parameter called the enqueue residence value (ERV), contained in parmlib member IEAOPTxx.

I/O priority queueing

I/O priority queueing is used to control deferred I/O requests. If this function is invoked, all deferred I/O requests, except paging and swapping, will be queued according to the I/O priorities associated with the requesting address spaces. Paging and swapping are always handled at the highest priority. An address space's I/O priority is by default the same as its dispatching priority. All address spaces in one mean-time-to-wait group fall into one I/O priority. In addition, address spaces that are time sliced have their I/O queued at their time slice priority. Changes to an address space's dispatching priority when the address space is time sliced up or down, do not affect the I/O priority.

An installation can assign an I/O priority that is higher or lower than the dispatching priority for selected groups of work. For example, if the installation is satisfied with the dispatching priority of an interactive application but would like the application's I/O requests to be processed before those of other address spaces executing at the same priority, the application could be given an I/O priority higher than its dispatching priority.

If I/O priority queueing is not invoked, all I/O requests are handled in a first-in/first-out (FIFO) manner.

DASD device allocation

Device allocation selects the most responsive DASD devices as candidates for permanent data sets on mountable devices (JCL specifies nonspecific VOLUME information or a specific volume and the volume is not mounted).

The ability of SRM to control DASD device allocation is limited by the decision an installation makes at system installation time and at initial program loading (IPL) time, as well as by the user's JCL parameters. SRM can only apply its selection rules to a set of DASD devices that are equally acceptable for scheduler allocation. This set of devices does not necessarily include all the DASD devices placed in an esoteric group during system installation. At that time, an esoteric group is defined by the UNITNAME macro and entered in the eligible device table (EDT). During system installation each esoteric group is partitioned into subgroups if either of the following conditions occurs:

- The group includes DASD devices that are common with another esoteric group.
- The group includes DASD devices that have certain generic differences. System installation partitions only esoteric groups that consist of magnetic tape or direct access devices.

For example, assume that you specify the following at system installation time:

```
UNITNAME=DASD,UNIT=((470,7),(478,8),(580,6))
UNITNAME=SYSDA,UNIT=((580,6))
```

Because of the intersection with SYSDA (580,6), the DASD group is divided into two subgroups: (470,7) and (478,8) in one subgroup and (580,6) in the other.

Allocation allows SRM to select from only one subgroup at a time. After allocating all devices in the first subgroup, allocation selects DASD devices from the next subgroup. Using the previous example, when a job requests UNIT=DASD, allocation tells SRM to select a device from the first group (470-476 and

478-47F) regardless of the relative use of channel paths 4 and 5. After all of the DASD devices in the first group have been allocated, allocation tells SRM to select devices from the second group (580-585).

Prevention of storage shortages

SRM periodically monitors the availability of three types of storage and attempts to prevent shortages from becoming critical. The three types of storage are:

- Auxiliary storage
- SQA
- Pageable frames.

Auxiliary storage

When more than a fixed percentage (constant MCCASMT1) of auxiliary storage slots have been allocated, SRM reduces demand for this resource by taking the following steps:

- LOGON, MOUNT and START commands are inhibited until the shortage is alleviated.
- Initiators are prevented from executing new jobs.
- The target MPL (both in and out targets) in each domain is set to its minimum value.
- The operator is informed of the shortage.
- Choosing from a subset of swappable address spaces, SRM stops the address space(s) acquiring slots at the fastest rate and prepares the address space for swap-out (logical swap). When SRM swaps-out an address space because of excessive slot usage, SRM informs the operator of the name of the job that is swapped out, permitting the operator to cancel the job.

If the percentage of auxiliary slots allocated continues to increase (constant MCCASMT2), SRM informs the operator that a critical shortage exists. SRM then prevents all unilateral swap-ins (except for domain zero). This action allows the operator to cancel jobs or add auxiliary paging space to alleviate the problem.

When the shortage has been alleviated, the operator is informed and SRM halts its efforts to reduce the demand for auxiliary storage.

SQA

When the number of available SQA and CSA pages falls below a threshold, SRM:

- Inhibits LOGON, MOUNT, and START commands until the shortage is alleviated.
- Informs the operator that an SQA shortage exists.

If the number of available SQA and CSA pages continues to decrease, SRM informs the operator that a critical shortage of SQA space exists and, except for domain zero, SRM prevents all unilateral swap-ins.

When the shortage has been alleviated, the operator is informed and SRM halts its efforts to prevent acquisition of SQA space.

Pageable frames

SRM attempts to ensure that enough pageable central storage is available to the system. SRM monitors the amount of pageable storage available, ensures that the currently available pageable storage is greater than a threshold, and takes continuous preventive action from the time it detects a shortage of pageable storage until the shortage is relieved.

When SRM detects a shortage of pageable frames caused by an excess of fixed or DREF storage, SRM uses event code ENVPC055 to signal an ENF event. When the shortage is relieved, SRM signals another ENVPC055 event to notify listeners that the shortage is relieved. SRM does not raise the signal for the “shortage relieved” condition until a delay of 30 seconds following the most recent occurrence of a fixed-storage shortage. The intent of signalling this event is to give system components and subsystems that use fixed or DREF storage an opportunity to help relieve the shortage.

Regardless of the cause of a shortage of pageable storage, SRM takes these actions:

- Inhibits LOGON, MOUNT, and START commands until the shortage is relieved
- Prevents initiators from executing new jobs
- Informs the operator that a shortage of pageable storage exists

Further SRM actions to relieve the shortage depend on the particular cause of the shortage.

The following system conditions can cause a shortage of pageable storage:

- Too many address spaces are already in storage.

  Too many address spaces in storage does not usually, of itself, cause a shortage of pageable storage because SRM performs MPL adjustment and logical swap threshold adjustment, which generally keep an adequate amount of fixed storage available to back the address spaces.
- Too much page fixing is taking place.

  One or more address spaces are using substantial amounts of storage either through explicit requests to fixed virtual storage or by obtaining virtual storage that is page fixed by attributes such as LSQA or SQA.

There are different types of pageable storage shortages:

- A shortage of pageable storage below 16 megabytes.
- A shortage when pageable storage has reached a threshold.
- A shortage when fixed and DREF allocated to CASTOUT=NO ESO Hiperspaces has reached a threshold.

If too much page fixing is the cause of the shortage of pageable storage, SRM:

1. Identifies the largest swappable user or users of fixed storage. If any of these users own more frames than three times the median fixed frame count, SRM begins to physically swap them out, starting with the user with the largest number of fixed pages. SRM continues to swap users out until it releases enough fixed storage to relieve the shortage.
2. Begins to physically swap out the logically-swapped out address spaces if swapping out the largest users of fixed storage does not relieve the shortage of pageable storage.
3. Decreases the MPLs for those domains that have the lowest contention indices if physically swapping out the logically-swapped out address spaces does not relieve the shortage of pageable storage. This MPL adjustment allows the swap analysis function of SRM to swap out enough address spaces to relieve the shortage.

SRM takes the following additional actions if the shortage of pageable storage reaches a critical threshold:

1. Informs the operator that there is a critical shortage.
2. Repeats all the steps described above that are applicable to the cause of the shortage.
3. Prevents any unilateral swap-in, except for domain zero.

When the shortage of pageable storage is relieved, SRM waits for a delay of 30 seconds following the most recent occurrence of a fixed shortage, and then:

- Allows new address spaces to be created through the LOGON, MOUNT, and START commands.
- Notifies the operator that the shortage is relieved.
- Allows initiators to process new jobs.
- Allows unilateral swap-ins.

Note: Those address spaces that SRM swapped out to relieve the shortage of pageable storage are not swapped back in if their storage requirements would potentially cause another shortage to occur.

Pageable frame stealing

Pageable frame stealing is the process of taking an assigned central storage frame away from an address space to make it available for other purposes, such as to satisfy a page fault or swap in an address space.

When there is a demand for pageable frames, SRM will steal those frames that have gone unreferenced for a long time and return them to the system. The unreferenced interval count (UIC) represents the time in seconds for a complete steal cycle. A complete steal cycle is the time the stealing routine needs to check all frames in the system. When there is a demand for storage, the stealing routine:

- tests the reference bit of a frame
- decides whether to steal the frame
- schedules the page-out.

When there is no demand for storage, no stealing occurs.

The UIC algorithm forecasts the UIC, based on the current stealing rate. The UIC can vary between 0 and 65535 and gets calculated every second. When there is no demand for storage in the system (no stealing occurs) the system has a UIC of 65535. If there is a very high demand for storage in the system, the system has a UIC close to 0.

Stealing takes place strictly on a demand basis, that is, there is no periodic stealing of long-unreferenced frames. A complete steal cycle can take days.

SRM modifies the stealing process for address spaces that it is managing and for address spaces that are storage critical. For these address spaces, SRM attempts to enforce the address space's real storage target that was set when SRM decided that the address space was to be managed.

I/O service units

The number of *I/O service units* is a measurement of individual data set I/O activity and JES spool reads and writes for all data sets associated with an address space. SRM calculates I/O service using I/O block (EXCP) counts.

When an address space executes in cross-memory mode (that is, during either secondary addressing mode or a cross-memory call), the EXCP counts are included in the I/O service total. This I/O service is not counted for the address space that is the target of the cross-memory reference.

Section 2: Basic SRM parameter concepts

This section discusses the OPT parameters for these basic SRM specifications:

- MPL adjustment control
- Transaction definition for CLISTs
- Directed VIO activity
- Alternate wait management

For explanations of the OPT parameters, see *z/OS MVS Initialization and Tuning Reference*.

See “Section 3: Advanced SRM parameter concepts” on page 89 for information about advanced OPT concepts.

MPL adjustment control

The OPT provides keywords to specify upper and lower thresholds for the variables that SRM uses to determine if it should increase, decrease, or leave the MPL unchanged. When one of these variables exceeds its threshold value, SRM regards this change as a signal to adjust the MPL. Table 17 on page 88 summarizes the internal names for the control variables, their thresholds, and conditions that can influence a change.

Table 17. Summary of MPL adjustment control

Control variable and internal name	Thresholds that can influence an MPL change:		Keyword in OPT
	decrease	increase	
CPU utilization (RCVCPUA)	>RCCCPUTH	<RCCCPUTL	RCCCPUT
Page fault rate (RCVPTR)	>RCCPTRTH	<RCCPTRTL	RCCPTRT (see note)
UIC (RCVUICA)	<RCCUICTL	>RCCUICTH	RCCUICT
Percentage of online storage fixed (RCVFXIOP)	>RCCFXTTH	<RCCFXTTL	RCCFXTT
Percentage of storage that is fixed within the first 16 megabytes (RCVMFXA)	>RCCFXETH	<RCCFXETL	RCCFXET

Note: The default thresholds for this keyword causes the corresponding control variable to have no effect on MPL adjustment.

Transaction definition for CLISTs

An installation can specify whether the individual commands in a TSO/E CLIST are treated as separate TSO/E commands for transaction control. Specifying CNTCLIST=YES causes a new transaction to be started for each command in the CLIST. A possible exposure of specifying CNTCLIST=YES is that long CLISTs composed of trivial and intermediate commands might monopolize a domain's MPL slots and cause interactive terminal users to be delayed. Specifying CNTCLIST=NO (the default) causes the entire CLIST to constitute a single transaction.

Directed VIO activity

VIO data set pages can be directed to a subset of the local paging data sets through *directed VIO*, which allows the installation to direct VIO activity away from selected local paging data sets that will be used only for non-VIO paging. With directed VIO, faster paging devices can be reserved for paging where good response time is important. The NONVIO system parameter, with the PAGE system parameter, allows the installation to define those local paging data sets that are not to be used for VIO, leaving the rest available for VIO activity. However, if space is depleted on the paging data sets made available for VIO paging, the non-VIO paging data sets will be used for VIO paging.

The installation uses the DVIO keyword to either activate or deactivate directed VIO.

Note: The NONVIO and PAGE system parameters are in the IEASYSxx parmlib member.

Alternate wait management

An installation can specify whether to activate or deactivate alternate wait management (AWM). If AWM is activated, SRM and LPAR cooperate to reduce low utilization effects and overhead.

For HIPERDISPATCH=NO (the default value), specifying CCCAWMT with any value in the range 1 to 499999 makes AWM active. Specifying CCCAWMT with any value in the range of 500000 to 1000000 makes AWM inactive. AWM is active or inactive only for any general purpose processor (CP) and IBM z Integrated Information Processor (zIIP). The default is 12000 (when AWM is active) for all CPU types.

For HIPERDISPATCH=YES (the default), the valid range for CCCAWMT is 1600 to 3200. For ZIIPAWMT, the valid range is 1600 to 499999. Any other value will be set to the default of 3200. Note that AWM cannot be turned off.

Dispatching mode control

An installation can switch between the HiperDispatch mode enabled or HiperDispatch mode disabled by specifying the HIPERDISPATCH keyword for the parmlib member IEAOPT.

For more information about the HIPERDISPATCH parameter, see *z/OS MVS Initialization and Tuning Reference*.

Section 3: Advanced SRM parameter concepts

This section includes information about selective enablement for I/O and adjustment of constants options.

Selective enablement for I/O

Selective enablement for I/O is a function that SRM uses to control the number of processors that are enabled for I/O interruptions. The intent of this function is to enable only the minimum number of processors needed to handle the I/O interruption activity without the system incurring excessive delays.

SRM enables at least two processors for I/O interruptions, if available. That is, if two processors can process the I/O interruptions without excessive delays, then only two processors need to be enabled for I/O interruptions. To determine if a change should be made to the number of processors that are enabled, SRM periodically monitors I/O interruptions.

By comparing this value to threshold values, SRM determines if another processor should be enabled or if an enabled processor should be disabled for I/O interruptions. If the computed value exceeds the upper threshold, I/O interruptions are being delayed, and another processor (if available) will be enabled for I/O interruptions. If the value is less than the lower threshold (and more than one processor is enabled), a processor will be disabled for I/O interruptions. The installation can change the threshold values using the CPENABLE parameter in the IEAOPTxx parmlib member.

In addition to enabling a processor when I/O activity requires it, SRM also enables another processor for I/O interruptions if one of the following occurs:

- An enabled processor is taken offline.
- An enabled processor has a hardware failure.
- SRM detects that no I/O interruptions have been taken for a predetermined period of time and concludes that the enabled processor is unable to accept interrupts.

An installation can use the CPENABLE keyword to specify low and high thresholds for the percentage of I/O interruptions to be processed through the test pending interrupt (TPI) instruction. SRM uses these thresholds to determine if a change should be made to the number of processors enabled for I/O interruptions. Instead of specifying the low and high thresholds explicitly, an installation can also use CPENABLE=SYSTEM, which applies low and high threshold values recommended by IBM. You can find more information about the CPENABLE setting in CPENABLE Recommendations for IBM Z and z/OS (www.ibm.com/support/pages/node/6353717) on IBM Techdocs.

The following chart gives the internal names of the control variables and indicates their relation to the condition.

Table 18. Summary of variables used to determine if changes are needed to the number of processors enabled for I/O interruptions

Control variable and internal name	Percentage of I/O Interruptions		Keyword in OPT
	under	over	
Percentage of I/O interruptions through TPI instruction (ICVTPIP)	< ICCTPILO	> ICCTPIHI	CPENABLE

Adjustment of constants options

Certain OPT parameters make it more convenient for installations with unique resource management requirements to change some SRM constants. The defaults provided are adequate for most installations.

A parameter needs to be specified only when its default is found to be unsuitable for a particular system environment. The following functions can be modified by parameters in the OPT:

- Enqueue residence control
- SRM invocation interval control
- Pageable storage control
- Central storage control

Enqueue residence control

This parameter, specified by the ERV keyword, defines the amount of CPU service that the address space is allowed to receive before it is considered for a workload recommendation swap out. The parameter applies to all swapped-in address spaces that are enqueued on a resource needed by another user. For more information, see "Enqueue delay minimization" on page 84.

SRM invocation interval control

This parameter, specified by the RMPTTOM keyword, controls the invocation interval for SRM timed algorithms. Increasing this parameter above the default value reduces the overhead caused by SRM algorithms, such as swap analysis and time slicing. However, when these algorithms are invoked at a rate less than the default, the accuracy of the data on which SRM decisions are made, and thus the decisions themselves, might be affected.

Pageable storage control

Two keywords are provided in the OPT to signal a shortage of pageable storage. Keyword MCCFXTPR specifies the percentage of storage that is fixed. Keyword MCCFXEPR specifies the percentage of storage, within the first 16 MB, that needs to be fixed before SRM detects a shortage. Table 19 on page 90 summarizes these keywords.

Table 19. Keywords provided in OPT to single pageable storage shortage

Control variable and internal name	Shortage of pageable storage exists	Keyword in OPT
Percentage of storage that is fixed RCETOTFX	>MCCFXTPR	MCCFXTPR
Percentage of storage that is fixed within the first 16 megabytes (RCEBELFX)	>MCCFXEPR	MCCFXEPR
Note: These variables are actual frame counts rather than percentages. SRM multiplies the MCCFXTPR threshold by the amount of online storage and multiplies the MCCFXEPR threshold by the amount of storage eligible for fixing in order to arrive at the threshold frame counts that it uses to compare against the actual frame counts. If MCCFXEPR x (amount of storage eligible for fixing) is greater than MCCFXTPR x (amount of online storage), then the threshold frame counts that SRM uses to compare against the actual frame counts are set equal.		

Central storage control

This parameter, specified by the MCCAFCTH keyword, indicates the number of frames on the available frame queue when stealing begins and ends. The range of values on this keyword determines the block size that SRM uses for stealing. In order to get a block into central storage, the lower value of the range must be greater than the block size.

Section 4: Guidelines

This section provides some guidelines for these tasks:

- Defining installation requirements and objectives
- Preparing the initial OPT

Defining installation requirements

Before specifying any parameters to SRM, an installation must define response and throughput requirements for its various classification of work. Examples of specific questions that should be answered are listed in the following sections. The applicability of these questions will, of course, vary from installation to installation. In addition, an installation must perform a capacity planning review as outlined by information available at IBM Techdocs (www.ibm.com/support/techdocs/atsmastr.nsf/Web/TechDocs). All of these tasks should be done on a periodic basis.

Subsystems

- *How many subsystems will be active at any one time and what are they?*
- *For IMS, how many active regions will there be?*
- *Will the subsystem address space(s) be nonswappable?*
- *What is the desired response time and how will it be measured?*

Batch

- *What is the required batch throughput or turnaround for various job classes?*
- *How much service do batch jobs require, and what service rate is needed to meet the turnaround requirement?*
  - An RMF workload report or reduction of SMF data in type 5 or type 30 records provides the average service consumed by jobs of different classes. The following approximations can be made:
    - Short jobs use 2,000 service units or less.
    - Long jobs use more than 2,000 units.
- *What is the average number of ready jobs?*
  - Most likely, this is the number of active initiators. A few extra initiators may be started to decrease turnaround times.

TSO/E

- *What is the number of terminals?*
- *What is the average number of ready users?*
  - As a guideline for installations new to TSO/E, assume that an installation doing program development on 3270 terminals will have two ready users for every ten users logged on. This average will vary, depending on the type of terminal and on the type of TSO/E session (data entry, problem solving, program development).
- *What is the required response time and expected transaction rate for different categories of TSO/E transactions at different times, such as peak hours?*
- *What is the expected response time for short transactions?*
- *How will this response time be measured?*
- *Should response time be different for select groups of TSO/E users?*
- *How should semi-trivial and non-trivial transactions be treated?*
- *How are they defined?*
  - An installation can use RMF workload reports or SMF data in type 34 and 35 records available to help define trivial and non-trivial TSO/E work. The following approximations can be made:
    - Short TSO/E commands use 20 service units or less.
    - Medium length commands use 20 - 100 service units.
    - Long TSO/E commands use 100 service units or more.
- *What is the required service rate for TSO/E users?*

  - If 2-second response time (as reported by RMF) is required for very short TSO/E commands (100 service units), the required service rate for such a transaction is 100/2 or 50 service units per second. Service rates for other types of transactions should be computed also.

General

- *What is the importance level of TSO/E, batch, IMS, and special batch classes in relation to one another?*
- *Which may be delayed or "tuned down" to satisfy other requirements?*
  - In other words, which response requirements are fixed and which are variable?
- *What percentage of system resources should each group receive?*

Preparing an initial OPT

There are several approaches to preparing an initial OPT.

- Use the default OPT.
- Modify the default OPT.
- Create a new OPT.

Processor version codes and SRM constants

The SRM constants are used to make z/OS installation specifications (such as IEAOPTxx parameters) transparent across the IBM processor range. SRM, the system resources manager, is the component of the system control program that determines which address spaces should be given access to system resources and the rate at which each address space is allowed to consume resources. The amount of service consumed by an address space is computed by SRM according to the formula:

```
service = CPU service + SRB service
```

The *CPU service* units are the task (TCB) execution time multiplied by the *SRM constant* for the processor model (as partitioned, if applicable). Similarly, the *SRB service* units are the service request block (SRB) execution time multiplied by the *SRM constant* for the processor. Thus, the SRM constants table relates processor execution time with the CPU/SRB portion of service as calculated by SRM. (Calculations will need to allow for the possibility that some of the CPU/SRB service time goes uncaptured by the reporting software.)

For the latest information about the processor version codes and SRM constants, see Processor version codes and SRM constants (www.ibm.com/support/pages/processor-version-codes-and-srm-constants).

If you plan to use these constants for purposes other than those suggested in this information, observe the following limitations:

- Actual customer workloads and performance may vary. For a more exact comparison of processors, see the internal throughput rate (ITR) numbers in IBM Z Large Systems Performance Reference (www.ibm.com/support/pages/ibm-z-large-systems-performance-reference).
- CPU time can vary for different runs of the same job step. One or more of the following factors might cause variations in the CPU time: CPU architecture (such as storage buffering), cycle stealing with integrated channels, and the amount of the queue searching (see *z/OS MVS System Management Facilities (SMF)*).
- The constants do not account for multiprocessor effects within logical partitions. For example, a logical 10-way partition on a particular machine has 94674.6 service units per second, while a 20-way partition on the same machine has 82901.6 service units per second.

Using SMF task time

For installations with no prior service data, the task time reported in SMF record Type 4, 5, 30, 34, and 35 records can be converted to service units using the tables in Processor version codes and SRM constants (www.ibm.com/support/pages/processor-version-codes-and-srm-constants).

Section 5: Installation management controls

This section contains information about commands for SRM-related installation management functions.

Operator commands related to SRM

The system operator can directly influence SRM's control of specific jobs or groups of jobs by entering commands from the console. The exact formats of these commands are defined in *z/OS MVS System Commands*.

The SET command with the OPT parameter is used to switch to a different OPT after an IPL. SRM bases all control decisions for existing and future jobs on the parameters in the new parmlib member.

Note: For IEAOPTxx, only one member can be active at a time, and members are not concatenated when you activate a new member. For instance, if you want to change one of the parameters in the active IEAOPTxx member, you have two options:

- Update the parameter in the active IEAOPTxx member, and issue the SET OPT=*xx* command to activate the change.

  - or -

- If you prefer to keep the active IEAOPTxx member intact, copy the entire contents of the active IEAOPTxx member to a new IEAOPTyy member and make your change in the IEAOPTyy member. Then, issue the SET OPT=*yy* command to activate the new member. Note that any parameters that were specified in the original IEAOPTxx member that are not also specified in the new IEAOPTyy member might get reset to their default values.

Appendix A. Accessibility

Accessible publications for this product are offered through IBM Documentation for z/OS (www.ibm.com/docs/en/zos).

If you experience difficulty with the accessibility of any z/OS documentation see How to Send Feedback to IBM to leave documentation feedback.

Notices

This information was developed for products and services that are offered in the USA or elsewhere.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing
IBM Corporation
North Castle Drive, MD-NC119
Armonk, NY 10504-1785
United States of America

For license inquiries regarding double-byte character set (DBCS) information, contact the IBM Intellectual Property Department in your country or send inquiries, in writing, to:

Intellectual Property Licensing
Legal and Intellectual Property Law
IBM Japan Ltd.
19-21, Nihonbashi-Hakozakicho, Chuo-ku
Tokyo 103-8510, Japan

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

This information could include missing, incorrect, or broken hyperlinks. Hyperlinks are maintained in only the HTML plug-in output for IBM Documentation. Use of hyperlinks in other output formats of this information is at your own risk.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Licensees of this program who wish to have information about it for the purpose of enabling: (i) the exchange of information between independently created programs and other programs (including this one) and (ii) the mutual use of the information which has been exchanged, should contact:

IBM Corporation
Site Counsel
2455 South Road

Poughkeepsie, NY 12601-5400
USA

Such information may be available, subject to appropriate terms and conditions, including in some cases, payment of a fee.

The licensed program described in this document and all licensed material available for it are provided by IBM under terms of the IBM Customer Agreement, IBM International Program License Agreement or any equivalent agreement between us.

Any performance data contained herein was determined in a controlled environment. Therefore, the results obtained in other operating environments may vary significantly. Some measurements may have been made on development-level systems and there is no guarantee that these measurements will be the same on generally available systems. Furthermore, some measurements may have been estimated through extrapolation. Actual results may vary. Users of this document should verify the applicable data for their specific environment.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

All statements regarding IBM's future direction or intent are subject to change or withdrawal without notice, and represent goals and objectives only.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs. The sample programs are provided "AS IS", without warranty of any kind. IBM shall not be liable for any damages arising out of your use of the sample programs.

Terms and conditions for product documentation

Permissions for the use of these publications are granted subject to the following terms and conditions.

Applicability

These terms and conditions are in addition to any terms of use for the IBM website.

Personal use

You may reproduce these publications for your personal, noncommercial use provided that all proprietary notices are preserved. You may not distribute, display or make derivative work of these publications, or any portion thereof, without the express consent of IBM.

Commercial use

You may reproduce, distribute and display these publications solely within your enterprise provided that all proprietary notices are preserved. You may not make derivative works of these publications, or

reproduce, distribute or display these publications or any portion thereof outside your enterprise, without the express consent of IBM.

Rights

Except as expressly granted in this permission, no other permissions, licenses or rights are granted, either express or implied, to the publications or any information, data, software or other intellectual property contained therein.

IBM reserves the right to withdraw the permissions granted herein whenever, in its discretion, the use of the publications is detrimental to its interest or, as determined by IBM, the above instructions are not being properly followed.

You may not download, export or re-export this information except in full compliance with all applicable laws and regulations, including all United States export laws and regulations.

IBM MAKES NO GUARANTEE ABOUT THE CONTENT OF THESE PUBLICATIONS. THE PUBLICATIONS ARE PROVIDED "AS-IS" AND WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, AND FITNESS FOR A PARTICULAR PURPOSE.

IBM Online Privacy Statement

IBM Software products, including software as a service solutions, ("Software Offerings") may use cookies or other technologies to collect product usage information, to help improve the end user experience, to tailor interactions with the end user, or for other purposes. In many cases no personally identifiable information is collected by the Software Offerings. Some of our Software Offerings can help enable you to collect personally identifiable information. If this Software Offering uses cookies to collect personally identifiable information, specific information about this offering's use of cookies is set forth below.

Depending upon the configurations deployed, this Software Offering may use session cookies that collect each user's name, email address, phone number, or other personally identifiable information for purposes of enhanced user usability and single sign-on configuration. These cookies can be disabled, but disabling them will also eliminate the functionality they enable.

If the configurations deployed for this Software Offering provide you as customer the ability to collect personally identifiable information from end users via cookies and other technologies, you should seek your own legal advice about any laws applicable to such data collection, including any requirements for notice and consent.

For more information about the use of various technologies, including cookies, for these purposes, see IBM's Privacy Policy at ibm.com®/privacy and IBM's Online Privacy Statement at ibm.com/privacy/details in the section entitled "Cookies, Web Beacons and Other Technologies," and the "IBM Software Products and Software-as-a-Service Privacy Statement" at ibm.com/software/info/product-privacy.

Policy for unsupported hardware

Various z/OS elements, such as DFSMSdfp, JES2, and MVS, contain code that supports specific hardware servers or devices. In some cases, this device-related element support remains in the product even after the hardware devices pass their announced End of Service date. z/OS may continue to service element code; however, it will not provide service related to unsupported hardware devices. Software problems related to these devices will not be accepted for service, and current service activity will cease if a problem is determined to be associated with out-of-support devices. In such cases, fixes will not be issued.

Minimum supported hardware

The minimum supported hardware for z/OS releases identified in z/OS announcements can subsequently change when service for particular servers or devices is withdrawn. Likewise, the levels of other software products supported on a particular release of z/OS are subject to the service support lifecycle of those

products. Therefore, z/OS and its product publications (for example, panels, samples, messages, and product documentation) can include references to hardware and software that is no longer supported.

- For information about software support lifecycle, see: IBM Lifecycle Support for z/OS (www.ibm.com/software/support/systemsz/lifecycle)
- For information about currently-supported IBM hardware, contact your IBM representative.

Programming Interface Information

This book is intended to help the customer initialize and tune the MVS element of z/OS. This book documents information that is NOT intended to be used as Programming Interfaces of z/OS.

Trademarks

IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at Copyright and Trademark information (www.ibm.com/legal/copytrade.shtml).

Microsoft, Windows, Windows NT, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.

Java™ and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates.

The registered trademark Linux®® is used pursuant to a sublicense from the Linux Foundation, the exclusive licensee of Linus Torvalds, owner of the mark on a worldwide basis.

UNIX is a registered trademark of The Open Group in the United States and other countries.

Index

E

F

G

H

I

J

K

L

M

N

O

P

R

S

T

U

V

W

Product Number: 5655-ZOS

SA23-1379-60